

程序 1: problem1/__init__.py

```
1 """Problem 1: bearing-sector intersection and localization diameter."""
```

程序 2: problem1/solve.py

```
1 """Intersect bearing sectors, compute diameter, and test disk coverage.
2
3 Run from the repository: python -m problem1.solve
4 Run from anywhere: python /absolute/path/to/problem1/solve.py --demo
5
6 Only the Python standard library is required. Geometry predicates, intersections,
7 and circle comparisons use exact fractions of the supplied floating coefficients.
8 Trigonometric coefficients themselves have normal machine precision.
9 """
10
11 from __future__ import annotations
12
13 import argparse
14 from dataclasses import dataclass
15 from fractions import Fraction
16 import html
17 import itertools
18 import json
19 import math
20 from pathlib import Path
21 from typing import Iterable, Sequence
22
23
24 HERE = Path(__file__).resolve().parent
25 Number = int | float | Fraction
26 Point = tuple[Fraction, Fraction]
27
28
29 def rational(value: Number) -> Fraction:
30     """Accept finite real numbers; booleans are not measurements."""
31     if isinstance(value, bool) or not isinstance(value, (int, float, Fraction)):
32         raise ValueError(f"Expected a finite real number, got {value!r}")
33     if isinstance(value, float) and not math.isfinite(value):
34         raise ValueError("All coordinates and coefficients must be finite")
35     return Fraction(value)
36
37
38 @dataclass(frozen=True)
39 class Measurement:
40     x: float
41     y: float
42     bearing_deg: float
43     error_deg: float = 1.0
44
45     def __post_init__(self) -> None:
46         for value in (self.x, self.y, self.bearing_deg, self.error_deg):
47             rational(value)
48         if not 0 <= self.error_deg < 90:
49             raise ValueError("error_deg must be in [0, 90); the official value is 1")
```

```

50
51
52 @dataclass(frozen=True)
53 class HalfPlane:
54     """Closed half-plane  $a \cdot x + b \cdot y \leq c$ , with exact rational coefficients."""
55
56     a: Number
57     b: Number
58     c: Number
59
60     def __post_init__(self) -> None:
61         for name in ("a", "b", "c"):
62             object.__setattr__(self, name, rational(getattr(self, name)))
63
64     def contains(self, point: Point) -> bool:
65         return self.a * point[0] + self.b * point[1] <= self.c
66
67
68 def direction(angle_deg: float) -> Point:
69     """Angle is counterclockwise from +x; exact cardinal axes avoid  $\sin(\pi)$ ."""
70     angle = float(angle_deg) % 360.0
71     cardinal = {
72         0.0: (1, 0), 90.0: (0, 1), 180.0: (-1, 0), 270.0: (0, -1)
73     }
74     if angle in cardinal:
75         return tuple(Fraction(v) for v in cardinal[angle]) # type: ignore[return-value]
76     # Reduce around the nearest cardinal axis, so theta and theta+180 use
77     # exactly opposite stored coefficients and 359/1-degree rays are symmetric.
78     quadrant = int((angle + 45.0) // 90.0)
79     rad = math.radians(angle - 90.0 * quadrant)
80     cosine, sine = Fraction(math.cos(rad)), Fraction(math.sin(rad))
81     return ((cosine, sine), (-sine, cosine), (-cosine, -sine),
82            (sine, -cosine))[quadrant % 4]
83
84
85 def bearing_halfplanes(measurements: Iterable[Measurement]) -> list[HalfPlane]:
86     result = []
87     for measurement in measurements:
88         x, y = rational(measurement.x), rational(measurement.y)
89         theta = float(measurement.bearing_deg) % 360.0
90         lo = direction(theta - measurement.error_deg)
91         hi = direction(theta + measurement.error_deg)
92         forward = direction(theta)
93         # cross(lo, X-S) >= 0; cross(hi, X-S) <= 0.
94         # The forward constraint is redundant for  $0 < \text{error} < 90$  degrees;
95         # it ensures an error-zero measurement is a ray instead of a full line.
96         for a, b in ((lo[1], -lo[0]), (-hi[1], hi[0]),
97                    (-forward[0], -forward[1])):
98             result.append(HalfPlane(a, b, a * x + b * y))
99     return result
100
101
102 def cross(a: Point, b: Point, c: Point) -> Fraction:
103     return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])
104

```

```

105
106 def distance_squared(a: Point, b: Point) -> Fraction:
107     return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
108
109
110 def convex_hull(points: Iterable[Point]) -> list[Point]:
111     """Counterclockwise hull with redundant collinear points removed."""
112     points = sorted(set(points))
113     if len(points) <= 1:
114         return points
115     lower: list[Point] = []
116     upper: list[Point] = []
117     for point in points:
118         while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
119             lower.pop()
120         lower.append(point)
121     for point in reversed(points):
122         while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
123             upper.pop()
124         upper.append(point)
125     return lower[:-1] + upper[::-1]
126
127
128 def diameter_squared_bruteforce(points: Sequence[Point]) -> tuple[Fraction, tuple[Point, Point]]:
129     """Independent  $O(k^2)$  reference for checking the rotating-calipers result."""
130     if not points:
131         raise ValueError("The empty set has no chosen diameter")
132     best = Fraction(0)
133     pair = (points[0], points[0])
134     for first, second in itertools.combinations(points, 2):
135         squared = distance_squared(first, second)
136         candidate = tuple(sorted((first, second)))
137         if squared > best or (squared == best and candidate < pair):
138             best, pair = squared, candidate
139     return best, pair
140
141
142 def diameter_squared_calipers(polygon: Sequence[Point]) -> tuple[Fraction, tuple[Point, Point]]:
143     """Exact  $O(k)$  diameter on a CCW convex hull without redundant collinear points.
144
145     Adapted from the teammate's antipodal-pair approach, retaining exact rational
146     comparisons. Equal support areas mean parallel supporting edges: consider
147     both endpoints, rather than dropping a possible farthest pair.
148     """
149     if len(polygon) <= 2:
150         return diameter_squared_bruteforce(polygon)
151     count = len(polygon)
152     opposite = 1
153     best = Fraction(0)
154     pair = (polygon[0], polygon[0])
155
156     def consider(first_index: int, second_index: int) -> None:
157         nonlocal best, pair
158         first, second = polygon[first_index], polygon[second_index]
159         squared = distance_squared(first, second)

```

```

160         candidate = tuple(sorted((first, second)))
161         if squared > best or (squared == best and candidate < pair):
162             best, pair = squared, candidate
163
164     for index in range(count):
165         next_index = (index + 1) % count
166
167         def area(other: int) -> Fraction:
168             return cross(polygon[index], polygon[next_index], polygon[other])
169
170         while area((opposite + 1) % count) > area(opposite):
171             opposite = (opposite + 1) % count
172         consider(index, opposite)
173         consider(next_index, opposite)
174         if area((opposite + 1) % count) == area(opposite):
175             consider(index, (opposite + 1) % count)
176             consider(next_index, (opposite + 1) % count)
177     return best, pair
178
179
180 def polygon_area(polygon: Sequence[Point]) -> Fraction:
181     """Exact area; a point and a segment have zero area."""
182     if len(polygon) < 3:
183         return Fraction(0)
184     origin = polygon[0]
185     return abs(sum((cross(origin, polygon[index], polygon[index + 1])
186                     for index in range(1, len(polygon) - 1)), Fraction(0))) / 2
187
188
189 def intersection_vertices(planes: Sequence[HalfPlane]) -> list[Point]:
190     vertices: set[Point] = set()
191     for first, second in itertools.combinations(planes, 2):
192         det = first.a * second.b - second.a * first.b
193         if det == 0:
194             continue
195         point = ((first.c * second.b - second.c * first.b) / det,
196                 (first.a * second.c - second.a * first.c) / det)
197         if all(plane.contains(point) for plane in planes):
198             vertices.add(point)
199     return convex_hull(vertices)
200
201
202 def feasible_witness(planes: Sequence[HalfPlane], vertices: Sequence[Point]) -> Point | None:
203     """No artificial box: test vertices, origin, and boundary projections.
204
205     A nonempty closed convex polyhedron has a nearest point to the origin.
206     It is the origin, lies on a single supporting line as its projection, or
207     has two linearly independent active boundaries and hence is a vertex.
208     """
209     if vertices:
210         return vertices[0]
211     zero = (Fraction(0), Fraction(0))
212     if all(plane.contains(zero) for plane in planes):
213         return zero
214     for plane in planes:

```

```

215     norm_squared = plane.a ** 2 + plane.b ** 2
216     if norm_squared == 0:
217         continue
218     point = (plane.a * plane.c / norm_squared, plane.b * plane.c / norm_squared)
219     if all(other.contains(point) for other in planes):
220         return point
221     return None
222
223
224 def recession_direction(planes: Sequence[HalfPlane]) -> Point | None:
225     """Find d != 0 with A*d <= 0, iff a nonempty region is unbounded."""
226     nonzero = [plane for plane in planes if plane.a != 0 or plane.b != 0]
227     if not nonzero:
228         return Fraction(1), Fraction(0)
229     for plane in nonzero:
230         for point in ((-plane.b, plane.a), (plane.b, -plane.a)):
231             if all(other.a * point[0] + other.b * point[1] <= 0 for other in nonzero):
232                 return point
233     return None
234
235
236 @dataclass(frozen=True)
237 class Circle:
238     center: Point
239     radius_squared: Fraction
240     support: tuple[Point, ...]
241
242     def contains(self, point: Point) -> bool:
243         return distance_squared(self.center, point) <= self.radius_squared
244
245
246 def pair_circle(first: Point, second: Point) -> Circle:
247     center = ((first[0] + second[0]) / 2, (first[1] + second[1]) / 2)
248     return Circle(center, distance_squared(first, second) / 4, (first, second))
249
250
251 def triple_circle(first: Point, second: Point, third: Point) -> Circle | None:
252     # Work relative to the first point; the rational computation is exact.
253     ux, uy = second[0] - first[0], second[1] - first[1]
254     vx, vy = third[0] - first[0], third[1] - first[1]
255     det = 2 * (ux * vy - uy * vx)
256     if det == 0:
257         return None
258     uu, vv = ux * ux + uy * uy, vx * vx + vy * vy
259     center = (first[0] + (uu * vy - vv * uy) / det,
260              first[1] + (ux * vv - vx * uu) / det)
261     return Circle(center, distance_squared(center, first), (first, second, third))
262
263
264 def minimum_enclosing_circle(points: Sequence[Point]) -> Circle:
265     """Enumerate all 1/2/3-point support circles; exact and deterministic.
266
267     O(k^4) arithmetic operations in the worst case. Intended for the small
268     vertex counts produced by a few bearing measurements, not large point clouds.
269     """

```

```

270     if not points:
271         raise ValueError("The empty set has no chosen enclosing circle")
272     if len(points) == 1:
273         return Circle(points[0], Fraction(0), (points[0],))
274     best: Circle | None = None
275     for first, second in itertools.combinations(points, 2):
276         circle = pair_circle(first, second)
277         if (best is None or circle.radius_squared < best.radius_squared) and all(
278             circle.contains(point) for point in points):
279             best = circle
280     for first, second, third in itertools.combinations(points, 3):
281         circle = triple_circle(first, second, third)
282         if circle is not None and (best is None or circle.radius_squared < best.radius_squared):
283             if all(circle.contains(point) for point in points):
284                 best = circle
285     if best is None:
286         raise ArithmeticError("No enclosing support circle found")
287     return best
288
289
290 def float_point(point: Point) -> list[float]:
291     return [float(point[0]), float(point[1])]
292
293
294 def circle_record(circle: Circle) -> dict:
295     radius = math.sqrt(float(circle.radius_squared))
296     return {"center": float_point(circle.center), "radius_m": radius,
297           "diameter_m": 2 * radius,
298           "support_points": [float_point(point) for point in circle.support]}
299
300
301 def analyze_halfplanes(halfplanes: Iterable[HalfPlane]) -> dict:
302     """Public general half-plane API, also used for degenerate-case verification."""
303     planes = list(halfplanes)
304     vertices = intersection_vertices(planes)
305     witness = feasible_witness(planes, vertices)
306     result = {
307         "status": "empty", "vertices": [float_point(point) for point in vertices],
308         "feasible_point": None if witness is None else float_point(witness),
309         "diameter_m": None, "diameter_endpoints": None,
310         "diameter_circle": None, "diameter_circle_covers": None,
311         "minimum_enclosing_circle": None,
312         "area_m2": None, "max_distance_from_diameter_center_m": None,
313         "optical_localization": None,
314     }
315     if witness is None:
316         result["reason"] = "The closed bearing constraints have no common point."
317         return result
318     ray = recession_direction(planes)
319     if ray is not None:
320         # Keep the direction small in the serialized output; scaling is positive.
321         scale = max(abs(ray[0]), abs(ray[1]))
322         result.update(status="unbounded", diameter_is_infinite=True,
323                       recession_direction=float_point((ray[0] / scale, ray[1] / scale)),
324                       reason="A nonzero direction d satisfies A*d <= 0; no finite diameter exists.")

```

```

325         return result
326     if not vertices:
327         raise ArithmeticError("A nonempty bounded polyhedron must have a vertex")
328     result["status"] = "point" if len(vertices) == 1 else "segment" if len(vertices) == 2 else "polygon"
329     maximum_squared, (first, second) = diameter_squared_calipers(vertices)
330     diameter_circle = pair_circle(first, second)
331     minimum_circle = minimum_enclosing_circle(vertices)
332     covers = all(diameter_circle.contains(point) for point in vertices)
333     assert covers == (minimum_circle.radius_squared <= maximum_squared / 4)
334     result.update(
335         diameter_m=math.sqrt(float(maximum_squared)),
336         diameter_endpoints=[float_point(first), float_point(second)],
337         diameter_circle={**circle_record(diameter_circle), "covers_region": covers},
338         diameter_circle_covers=covers,
339         minimum_enclosing_circle=circle_record(minimum_circle),
340         area_m2=float(polygon_area(vertices)),
341         max_distance_from_diameter_center_m=math.sqrt(float(max(
342             distance_squared(diameter_circle.center, point) for point in vertices))),
343         optical_localization={
344             "radius_m": 20.0,
345             "guaranteed_from_enclosing_center": minimum_circle.radius_squared <= 400,
346             "radius_margin_m": 20.0 - math.sqrt(float(minimum_circle.radius_squared)),
347             "diameter_only_sufficient": maximum_squared <= 1200,
348             "diameter_only_necessary": maximum_squared <= 1600,
349             "scope": "Geometric coverage of the angular region; no travel or simulator action implied.",
350         },
351     )
352     return result
353
354
355 def localize(measurements: Iterable[Measurement]) -> dict:
356     measurements = list(measurements)
357     result = analyze_halfplanes(bearing_halfplanes(measurements))
358     result["measurement_count"] = len(measurements)
359     result["model"] = "Closed angular sectors only; no range or target-disk clipping."
360     result["angle_convention"] = "Degrees counterclockwise from the positive x-axis."
361     return result
362
363
364 def load_measurements(path: Path) -> tuple[dict, list[Measurement]]:
365     document = json.loads(path.read_text(encoding="utf-8-sig"))
366     if not isinstance(document, dict) or not isinstance(document.get("measurements"), list):
367         raise ValueError("Input must be an object containing a measurements list")
368     measurements = []
369     for index, row in enumerate(document["measurements"]):
370         if not isinstance(row, dict) or not {"x", "y", "bearing_deg"} <= row.keys():
371             raise ValueError(f"Measurement {index} requires x, y, bearing_deg")
372         measurements.append(Measurement(row["x"], row["y"], row["bearing_deg"], row.get("error_deg", 1.0)))
373     return document, measurements
374
375
376 def write_json(path: Path, value: dict) -> None:
377     path.parent.mkdir(parents=True, exist_ok=True)
378     path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
379         ↪ encoding="utf-8")

```

```

379
380
381 def write_region_svg(path: Path, result: dict, title: str) -> None:
382     """Write a standalone vector plot, focused on the localization region."""
383     vertices = result["vertices"]
384     circle = result["minimum_enclosing_circle"]
385     if circle is None:
386         return
387     cx, cy = circle["center"]
388     radius = max(circle["radius_m"], 0.5)
389     width, height = 860, 660
390     plot_size = 470
391     scale = plot_size / (2.5 * radius)
392     ox, oy = width / 2, 325
393
394     def screen(point: list[float]) -> tuple[float, float]:
395         return ox + (point[0] - cx) * scale, oy - (point[1] - cy) * scale
396
397     coordinates = " ".join(f"{x:.5f},{y:.5f}" for x, y in map(screen, vertices))
398     diameter = result["diameter_circle"]
399     dx, dy = screen(diameter["center"])
400     pieces = [
401         '<svg xmlns="http://www.w3.org/2000/svg" width="860" height="660" viewBox="0 0 860 660">',
402         '<rect width="860" height="660" fill="#f6f8fc"/>',
403         '<g font-family="Arial, sans-serif" fill="#172b4d">',
404         f'<text x="42" y="43" font-size="23" font-weight="700">{html.escape(title)}</text>',
405         '<text x="42" y="72" font-size="14">Self-created example; coordinates in metres</text>',
406         f'<circle cx="{ox}" cy="{oy}" r="{circle["radius_m"] * scale}" fill="none" stroke="#187c6c" '
407         ↪ 'stroke-width="2.5"/>',
408         f'<circle cx="{dx}" cy="{dy}" r="{diameter["radius_m"] * scale}" fill="none" stroke="#df7524" '
409         ↪ 'stroke-width="2" stroke-dasharray="7 5"/>',
410         f'<polygon points="{coordinates}" fill="#467fe633" stroke="#315a9c" stroke-width="2.5"/>',
411     ]
412
413     for index, vertex in enumerate(vertices):
414         x, y = screen(vertex)
415         pieces.append(f'<circle cx="{x}" cy="{y}" r="4" fill="#315a9c"/>')
416         pieces.append(f'<text x="{x + 9}" y="{y - 10}" font-size="14">V{index + 1}</text>')
417
418     values = [
419         f'Region diameter D = {result["diameter_m"]:.6f} m',
420         f'Minimum covering radius R = {circle["radius_m"]:.6f} m',
421         f'Diameter-D disk covers region: {str(result["diameter_circle_covers"]).upper()}',
422     ]
423
424     pieces.extend(f'<text x="42" y="{555 + index * 25}" font-size="16">{value}</text>' for index, value in
425         ↪ enumerate(values))
426
427     pieces.extend([
428         '<line x1="480" y1="557" x2="520" y2="557" stroke="#187c6c" stroke-width="3"/>',
429         '<text x="532" y="562" font-size="14">Minimum covering circle</text>',
430         '<line x1="480" y1="588" x2="520" y2="588" stroke="#df7524" stroke-width="3" stroke-dasharray="7 '
431         ↪ '5"/>',
432         '<text x="532" y="593" font-size="14">Circle with diameter D</text>',
433         '</g></svg>',
434     ])
435
436     path.parent.mkdir(parents=True, exist_ok=True)
437     path.write_text("\n".join(pieces) + "\n", encoding="utf-8")
438
439

```



```

430
431 def main() -> None:
432     parser = argparse.ArgumentParser(description=__doc__)
433     parser.add_argument("--input", type=Path, help="JSON input; relative paths are resolved against
↳ problem1/")
434     parser.add_argument("--output", type=Path, help="JSON output; relative paths are resolved against
↳ problem1/")
435     parser.add_argument("--demo", action="store_true", help="Run all self-created examples (also the
↳ default)")
436     args = parser.parse_args()
437     if args.demo and args.input:
438         parser.error("Choose --demo or --input, not both")
439     if args.output and not args.input:
440         parser.error("--output requires --input")
441     paths = [args.input if args.input.is_absolute() else HERE / args.input] if args.input else
↳ sorted((HERE / "examples").glob("*.json"))
442     for path in paths:
443         document, measurements = load_measurements(path)
444         result = localize(measurements)
445         result["example_name"] = document.get("name", path.stem)
446         result["data_origin"] = document.get("data_origin", "User-supplied input; no official benchmark
↳ implied.")
447         output = (args.output if args.output.is_absolute() else HERE / args.output) if args.output else
↳ HERE / "results" / f"{path.stem}.result.json"
448         write_json(output, result)
449         write_region_svg(output.with_suffix(".svg"), result, document.get("plot_title", path.stem))
450         print(f'{path.name}: status={result["status"]}, diameter={result["diameter_m"]},
↳ covers={result["diameter_circle_covers"]}')
451         print(f" {output}")
452
453
454 if __name__ == "__main__":
455     main()

```

程序 3: problem1/tests/__init__.py

```

1 """Meaningful geometry and problem-model regression tests."""

```

程序 4: problem1/tests/test_geometry.py

```

1 from __future__ import annotations
2
3 from fractions import Fraction
4 import itertools
5 import json
6 import math
7 from pathlib import Path
8 import random
9 import subprocess
10 import sys
11 import tempfile
12 import unittest
13
14
15 ROOT = Path(__file__).resolve().parents[2]

```

```

16 if str(ROOT) not in sys.path:
17     sys.path.insert(0, str(ROOT))
18
19 from problem1.solve import (HalfPlane, Measurement, analyze_halfplanes, bearing_halfplanes,
20                             convex_hull, diameter_squared_bruteforce, diameter_squared_calipers,
21                             direction, distance_squared, intersection_vertices,
22                             load_measurements, localize, minimum_enclosing_circle, polygon_area)
23
24
25 class HalfPlaneGeometryTests(unittest.TestCase):
26     def test_rectangle_exact_diameter_and_cover(self):
27         result = analyze_halfplanes([HalfPlane(-1, 0, 0), HalfPlane(1, 0, 3),
28                                     HalfPlane(0, -1, 0), HalfPlane(0, 1, 4)])
29         self.assertEqual(result["status"], "polygon")
30         self.assertEqual(result["diameter_m"], 5)
31         self.assertEqual(result["minimum_enclosing_circle"]["center"], [1.5, 2.0])
32         self.assertTrue(result["diameter_circle_covers"])
33
34     def test_empty_parallel_constraints(self):
35         result = analyze_halfplanes([HalfPlane(1, 0, 0), HalfPlane(-1, 0, -1)])
36         self.assertEqual(result["status"], "empty")
37         self.assertIsNone(result["diameter_m"])
38
39     def test_nonempty_halfplane_without_vertices(self):
40         result = analyze_halfplanes([HalfPlane(-1, 0, -3)])
41         self.assertEqual(result["status"], "unbounded")
42         self.assertEqual(result["vertices"], [])
43         self.assertEqual(result["feasible_point"], [3.0, 0.0])
44
45     def test_unbounded_strip_without_vertices(self):
46         result = analyze_halfplanes([HalfPlane(-1, 0, -3), HalfPlane(1, 0, 4)])
47         self.assertEqual(result["status"], "unbounded")
48         self.assertTrue(result["diameter_is_infinite"])
49
50     def test_unbounded_line_without_vertices(self):
51         result = analyze_halfplanes([HalfPlane(-1, 0, -3), HalfPlane(1, 0, 3)])
52         self.assertEqual(result["status"], "unbounded")
53         self.assertEqual(result["feasible_point"], [3.0, 0.0])
54
55     def test_unconstrained_plane(self):
56         self.assertEqual(analyze_halfplanes([])["status"], "unbounded")
57
58     def test_zero_normal_redundant_and_inconsistent(self):
59         self.assertEqual(analyze_halfplanes([HalfPlane(0, 0, 1)])["status"], "unbounded")
60         self.assertEqual(analyze_halfplanes([HalfPlane(0, 0, -1)])["status"], "empty")
61
62     def test_bounded_single_point(self):
63         result = analyze_halfplanes([HalfPlane(-1, 0, -2), HalfPlane(1, 0, 2),
64                                     HalfPlane(0, -1, -3), HalfPlane(0, 1, 3)])
65         self.assertEqual(result["status"], "point")
66         self.assertEqual(result["vertices"], [[2, 3]])
67         self.assertEqual(result["diameter_m"], 0)
68         self.assertTrue(result["diameter_circle_covers"])
69
70     def test_bounded_segment(self):

```

```

71     result = analyze_halfplanes([HalfPlane(-1, 0, -2), HalfPlane(1, 0, 2),
72                                HalfPlane(0, -1, -3), HalfPlane(0, 1, 8)])
73     self.assertEqual(result["status"], "segment")
74     self.assertEqual(result["diameter_m"], 5)
75     self.assertEqual(result["minimum_enclosing_circle"]["radius_m"], 2.5)
76
77     def test_near_parallel_is_large_bounded_not_unbounded(self):
78         epsilon = Fraction(1, 10**14)
79         result = analyze_halfplanes([HalfPlane(0, -1, 0), HalfPlane(-epsilon, 1, 0),
80                                    HalfPlane(epsilon, 1, 1)])
81         self.assertEqual(result["status"], "polygon")
82         self.assertEqual(result["diameter_m"], 1e14)
83         self.assertEqual(len(result["vertices"]), 3)
84
85     def test_acute_triangle_needs_more_than_half_diameter_radius(self):
86         # Exact rational acute triangle (0,0), (4,0), (2,3).
87         result = analyze_halfplanes([HalfPlane(0, -1, 0), HalfPlane(-3, 2, 0),
88                                    HalfPlane(3, 2, 12)])
89         self.assertEqual(result["diameter_m"], 4)
90         self.assertFalse(result["diameter_circle_covers"])
91         self.assertAlmostEqual(result["minimum_enclosing_circle"]["radius_m"], 13 / 6)
92
93     def test_circle_contains_polygon_not_only_support_points(self):
94         # An obtuse triangle has a two-point supported covering circle.
95         vertices = [(Fraction(0), Fraction(0)), (Fraction(8), Fraction(0)),
96                    (Fraction(2), Fraction(1))]
97         circle = minimum_enclosing_circle(vertices)
98         self.assertEqual(circle.radius_squared, 16)
99         self.assertTrue(all(circle.contains(point) for point in vertices))
100
101     def test_redundant_and_rescaled_constraints_leave_geometry_unchanged(self):
102         base = [HalfPlane(-1, 0, 0), HalfPlane(1, 0, 3),
103                HalfPlane(0, -1, 0), HalfPlane(0, 1, 4)]
104         changed = [HalfPlane(p.a * 10**16, p.b * 10**16, p.c * 10**16) for p in base]
105         changed.extend([HalfPlane(1, 1, 100), base[0]])
106         self.assertEqual(analyze_halfplanes(base), analyze_halfplanes(changed))
107
108
109 class CalipersAndCoverageTests(unittest.TestCase):
110     def test_calipers_matches_independent_exact_pairs_on_random_hulls(self):
111         rng = random.Random(20260911)
112         for _ in range(200):
113             hull = convex_hull((Fraction(rng.randint(-1000, 1000)),
114                                Fraction(rng.randint(-1000, 1000))) for _ in range(35))
115             self.assertEqual(diameter_squared_calipers(hull), diameter_squared_bruteforce(hull))
116
117     def test_parallel_support_ties_and_large_translation(self):
118         for offset in (0, 10**12):
119             for coordinates in [(0, 0), (3, 0), (3, 4), (0, 4)],
120                                [(-4, 0), (-2, -3), (2, -3), (4, 0), (2, 3), (-2, 3)]:
121                 hull = convex_hull((Fraction(x + offset), Fraction(y - offset)) for x, y in coordinates)
122                 self.assertEqual(diameter_squared_calipers(hull), diameter_squared_bruteforce(hull))
123
124     def test_calipers_degenerate_cases(self):
125         point = (Fraction(2), Fraction(3))

```

```

126         self.assertEqual(diameter_squared_calipers([point]), (0, (point, point)))
127         second = (Fraction(5), Fraction(7))
128         self.assertEqual(diameter_squared_calipers([point, second]), (25, (point, second)))
129         with self.assertRaises(ValueError):
130             diameter_squared_calipers([])
131
132     def test_area_is_translation_invariant_and_zero_for_degenerate_sets(self):
133         rectangle = [(0, 0), (3, 0), (3, 4), (0, 4)]
134         for offset in (0, 10**12):
135             points = [(Fraction(x + offset), Fraction(y - offset)) for x, y in rectangle]
136             self.assertEqual(polygon_area(points), 12)
137             self.assertEqual(polygon_area(list(reversed(points))), 12)
138             self.assertEqual(polygon_area(points[:2]), 0)
139         self.assertEqual(polygon_area([]), 0)
140
141     def test_optical_decision_uses_radius_and_preserves_exact_threshold(self):
142         # A length-40 segment satisfies the radius condition exactly, while
143         # the diameter-only sufficient condition deliberately cannot certify it.
144         result = analyze_halfplanes([HalfPlane(-1, 0, 0), HalfPlane(1, 0, 40),
145                                     HalfPlane(0, -1, 0), HalfPlane(0, 1, 0)])
146         decision = result["optical_localization"]
147         self.assertTrue(decision["guaranteed_from_enclosing_center"])
148         self.assertTrue(decision["diameter_only_necessary"])
149         self.assertFalse(decision["diameter_only_sufficient"])
150         self.assertEqual(decision["radius_margin_m"], 0)
151         # A tiny true excess cannot be accepted by a floating tolerance.
152         length = Fraction(40) + Fraction(1, 10**12)
153         too_long = analyze_halfplanes([HalfPlane(-1, 0, 0), HalfPlane(1, 0, length),
154                                       HalfPlane(0, -1, 0), HalfPlane(0, 1, 0)])
155         self.assertFalse(too_long["optical_localization"]["guaranteed_from_enclosing_center"])
156         self.assertFalse(too_long["optical_localization"]["diameter_only_necessary"])
157
158     def test_diameter_midpoint_can_be_worse_than_minimum_circle_center(self):
159         result = analyze_halfplanes([HalfPlane(0, -1, 0), HalfPlane(-3, 2, 0),
160                                     HalfPlane(3, 2, 12)])
161         self.assertEqual(result["area_m2"], 6)
162         self.assertEqual(result["max_distance_from_diameter_center_m"], 3)
163         self.assertLess(result["minimum_enclosing_circle"]["radius_m"], 3)
164
165     def test_exact_diameter_40_is_necessary_but_not_sufficient(self):
166         # Exact acute triangle (0,0), (40,0), (20,30), avoiding trig rounding.
167         result = analyze_halfplanes([HalfPlane(0, -1, 0), HalfPlane(-3, 2, 0),
168                                     HalfPlane(3, 2, 120)])
169         self.assertEqual(result["diameter_m"], 40)
170         self.assertTrue(result["optical_localization"]["diameter_only_necessary"])
171         self.assertFalse(result["optical_localization"]["guaranteed_from_enclosing_center"])
172         self.assertAlmostEqual(result["minimum_enclosing_circle"]["radius_m"], 65 / 3)
173
174     def test_empty_and_unbounded_have_no_finite_coverage_decision(self):
175         for planes in ([], [HalfPlane(0, 0, -1)]):
176             result = analyze_halfplanes(planes)
177             self.assertIsNone(result["area_m2"])
178             self.assertIsNone(result["optical_localization"])
179
180

```

```

181 class BearingModelTests(unittest.TestCase):
182     def test_angles_wrap_consistently(self):
183         self.assertEqual(direction(0), direction(360))
184         self.assertEqual(direction(-1), direction(359))
185         first = localize([Measurement(0, 100, 0), Measurement(100, 0, 90)])
186         second = localize([Measurement(0, 100, 360), Measurement(100, 0, -270)])
187         self.assertEqual(first, second)
188
189     def test_north_and_east_follow_official_convention(self):
190         planes = bearing_halfplanes([Measurement(0, 0, 90)])
191         self.assertTrue(all(p.contains((Fraction(0), Fraction(10))) for p in planes))
192         self.assertFalse(all(p.contains((Fraction(10), Fraction(0))) for p in planes))
193         self.assertFalse(all(p.contains((Fraction(0), Fraction(-10))) for p in planes))
194
195     def test_zero_error_is_ray_not_whole_line(self):
196         result = localize([Measurement(0, 0, 0, 0), Measurement(-1, 0, 180, 0)])
197         self.assertEqual(result["status"], "empty")
198         segment = localize([Measurement(0, 0, 0, 0), Measurement(10, 0, 180, 0)])
199         self.assertEqual(segment["status"], "segment")
200         self.assertEqual(segment["diameter_m"], 10)
201
202     def test_zero_error_perpendicular_rays_make_point(self):
203         result = localize([Measurement(0, 1, 0, 0), Measurement(2, 0, 90, 0)])
204         self.assertEqual(result["status"], "point")
205         self.assertEqual(result["vertices"], [[2, 1]])
206
207     def test_same_direction_sectors_are_unbounded(self):
208         result = localize([Measurement(0, 0, 0), Measurement(0, 10, 0)])
209         self.assertEqual(result["status"], "unbounded")
210
211     def test_unbounded_does_not_become_finite_from_vertex_pairs(self):
212         result = localize([Measurement(0, 0, 0)])
213         self.assertEqual(result["vertices"], [[0, 0]])
214         self.assertEqual(result["status"], "unbounded")
215         self.assertIsNone(result["minimum_enclosing_circle"])
216
217     def test_translation_preserves_size_even_at_large_coordinate_offset(self):
218         original = localize([Measurement(0, 100, 0), Measurement(100, 0, 90)])
219         moved = localize([Measurement(10**12, 10**12 + 100, 0),
220                          Measurement(10**12 + 100, 10**12, 90)])
221         self.assertEqual(original["diameter_m"], moved["diameter_m"])
222         self.assertEqual(original["minimum_enclosing_circle"]["radius_m"],
223                          moved["minimum_enclosing_circle"]["radius_m"])
224
225     def test_positive_scale_preserves_relative_geometry(self):
226         first = localize([Measurement(0, 100, 0), Measurement(100, 0, 90)])
227         scaled = localize([Measurement(0, 700, 0), Measurement(700, 0, 90)])
228         self.assertAlmostEqual(scaled["diameter_m"], first["diameter_m"] * 7)
229         self.assertEqual(scaled["diameter_circle_covers"], first["diameter_circle_covers"])
230
231     def test_adding_consistent_observation_cannot_increase_diameter(self):
232         measurements = [Measurement(0, 100, 0), Measurement(100, 0, 90)]
233         first = localize(measurements)
234         extra = localize(measurements + [Measurement(200, 100, 180)])
235         self.assertLessEqual(extra["diameter_m"], first["diameter_m"])

```

```

236
237 def test_equilateral_counterexample_is_actual_bearing_intersection(self):
238     document, measurements = load_measurements(ROOT / "problem1/examples/triangle_counterexample.json")
239     result = localize(measurements)
240     self.assertEqual(result["status"], "polygon")
241     self.assertEqual(len(result["vertices"]), 3)
242     self.assertAlmostEqual(result["diameter_m"], 20, places=9)
243     self.assertAlmostEqual(result["minimum_enclosing_circle"]["radius_m"], 20 / math.sqrt(3), places=9)
244     self.assertFalse(result["diameter_circle_covers"])
245     source = tuple(Fraction(value) for value in document["reference_source"])
246     self.assertTrue(all(p.contains(source) for p in bearing_halfplanes(measurements)))
247     for measurement in measurements:
248         distance = math.hypot(float(source[0]) - measurement.x, float(source[1]) - measurement.y)
249         self.assertGreater(distance, 5)
250         self.assertLess(distance, 1500)
251
252 def test_teammate_optical_counterexample_is_physically_compatible(self):
253     document, measurements = load_measurements(ROOT /
254     ↪ "problem1/examples/triangle_optical_counterexample.json")
255     result = localize(measurements)
256     self.assertEqual(result["status"], "polygon")
257     self.assertEqual(len(result["vertices"]), 3)
258     self.assertAlmostEqual(result["diameter_m"], 40, places=9)
259     self.assertAlmostEqual(result["minimum_enclosing_circle"]["radius_m"], 40 / math.sqrt(3), places=9)
260     self.assertFalse(result["optical_localization"]["guaranteed_from_enclosing_center"])
261     source = tuple(Fraction(value) for value in document["reference_source"])
262     self.assertTrue(all(p.contains(source) for p in bearing_halfplanes(measurements)))
263     for measurement in measurements:
264         actual = math.degrees(math.atan2(float(source[1]) - measurement.y, float(source[0]) -
265         ↪ measurement.x))
266         error = (measurement.bearing_deg - actual + 180) % 360 - 180
267         self.assertLessEqual(abs(error), 1)
268         distance = math.hypot(float(source[0]) - measurement.x, float(source[1]) - measurement.y)
269         self.assertGreater(distance, 5)
270         self.assertLess(distance, document["compatible_reception_radius_m"])
271
272 def test_teammate_wide_baseline_source_and_coverage(self):
273     document, measurements = load_measurements(ROOT / "problem1/examples/wide_baseline_crossing.json")
274     source = tuple(Fraction(value) for value in document["reference_source"])
275     planes = bearing_halfplanes(measurements)
276     self.assertTrue(all(plane.contains(source) for plane in planes))
277     result = localize(measurements)
278     self.assertEqual(result["status"], "polygon")
279     self.assertEqual(len(result["vertices"]), 4)
280     self.assertTrue(result["optical_localization"]["guaranteed_from_enclosing_center"])
281
282 def test_random_consistent_sources_and_vertex_feasibility(self):
283     rng = random.Random(20260910)
284     for _ in range(12):
285         source = (rng.uniform(-500, 500), rng.uniform(-500, 500))
286         measurements = []
287         for angle in (0, 90, 180):
288             rad = math.radians(angle + rng.uniform(-10, 10))
289             station = (source[0] - 300 * math.cos(rad), source[1] - 300 * math.sin(rad))
290             bearing = math.degrees(math.atan2(source[1] - station[1], source[0] - station[0]))

```

```

289         measurements.append(Measurement(*station, bearing + rng.uniform(-0.8, 0.8)))
290     planes = bearing_halfplanes(measurements)
291     exact_source = tuple(Fraction(value) for value in source)
292     self.assertTrue(all(plane.contains(exact_source) for plane in planes))
293     vertices = intersection_vertices(planes)
294     self.assertTrue(vertices)
295     self.assertTrue(all(plane.contains(v) for v in vertices for plane in planes))
296     result = localize(measurements)
297     self.assertEqual(result["status"], "polygon")
298     circle = minimum_enclosing_circle(vertices)
299     self.assertTrue(circle.contains(exact_source))
300     # Any convex combination is a feasible interior point and its
301     # distances cannot exceed the vertex diameter (independent check).
302     midpoints = [(a[0] + b[0]) / 2, (a[1] + b[1]) / 2]
303         for a, b in itertools.combinations(vertices, 2)]
304     diameter_squared = max(distance_squared(a, b) for a in vertices for b in vertices)
305     self.assertTrue(all(distance_squared(a, b) <= diameter_squared for a in midpoints for b in
    ↪ midpoints))
306
307     def test_invalid_numbers_and_error_bounds_rejected(self):
308         for bad in (float("nan"), float("inf"), True, "20"):
309             with self.assertRaises(ValueError):
310                 Measurement(bad, 0, 0)
311         for error in (-1, 90, 180):
312             with self.assertRaises(ValueError):
313                 Measurement(0, 0, 0, error)
314
315     def test_cli_runs_outside_repository_and_writes_valid_json(self):
316         with tempfile.TemporaryDirectory() as temporary:
317             destination = Path(temporary) / "result.json"
318             process = subprocess.run(
319                 [sys.executable, str(ROOT / "problem1/solve.py"),
320                  "--input", "examples/two_station_crossing.json", "--output", str(destination)],
321                 cwd=temporary, capture_output=True, text=True, encoding="utf-8", check=False)
322             self.assertEqual(process.returncode, 0, process.stderr)
323             result = json.loads(destination.read_text(encoding="utf-8"))
324             self.assertEqual(result["status"], "polygon")
325             self.assertTrue(destination.with_suffix(".svg").is_file())
326
327
328 if __name__ == "__main__":
329     unittest.main()

```

程序 5: problem2/benchmark_scoring.py

```

1 """Verify capped-scan equivalence and measure actual work saved, offline only."""
2 from __future__ import annotations
3
4 import argparse
5 from datetime import datetime, timezone
6 import json
7 from pathlib import Path
8 import statistics
9 import sys
10 import time

```

```

11
12 import solve
13
14 ROOT = Path(__file__).resolve().parent
15
16
17 def point_signature(row: dict) -> tuple:
18     return tuple(row["local_xy_m"])
19
20
21 def query_benchmark(name: str, first: tuple, bearing: float, error: float,
22                     step: float, repeats: int) -> dict:
23     polygon = solve.first_region_outer_local(first, bearing, error, 360)
24     evaluator = solve.ContinuousDiameterEvaluator(polygon, error, step, circle_sides=360)
25     points = [(a, b) for a in (250., 500., 750., 950.) for b in (100., 400., 660., 800.)
26               if solve.guaranteed_reception((a, b), error)
27               and solve.direction_clearance((a, b), error) > 5]
28     # Include the mirrored branch without doubling all benchmark work.
29     points.extend((a, -b) for a, b in points[:4])
30     times = {False: [], True: []}
31     rows_by_mode = {}
32     for repetition in range(repeats):
33         # Alternate order to reduce warm-up/order bias. Wall times remain local observations.
34         for early in ((False, True) if repetition % 2 == 0 else (True, False)):
35             started = time.perf_counter()
36             rows = [solve._candidate_row(q, first, bearing, evaluator, early_stop=early)
37                     for q in points]
38             times[early].append(time.perf_counter() - started)
39             rows_by_mode[early] = rows
40     full, fast = rows_by_mode[False], rows_by_mode[True]
41     maximum_difference = max(abs(a["diameter_upper_bound_m"] - b["diameter_upper_bound_m"])
42                              for a, b in zip(full, fast))
43     if maximum_difference > 1e-9:
44         raise AssertionError(f"Capped score mismatch: {name}, {maximum_difference}")
45     if [point_signature(r) for r in sorted(full, key=solve._score_key)] != [
46         point_signature(r) for r in sorted(fast, key=solve._score_key)]:
47         raise AssertionError(f"Candidate ranking changed: {name}")
48     for a, b in zip(full, fast):
49         if not b["score_exact_for_capped_objective"]:
50             raise AssertionError("Early scan must determine the complete capped score")
51         if b["scanned_cells"] > a["scanned_cells"]:
52             raise AssertionError("Early scan did more cells than its full counterpart")
53         if b["early_stopped"]:
54             if b["geometric_diameter_upper_bound_m"] is not None:
55                 raise AssertionError("A partial maximum must not be reported as a complete B")
56             if b["geometric_max_lower_bound_m"] < b["analytic_diameter_upper_bound_m"]:
57                 raise AssertionError("Early stopping occurred before the analytic cap was reached")
58             elif b["geometric_diameter_upper_bound_m"] != a["geometric_diameter_upper_bound_m"]:
59                 raise AssertionError("Completed scans disagree on B")
60     full_cells = sum(r["scanned_cells"] for r in full)
61     fast_cells = sum(r["scanned_cells"] for r in fast)
62     full_seconds, fast_seconds = statistics.median(times[False]), statistics.median(times[True])
63     return {"name": name, "first_station_xy_m": first, "first_bearing_deg": bearing,
64            "error_deg": error, "angle_step_deg": evaluator.step, "circle_sides": 360,
65            "candidate_count": len(points), "repeats": repeats,

```



```

66         "early_stopped_candidates": sum(r["early_stopped"] for r in fast),
67         "full_scanned_cells": full_cells, "early_scanned_cells": fast_cells,
68         "saved_cell_fraction": 1 - fast_cells / full_cells,
69         "full_median_seconds": full_seconds, "early_median_seconds": fast_seconds,
70         "measured_speed_ratio": full_seconds / fast_seconds,
71         "max_score_difference_m": maximum_difference, "ranking_identical": True,
72         "cells_counted_per_repeat": True,
73         "candidate_scores": [{"local_xy_m": a["local_xy_m"],
74                               "full_capped_score_m": a["diameter_upper_bound_m"],
75                               "early_capped_score_m": b["diameter_upper_bound_m"],
76                               "full_B_m": a["geometric_diameter_upper_bound_m"],
77                               "early_B_m": b["geometric_diameter_upper_bound_m"],
78                               "analytic_A_m": b["analytic_diameter_upper_bound_m"],
79                               "scanned_cells": b["scanned_cells"],
80                               "early_stopped": b["early_stopped"]} for a, b in zip(full, fast)]}
81
82
83 def plan_benchmark() -> dict:
84     plans, times = {}, {}
85     for early in (False, True):
86         started = time.perf_counter()
87         plans[early] = solve.choose_second_refined((0., 0.), 0., early_stop=early)
88         times[early] = time.perf_counter() - started
89     full, fast = plans[False], plans[True]
90     for field in ("selected", "fastest_near_best"):
91         if point_signature(full[field]) != point_signature(fast[field]):
92             raise AssertionError(f"End-to-end recommendation changed: {field}")
93         if abs(full[field]["diameter_upper_bound_m"] - fast[field]["diameter_upper_bound_m"]) > 1e-9:
94             raise AssertionError(f"End-to-end score changed: {field}")
95     for field in ("shortlist", "pareto_frontier"):
96         if [point_signature(r) for r in full[field]] != [point_signature(r) for r in fast[field]]:
97             raise AssertionError(f"End-to-end candidate set changed: {field}")
98     if [h["selected_local_xy_m"] for h in full["search_history"]] != [
99         h["selected_local_xy_m"] for h in fast["search_history"]]:
100         raise AssertionError("Early stopping changed the adaptive search path")
101     if full["safe_finite_bound_grid_count"] != fast["safe_finite_bound_grid_count"]:
102         raise AssertionError("The two modes visited different candidate sets")
103     return {"first_station_xy_m": [0., 0.], "first_bearing_deg": 0.,
104            "same_unified_geometry": True, "selected_identical": True,
105            "search_path_and_candidate_order_identical": True,
106            "selected_local_xy_m": fast["selected"]["local_xy_m"],
107            "selected_score_m": fast["selected"]["diameter_upper_bound_m"],
108            "visited_candidate_count": fast["safe_finite_bound_grid_count"],
109            "full_seconds": times[False], "early_seconds": times[True],
110            "measured_speed_ratio": times[False] / times[True],
111            "full_scan_statistics": full["scan_statistics"],
112            "early_scan_statistics": fast["scan_statistics"]}
113
114
115 def run(output: Path, repeats: int = 2, include_plan: bool = True) -> dict:
116     scenarios = [("origin_coarse", (0., 0.), 0., 1., 1.),
117                  ("origin_fine", (0., 0.), 0., 1., .1),
118                  ("boundary_fine", (1500., 0.), 0., 1., .1),
119                  ("rounded_bearing_error", (400., -200.), 25., 1.005, 1.)]
120     queries = []

```

```

121     for args in scenarios:
122         row = query_benchmark(*args, repeats)
123         queries.append(row)
124         print(f"{row['name']}: scores identical, cells {row['full_scanned_cells']} ->
            ↳ {row['early_scanned_cells']}", flush=True)
125     plan = plan_benchmark() if include_plan else None
126     result = {"passed": True, "verified_at_utc": datetime.now(timezone.utc).isoformat(),
127              "python_version": sys.version.split()[0],
128              "scope": "Synthetic offline workload. Same unified geometry, candidates and angle grid; only
            ↳ early stopping changes. No official simulator.",
129              "query_benchmarks": queries, "end_to_end_plan": plan,
130              "interpretation": "Saved cell counts are deterministic for these inputs. Local wall times
            ↳ are observational, not a universal speedup guarantee. No pruning based on a large upper
            ↳ bound is used."}
131     output.mkdir(parents=True, exist_ok=True)
132     (output / "scoring_benchmark.json").write_text(json.dumps(result, ensure_ascii=False, indent=2,
            ↳ allow_nan=False) + "\n", encoding="utf-8")
133     lines = ["# 第二问提前停止: 等价性与计算量验证", "",
134             "本实验只改变提前停止开关: 统一几何模型、候选点、误差和读数网格均相同。
            ↳ 比较的是同一模型下的完整扫描与短路扫描, 不是与历史上较松模型混比。均为本地自建实验。", "",
135             "| 场景 | 点数 | 提前停止点数 | 完整扫描单元 | 实际扫描单元 | 节省比例 | 完整/短路耗时比 |",
136             "| --- | ---: | ---: | ---: | ---: | ---: | ---: |"]
137     for row in queries:
138         lines.append(f"| {row['name']} | {row['candidate_count']} | {row['early_stopped_candidates']} |
            ↳ {row['full_scanned_cells']} | {row['early_scanned_cells']} | {row['saved_cell_fraction']:.2%} |
            ↳ {row['measured_speed_ratio']:.3f} |")
139     lines += ["", "各场景最终评分和候选排序全部一致: 短路时完整几何上界 B 记为 null, 已扫描最大值只记为 B 的下界,
            ↳ 最终有效保证由 A 提供。", "",
140             f"单点评分耗时为交替先后顺序执行 {repeats} 次的中位数: 表中扫描单元数按一次运行计。耗时比大于 1
            ↳ 表示本次短路运行较快, 小于 1 表示较慢, 不选择性删去无提速或变慢场景。"]
141     if plan is not None:
142         full_cells = plan["full_scan_statistics"]["overall"]["scanned_cells"]
143         early_cells = plan["early_scan_statistics"]["overall"]["scanned_cells"]
144         lines += ["", "## 完整选点流程", "",
145                 f"默认原点案例访问 {plan['visited_candidate_count']} 个候选; 两种模式的每层优选点、
            ↳ 最终推荐点、近优备选和时间/直径非劣集合均一致。推荐点为 {plan['selected_local_xy_m']},
            ↳ 最终上界 {plan['selected_score_m']:.6f} 米。", "",
146                 f"包括分层评分和最终复核, 扫描单元从 {full_cells:,} 减至 {early_cells:,}, 节省
            ↳ {full_cells-early_cells:,} 个 ({1-early_cells/full_cells:.3%}); 触发提前停止
            ↳ {plan['early_scan_statistics']['overall']['early_stopped_candidates']} 次。", "",
147                 f"本次完整扫描 {plan['full_seconds']:.3f} 秒, 提前停止 {plan['early_seconds']:.3f} 秒,
            ↳ 耗时比 {plan['measured_speed_ratio']:.3f}。这是单次整流程观测; 精确分层扫描计数见 JSON。
            ↳ "]
148     lines += ["", "## 边界", "", "当 B 始终小于 A, 或 A 为无穷时, 不能依据 A 提前停止;
            ↳ 因此细网格可能几乎不节省扫描。提前停止只在已扫最大值达到 A 时生效, 并不允许用 A 较大为理由淘汰候选点。",
            ↳ "",
149             "复现: `python problem2/benchmark_scoring.py`。`--quick` 跳过完整选点双模式对照并默认写入独立的
            ↳ `results/quick/`。", ""]
150     (output / "scoring_benchmark.md").write_text("\n".join(lines), encoding="utf-8")
151     return result
152
153
154 def main() -> None:
155     parser = argparse.ArgumentParser(description=__doc__)

```

```

156     parser.add_argument("--output", type=Path)
157     parser.add_argument("--repeats", type=int, default=2)
158     parser.add_argument("--quick", action="store_true")
159     args = parser.parse_args()
160     if args.repeats < 1:
161         parser.error("--repeats must be positive")
162     output = args.output or (ROOT / "results/quick" if args.quick else ROOT / "results")
163     output = output if output.is_absolute() else ROOT / output
164     run(output, args.repeats, not args.quick)
165
166
167 if __name__ == "__main__":
168     main()

```

程序 6: problem2/compare_strategies.py

```

1  """ 同一先验、同一上界精度比较合并前后策略；自建数据，不连接模拟器。"""
2  from __future__ import annotations
3
4  import argparse
5  import json
6  import math
7  from pathlib import Path
8
9  import solve
10
11 ROOT = Path(__file__).resolve().parent
12
13
14 def validate_reference_config(name: str, config: dict) -> None:
15     """ 队友保存坐标仅对应这两组固定输入；阻止改样例后静默混用评分条件。"""
16     expected = {"first_station_xy_m": [0., 0.] if name == "synthetic_case" else [1500., 0.],
17                 "first_bearing_deg": 0., "selection_mode": "refined", "error_deg": 1.,
18                 "move_budget_m": 1100., "grid_step_m": 10., "near_best_tolerance": .1,
19                 "circle_sides": 360, "final_angle_step_deg": .1}
20     for key, value in expected.items():
21         if config.get(key, value) != value:
22             raise ValueError(f"固定比较案例 {name} 的 {key} 已改变；请用 solve.py 运行自定义案例，
23                               ↪ 或同步更新全部基线与评分参数")
24     default_stages = ((100., 5., 1.), (20., 1., .5), (5., .25, .25))
25     if tuple(s) for s in config.get("search_stages", default_stages)) != default_stages:
26         raise ValueError(" 固定比较案例的搜索网格已改变，请同步更新实验定义")
27
28 def compact(row: dict) -> dict:
29     return {k: v for k, v in row.items() if "polygon" not in k}
30
31
32 def sample_strategy(first: tuple, first_bearing: float, q: tuple,
33                    polygon: list, upper: float) -> dict:
34     count, received, near, observations, largest, violations = 0, 0, 0, 0, 0., 0
35     candidate = solve.prepare_candidate_region_local(polygon, q, circle_sides=360)
36     for ir in range(61):
37         r = 6+(1500-6)*ir/60
38         for ia in range(21):

```

```

39     angle = math.radians(-1+2*ia/20)
40     source = (r*math.cos(angle), r*math.sin(angle))
41     world = solve.rotate_local(source, first, first_bearing)
42     if math.hypot(*world) > 1800+solve.EPS:
43         continue
44     count += 1
45     d = solve.distance(source, q)
46     if d > max(1000., r)+1e-8:
47         continue
48     received += 1
49     if d <= 5:
50         near += 1
51         continue
52     for error in (-1., -.5, 0., .5, 1.):
53         theta = solve.bearing(source, q)+error
54         posterior = solve.clip_candidate_observation_local(candidate, theta)
55         if not posterior:
56             raise AssertionError(" 相容自建观测不应得到空区域")
57         diameter = solve.polygon_diameter(posterior)
58         largest = max(largest, diameter)
59         violations += diameter > upper+1e-6
60         observations += 1
61     return {"source_grid_count": count, "source_grid_received": received,
62           "source_grid_near": near, "direction_error_combinations": observations,
63           "sample_reception_fraction": received/count if count else None,
64           "sampled_outer_posterior_max_diameter_m": largest,
65           "geometric_upper_violations": violations,
66           "sample_note": " 网格比例不是成功概率；最大值是有限自建情景的外包直径，不是连续最坏值"}
67
68
69 def run_comparison(output_dir: Path, include_extended: bool = True) -> dict:
70     output_dir.mkdir(parents=True, exist_ok=True)
71     scenarios, origin_plan = [], None
72     for name in ("synthetic_case", "boundary_case"):
73         config = json.loads((ROOT/"examples"/f"{name}.json").read_text(encoding="utf-8"))
74         validate_reference_config(name, config)
75         first, theta = tuple(config["first_station_xy_m"]), config["first_bearing_deg"]
76         plan = solve.run_example(config, output_dir if name == "synthetic_case" else output_dir/"boundary")
77         print(f"{name}: selected={plan['selected']}['local_xy_m']",
78               ↪ bound={plan['selected']}['diameter_upper_bound_m']:.6f}", flush=True)
79         polygon = solve.first_region_outer_local(first, theta)
80         evaluator = solve.ContinuousDiameterEvaluator(polygon, angle_step_deg=.1)
81         legacy = solve.choose_second(first, theta)["selected"]
82         # 队友 ZIP 已保存输出的坐标作为固定基线；不要求运行/依赖临时解压项目。
83         teammate = ((848.0480961564259, 529.9192642332049) if name == "synthetic_case"
84                     else (222.73863607376248, 222.73863607376248))
85         points = [("legacy_analytic", " 原仓解析选点", tuple(legacy["local_xy_m"])),
86                  ("teammate_saved", " 队友保存选点", teammate),
87                  ("merged", " 融合选点", tuple(plan["selected"]['local_xy_m'])),
88                  ("merged_faster", "融合10%容差内最短已访点",
89                   ↪ tuple(plan["fastest_near_best"]['local_xy_m']))]
89     if name == "synthetic_case":
90         origin_plan = plan
91         points = [("forward", " 沿示向前进 750m", (750., 0.)),
92                  ("transverse", " 横向移动 750m", (0., 750.)),

```

```

92         ("simple", " 简明安全点 800,600", (800., 600.))] + points
93     rows = []
94     for key, label, point in points:
95         # 比较报告需要完整 B: 短路的部分最大值不能充作完整上界。
96         row = compact(solve._candidate_row(point, first, theta, evaluator, early_stop=False))
97         row.update({"key": key, "label": label,
98                   "guaranteed_reception": solve.guaranteed_reception(point)})
99         row.update(sample_strategy(first, theta, point, polygon,
100                                ↪ row["geometric_diameter_upper_bound_m"]))
101         if row["geometric_upper_violations"]:
102             raise AssertionError(f" 几何上界检查失败: {name}/{key}")
103         if row["guaranteed_reception"] and row["source_grid_count"] != row["source_grid_received"]:
104             raise AssertionError(f" 安全点收信检查失败: {name}/{key}")
105         rows.append(row)
106     scenarios.append({"name": name, "first_station_xy_m": first,
107                    "first_bearing_deg": theta, "error_deg": 1.,
108                    "circle_sides": 360, "angle_step_deg": .1,
109                    "move_budget_m": 1100., "rows": rows})
110 budgets, sensitivity = [], []
111 if include_extended:
112     polygon = solve.first_region_outer_local((0., 0.), 0.)
113     evaluator = solve.ContinuousDiameterEvaluator(polygon, angle_step_deg=.1)
114     for budget in (600., 800., 1000., 1100.):
115         plan = origin_plan if budget == 1100 else solve.choose_second_refined((0., 0.), 0.,
116                                ↪ move_budget_m=budget)
117         baseline = plan["baseline_analytic_selected"]
118         baseline_row = solve._candidate_row(tuple(baseline["local_xy_m"]), (0., 0.), 0., evaluator,
119                                ↪ early_stop=False)
120         selected_row = solve._candidate_row(tuple(plan["selected"]["local_xy_m"]), (0., 0.), 0.,
121                                ↪ evaluator, early_stop=False)
122         budgets.append({"move_budget_m": budget, "selected": compact(selected_row),
123                       "legacy_local_xy_m": baseline["local_xy_m"],
124                       "legacy_same_geometric_bound_m":
125                                ↪ baseline_row["geometric_diameter_upper_bound_m"],
126                       "legacy_same_capped_bound_m": baseline_row["diameter_upper_bound_m"]})
127         print(f"budget {budget:g}: {plan['selected']['diameter_upper_bound_m']:.6f} m", flush=True)
128     fixed_points = [(r["key"], tuple(r["local_xy_m"])) for r in scenarios[0]["rows"]
129                    if r["key"] in ("legacy_analytic", "teammate_saved", "merged")]
130     for sides, step in ((180, .2), (360, .1), (720, .05)):
131         ev = solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0, 0), 0,
132                                ↪ circle_sides=sides),
133                                angle_step_deg=step, circle_sides=sides)
134         sensitivity.append({"circle_sides": sides, "angle_step_deg": ev.step,
135                           "fixed_candidate_bounds": [{"key": key, **compact(solve._candidate_row(q,
136                                ↪ (0., 0.), 0., ev, early_stop=False))} for key, q in fixed_points]})
137     fine = solve.choose_second_refined((0, 0), 0, circle_sides=720,
138                                stages=((100., 5., 1.), (20., 1., .5), (2.5, .125, .125)),
139                                final_angle_step_deg=.05)
140     fine_evaluator = solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0, 0), 0,
141                                ↪ circle_sides=720),
142                                angle_step_deg=.05, circle_sides=720)
143     spatial_refinement = {"radial_step_m": 2.5, "azimuth_step_deg": .125,
144                          "circle_sides": 720, "final_angle_step_deg": .05,

```

```

137         "selected":
            ↪ compact(solve._candidate_row(tuple(fine["selected"]["local_xy_m"]), (0.,
            ↪ 0.), 0., fine_evaluator, early_stop=False)),
138         "default_point_same_fine_geometric_bound_m":
            ↪ fine_evaluator.evaluate(tuple(origin_plan["selected"]["local_xy_m"]))["r"]
            ↪ geometric_diameter_upper_bound_m",
139         "visited_candidate_count": fine["safe_finite_bound_grid_count"]}]
140     print(f"fine search: {fine['selected']['diameter_upper_bound_m']:.6f} m", flush=True)
141 else:
142     spatial_refinement = None
143 result = {"data_provenance": " 自建离线数学实验；不是官方数据、官方模拟器成绩或真实场地效果",
144         "metric": "统一后验模型：首验、第二接收距离圆、第二扇区、5米有效投影及条带；M=360、
            ↪ h=0.1度且扇区/条带/投影同步增宽。评分C=min(A,B)，表中B均完整扫描。",
145         "comparison_note": "同一模型和精度比较C才能评价选点变化；与旧模型数值之差还含几何约束变紧，
            ↪ 均非实际点估计误差。提前停止等价性另见scoring_benchmark.json。",
146         "source_grid": {"radial_count": 61, "radius_min_m": 6, "radius_max_m": 1500,
            ↪ "angle_count": 21, "first_angle_range_deg": [-1, 1], "second_errors_deg":
            ↪ [-1, -.5, 0, .5, 1],
147         "target_disk_filter": True, "reception_radius": "max(1000,r1)"},
148         "scenarios": scenarios, "budgets": budgets, "precision_sensitivity": sensitivity,
149         "spatial_refinement": spatial_refinement}
150 (output_dir/"strategy_comparison.json").write_text(json.dumps(result, ensure_ascii=False, indent=2,
151     ↪ allow_nan=False)+"\n", encoding="utf-8")
152 save_report(result, output_dir/"strategy_comparison.md")
153 return result
154
155
156 def save_report(result: dict, path: Path) -> None:
157     lines = ["# 第二问合并后的同口径比较", "", result["data_provenance"]+"。", "",
158         "全部策略使用统一后验约束、360边圆外包和0.1°读数网格；扇区、条带及近区投影同步增宽。主评分
            ↪ C=min(A,B)，对应同一个最坏直径目标。诊断列 B 均完整扫描，不使用短路中的部分最大值。
            ↪ 不安全点的结果仅对返回示向度的分支有效。", ""]
159     for scenario in result["scenarios"]:
160         lines += [f"## 首点 {scenario['first_station_xy_m']}、示向度 {scenario['first_bearing_deg']}°", "",
161             "| 策略 | 第二点局部坐标/m | 评分C/m | 完整B/m | 移动+检测/s | 保证收信 | 自建网格收信 |",
162             ↪ "|---|---|---|---|---|---|---|"]
163     for row in scenario["rows"]:
164         a, b = row["local_xy_m"]
165         lines.append(f"| {row['label']} | ({a:.3f},{b:.3f}) | {row['diameter_upper_bound_m']:.6f} |
            ↪ {row['geometric_diameter_upper_bound_m']:.6f} | {row['move_and_detect_time_s']:.3f} | {'是'
            ↪ if row['guaranteed_reception'] else '否'} |
            ↪ {row['source_grid_received']}/{row['source_grid_count']} |")
166     legacy = next(r for r in scenario["rows"] if r["key"] == "legacy_analytic")
167     merged = next(r for r in scenario["rows"] if r["key"] == "merged")
168     change = 1-merged["diameter_upper_bound_m"]/legacy["diameter_upper_bound_m"]
169     lines += ["", f"在同一上界指标下，融合点相对原仓点下降 {change:.2%}。
            ↪ 这表示所选策略获得更好的最坏直径保证；不能解释为真实定位误差同比下降。", ""]
170 if result["budgets"]:
171     lines += ["## 移动预算折中", "", "移动时间仅为辅助报告；默认优先最小直径上界，精度平局时才优先短路程。
            ↪ 以下预算变化属于不同约束下的实验。", "", "| 预算/m | 融合点局部坐标/m | 原仓点的同口径C/m | 融合C/m
            ↪ | 移动+检测/s |", "|---|---|---|---|---|"]
172     for row in result["budgets"]:
173         s = row["selected"]
            a, b = s["local_xy_m"]

```

```

174         lines.append(f"| {row['move_budget_m']:.0f} | ({a:.3f},{b:.3f}) |
    ↪     {row['legacy_same_capped_bound_m']:.6f} | {s['diameter_upper_bound_m']:.6f} |
    ↪     {s['move_and_detect_time_s']:.3f} |")
175     lines += ["", "## 数值精度与空间细化", "", "固定三个候选点，只改变圆外包/读数网格精度，
    ↪     避免把换点和换评分混在一起。", "",
176         "    | 圆边数 | 读数步长/° | 原仓点B/m | 队友点B/m | 默认融合点B/m |",
    ↪     "    |---:|---:|---:|---:|---:|"]
177     for row in result["precision_sensitivity"]:
178         values = " | ".join(f"{r['geometric_diameter_upper_bound_m']:.6f}" for r in
    ↪     row["fixed_candidate_bounds"])
179         lines.append(f"| {row['circle_sides']} | {row['angle_step_deg']:.2f} | {values} |")
180     fine = result["spatial_refinement"]
181     s = fine["selected"]
182     a, b = s["local_xy_m"]
183     lines += ["", f"进一步使用720边、0.05°读数网格和最后一层2.5m/0.125°空间细化，选点为({a:.6f},{b:.6f})m，
    ↪     上界为{s['geometric_diameter_upper_bound_m']:.6f}m;
    ↪     默认融合点在同一细评分下为{fine['default_point_same_fine_geometric_bound_m']:.6f}m。
    ↪     选点和排序可能随网格变化，未证明连续全局最优。", "",
184     lines += ["## 数据与保证边界", "", "每个场景先在6至1500米取61个距离、首方位±1°内取21个角度，
    ↪     再滤去目标圆外源；接收半径取与首次收信相容的最小值max(1000,r1)。对正常示向度分支枚举5个第二点固定误差。
    ↪     原点有1281个源组合，边界场景因目标圆过滤而较少。不同误差是不同自建情景，不是同点重复检测后平均。", "",
185         "原始JSON保留样本外包直径最大值、正常读数组合数、上界违反次数及各评分参数。
    ↪     网格收信率不是现实成功率；有限验证不能代替四圆盘及集合包含证明。", "",
186         "20米内一次光学定位应另核验最小覆盖圆半径；C/2并非一般覆盖半径保证。
    ↪     只凭直径上界可用 $C/\sqrt{3} \leq 20$ 作为充分条件，原点两点定位结果尚不满足。", "",
187         "复现：`python problem2/compare_strategies.py`。`--quick`跳过预算与精度/空间敏感性，
    ↪     默认写入`results/quick/`；固定案例参数变化会明确报错，避免基线与融合策略混用不同条件。", ""]
188     path.write_text("\n".join(lines), encoding="utf-8")
189
190
191 def main() -> None:
192     parser = argparse.ArgumentParser(description=__doc__)
193     parser.add_argument("--output", type=Path)
194     parser.add_argument("--quick", action="store_true")
195     args = parser.parse_args()
196     output = args.output or (ROOT/"results"/"quick" if args.quick else ROOT/"results")
197     output = output if output.is_absolute() else ROOT/output
198     run_comparison(output, include_extended=not args.quick)
199
200
201 if __name__ == "__main__":
202     main()

```

程序 7: problem2/solve.py

```

1 """B 问题 2: 有收信保证的第二检测点选择；仅使用 Python 标准库。"""
2 from __future__ import annotations
3
4 import argparse
5 import csv
6 import json
7 import math
8 from functools import lru_cache
9 from pathlib import Path
10 from typing import Callable, Iterable

```

```

11
12 ROOT = Path(__file__).resolve().parent
13 Point = tuple[float, float]
14 EPS = 1e-9
15
16
17 def distance(a: Point, b: Point) -> float:
18     return math.hypot(a[0] - b[0], a[1] - b[1])
19
20
21 def rotate_local(p: Point, origin: Point, bearing_deg: float) -> Point:
22     t = math.radians(bearing_deg)
23     return (origin[0] + p[0] * math.cos(t) - p[1] * math.sin(t),
24             origin[1] + p[0] * math.sin(t) + p[1] * math.cos(t))
25
26
27 def to_local(p: Point, origin: Point, bearing_deg: float) -> Point:
28     t = math.radians(bearing_deg)
29     x, y = p[0] - origin[0], p[1] - origin[1]
30     return x * math.cos(t) + y * math.sin(t), -x * math.sin(t) + y * math.cos(t)
31
32
33 def angular_error(a: float, b: float) -> float:
34     return (a - b + 180.0) % 360.0 - 180.0
35
36
37 def bearing(source: Point, station: Point) -> float:
38     return math.degrees(math.atan2(source[1] - station[1], source[0] - station[0])) % 360
39
40
41 def safe_centers(error_deg: float = 1.0) -> list[Point]:
42     """ 四圆盘中心，相对于第一点的示向度局部坐标。"""
43     e = math.radians(error_deg)
44     return [(r * math.cos(e), sign * r * math.sin(e))
45             for r in (5.0, 1000.0) for sign in (-1, 1)]
46
47
48 def guaranteed_reception(q_local: Point, error_deg: float = 1.0) -> bool:
49     return all(distance(q_local, c) <= 1000.0 + EPS for c in safe_centers(error_deg))
50
51
52 def diameter_bound(q_local: Point, error_deg: float = 1.0) -> float:
53     """ 每一种兼容第二次 direction 读数的定位区域直径的统一上界，米。
54
55     来源是保守矩形角度界和两条有界宽度带的交集，而非线性化方差。
56     不满足四圆盘安全约束或夹角无法保证时返回无穷。
57     """
58     if not guaranteed_reception(q_local, error_deg):
59         return math.inf
60     a, b = q_local
61     e = math.radians(error_deg)
62     h = abs(b) - 1500.0 * math.sin(e)
63     k = max(abs(a - 5.0 * math.cos(e)), abs(a - 1500.0))
64     if h <= 0:
65         return math.inf

```



```

66     beta = math.atan2(h, k) - e
67     if beta <= 0:
68         return math.inf
69     r2_max = max(distance(q_local, (r * math.cos(e), sign * r * math.sin(e)))
70                 for r in (5.0, 1500.0) for sign in (-1, 1))
71     w1, w2 = 1500.0 * math.sin(e), r2_max * math.sin(e)
72     return 2.0 * math.sqrt(w1*w1 + w2*w2 + 2.0*w1*w2*math.cos(beta)) / math.sin(beta)
73
74
75 def prior_nonempty(station: Point, bearing_deg: float, error_deg: float) -> bool:
76     """ 检查首读数闭包与目标圆是否相交; 首距 =5 的退化边界由文档说明。"""
77     cx, cy = to_local((0.0, 0.0), station, bearing_deg)
78     direction = math.degrees(math.atan2(cy, cx))
79     phi = max(-error_deg, min(error_deg, direction))
80     e = math.radians(phi)
81     projection = cx * math.cos(e) + cy * math.sin(e)
82     r = max(5.0, min(1500.0, projection))
83     return math.hypot(cx-r*math.cos(e), cy-r*math.sin(e)) <= 1800.0 + EPS
84
85
86 def choose_second(station: Point, bearing_deg: float, *, grid_step_m: float = 10.0,
87                 move_budget_m: float = 1100.0, tolerance: float = 0.10,
88                 error_deg: float = 1.0) -> dict:
89     """ 在明确有限网格上选择最小保证上界点; 不要求机器人在目标圆内。"""
90     values = (*station, bearing_deg, grid_step_m, move_budget_m, tolerance, error_deg)
91     if not all(math.isfinite(v) for v in values):
92         raise ValueError(" 所有输入必须为有限数值")
93     if grid_step_m <= 0 or move_budget_m <= 0 or tolerance < 0:
94         raise ValueError(" 网格步长、移动预算必须为正, 容差不得为负")
95     if not 0 < error_deg < 10:
96         raise ValueError(" 本实现要求 0 < error_deg < 10; 原题取 1 度")
97     if not prior_nonempty(station, bearing_deg, error_deg):
98         raise ValueError(" 第一点坐标/示向度与 1800 米目标圆、5 至 1500 米范围不相容")
99     candidates = []
100     # 四圆盘已蕴含 a>=0、a<=1005、|b|<=1000; 只搜这一有界范围。
101     for ia in range(math.floor(1005.0 / grid_step_m) + 1):
102         a = ia * grid_step_m
103         for ib in range(1, math.floor(1000.0 / grid_step_m) + 1):
104             b = ib * grid_step_m
105             if math.hypot(a, b) > move_budget_m + EPS:
106                 continue
107             q = (a, b)
108             bound = diameter_bound(q, error_deg)
109             if not math.isfinite(bound):
110                 continue
111             # 对称两侧均保留; 评分相同时优先正侧, 方便结果可重复。
112             for sign in (1, -1):
113                 loc = (a, sign * b)
114                 candidates.append({"local_xy_m": list(loc),
115                                   "xy_m": list(rotate_local(loc, station, bearing_deg)),
116                                   "diameter_upper_bound_m": bound,
117                                   "move_distance_m": math.hypot(a, b),
118                                   "move_and_detect_time_s": math.hypot(a, b) / 5.0 + 5.0})
119     if not candidates:
120         raise ValueError(" 此网格/移动预算没有可提供非退化统一夹角保证的点; 减小步长或增加预算")

```

```

121     candidates.sort(key=lambda x: (x["diameter_upper_bound_m"], x["move_distance_m"],
122                                   -x["local_xy_m"][1], x["local_xy_m"][0]))
123     selected = candidates[0]
124     threshold = selected["diameter_upper_bound_m"] * (1.0 + tolerance)
125     shortlist = [q for q in candidates if q["diameter_upper_bound_m"] <= threshold + EPS]
126     return {
127         "method": "four_disk_reception_guarantee_and_uniform_diameter_bound",
128         "data_provenance": " 自建示例/用户输入; 不是官方模拟器测试数据",
129         "first_station_xy_m": list(station), "first_bearing_deg": bearing_deg % 360,
130         "error_deg": error_deg, "grid_step_m": grid_step_m, "move_budget_m": move_budget_m,
131         "near_best_tolerance": tolerance,
132         "guaranteed_reception": True,
133         "optimization_scope": " 有限网格上统一直径上界最小; 未证明连续域全局最优",
134         "continuous_candidate_region": " 四圆盘交集  $\cap$  移动预算圆  $\cap$   $\{D_{\text{bound}} \leq \text{threshold}\}$ ",
135         "candidate_threshold_m": threshold,
136         "selected": selected,
137         "safe_finite_bound_grid_count": len(candidates),
138         "near_best_grid_count": len(shortlist),
139         "shortlist": shortlist,
140     }
141
142
143 def clip_polygon(vertices: list[Point], normal: Point, offset: float) -> list[Point]:
144     """ 裁剪半平面  $\text{normal} \cdot \mathbf{x} \leq \text{offset}$ ; 容差只用于包含边界。"""
145     if not vertices:
146         return []
147     out = []
148     for a, b in zip(vertices, vertices[1:] + vertices[:1]):
149         fa = normal[0] * a[0] + normal[1] * a[1] - offset
150         fb = normal[0] * b[0] + normal[1] * b[1] - offset
151         ain, bin_ = fa <= EPS, fb <= EPS
152         if ain:
153             out.append(a)
154         if ain != bin_:
155             # 分类和交点都使用同一外扩边界 fa=EPS, 避免边界容差导致线段外推。
156             t = max(0., min(1., (fa-EPS)/(fa-fb)))
157             out.append((a[0] + t * (b[0]-a[0]), a[1] + t * (b[1]-a[1])))
158     return out
159
160
161 def posterior_outer_polygon(first: Point, first_bearing: float,
162                             second: Point, second_bearing: float,
163                             error_deg: float = 1.0, circle_sides: int = 720) -> list[Point]:
164     """ 统一几何模型下实际第二读数的外包; 与选点评分共用全部约束。
165
166     5 米近区由有效投影下界保守放松, 不是原题纯示向度多边形的替代。
167     """
168     q_local = to_local(second, first, first_bearing)
169     prior = first_region_outer_local(first, first_bearing, error_deg, circle_sides)
170     candidate = prepare_candidate_region_local(prior, q_local, error_deg, circle_sides)
171     vertices = clip_candidate_observation_local(candidate, second_bearing-first_bearing)
172     return [rotate_local(p, first, first_bearing) for p in vertices]
173
174
175 def polygon_diameter(vertices: Iterable[Point]) -> float:

```

```

176     points = list(vertices)
177     return max((distance(a, b) for a in points for b in points), default=0.0)
178
179
180 def clip_bearing(vertices: list[Point], station: Point, theta: float,
181                 error_deg: float = 1.0) -> list[Point]:
182     """ 用前向示向扇形裁剪, 正确处理跨 0 度的读数。"""
183     for angle, sign in ((theta-error_deg, -1), (theta+error_deg, 1)):
184         r = math.radians(angle)
185         n = (-sign*math.sin(r), sign*math.cos(r))
186         vertices = clip_polygon(vertices, n, n[0]*station[0]+n[1]*station[1])
187     return vertices
188
189
190 @lru_cache(maxsize=16)
191 def _circle_normals(circle_sides: int) -> tuple[Point, ...]:
192     return tuple((math.cos(2*math.pi*i/circle_sides), math.sin(2*math.pi*i/circle_sides))
193                 for i in range(circle_sides))
194
195
196 def prepare_candidate_region_local(first_polygon: list[Point], q_local: Point,
197                                   error_deg: float = 1., circle_sides: int = 360) -> dict:
198     """ 每候选只构造一次与第二读数无关的共用区域。
199
200     first_polygon 必须来自同一首读数的 first_region_outer_local; 其中已有
201     首扇区、目标圆、首接收圆、首条带和首距有效下界。再裁入第二距离上界圆。
202     """
203     if circle_sides < 8 or int(circle_sides) != circle_sides:
204         raise ValueError("circle_sides 必须是至少为 8 的整数")
205     if not math.isfinite(error_deg) or not 0 < error_deg < 90:
206         raise ValueError(" 误差半角须为 (0,90) 度内的有限数值")
207     if not all(math.isfinite(v) for v in q_local):
208         raise ValueError(" 候选坐标必须为有限数值")
209     radius = 1500.
210     if guaranteed_reception(q_local, error_deg):
211         e = math.radians(error_deg)
212         radius = min(radius, max(distance(q_local, (r*math.cos(e), s*r*math.sin(e)))
213                                 for r in (5., 1500.) for s in (-1, 1)))
214     polygon = list(first_polygon)
215     # 凸多边形的全部顶点已在真圆内时, 无需再枚举相切半平面。
216     if any(distance(v, q_local) > radius+EPS for v in polygon):
217         for n in _circle_normals(circle_sides):
218             polygon = clip_polygon(polygon, n, radius+n[0]*q_local[0]+n[1]*q_local[1])
219             if not polygon:
220                 break
221     return {"polygon_local_xy_m": polygon, "q_local_xy_m": tuple(q_local),
222           "error_deg": error_deg, "circle_sides": circle_sides,
223           "second_distance_upper_m": radius,
224           "outer_distance_upper_m": max((distance(v, q_local) for v in polygon), default=0.)}
225
226
227 def _observation_normals(angle_deg: float, error_deg: float) -> tuple[Point, ...]:
228     t, lo, hi = map(math.radians, (angle_deg, angle_deg-error_deg, angle_deg+error_deg))
229     return ((math.sin(lo), -math.cos(lo)), (-math.sin(hi), math.cos(hi)),
230           (-math.cos(t), -math.sin(t)), (-math.sin(t), math.cos(t)))

```

```

231
232
233 def _observation_limits(candidate: dict, error_deg: float) -> tuple[float, float]:
234     base = candidate["error_deg"]
235     if not math.isfinite(error_deg) or not base <= error_deg < 90:
236         raise ValueError(" 第二读数半角不得小于原始误差, 且必须小于 90 度")
237     extra = math.radians(error_deg-base)
238     # 原条带随读数中心旋转; 外切圆允许 ||x-q|| 略大于真实距离上界。
239     # 法向差的范数<=2 sin(extra/2), 故以下增宽还覆盖实际外包的每个点。
240     width = (candidate["second_distance_upper_m"]*math.sin(math.radians(base))+
241             2*candidate["outer_distance_upper_m"]*math.sin(extra/2))
242     return width, 5*math.cos(math.radians(error_deg))
243
244
245 def _clip_candidate_observation(candidate: dict, normals: tuple[Point, ...],
246                                width: float, near_projection: float) -> list[Point]:
247     q = candidate["q_local_xy_m"]
248     polygon = candidate["polygon_local_xy_m"]
249     for n, delta in ((normals[0], 0.), (normals[1], 0.),
250                     (normals[2], -near_projection), (normals[3], width),
251                     ((-normals[3][0], -normals[3][1]), width)):
252         polygon = clip_polygon(polygon, n, n[0]*q[0]+n[1]*q[1]+delta)
253         if not polygon:
254             break
255     return polygon
256
257
258 def clip_candidate_observation_local(candidate: dict, second_bearing_local_deg: float,
259                                     error_deg: float | None = None) -> list[Point]:
260     """ 实际后验/评分单元共用裁剪: 扇区、近区投影下界和有效第二条带。
261
262     默认用原始误差; 评分传入原误差 + 半格。后者还同步增宽条带及近区投影,
263     保证原始外包被最近读数单元包含 (双精度容差除外)。
264     """
265     if not math.isfinite(second_bearing_local_deg):
266         raise ValueError(" 第二读数必须为有限数值")
267     error = candidate["error_deg"] if error_deg is None else error_deg
268     width, near_projection = _observation_limits(candidate, error)
269     return _clip_candidate_observation(candidate, _observation_normals(second_bearing_local_deg, error),
270                                     width, near_projection)
271
272
273 def first_region_outer_local(station: Point, bearing_deg: float,
274                              error_deg: float = 1.0,
275                              circle_sides: int = 360) -> list[Point]:
276     """ 首读数可行集在首点局部坐标中的外包; 目标圆约束源而非机器人。
277
278     以外切半平面近似两个圆, 并保留首距下界的有效线性约束
279      $x \geq 5 \cos(\text{error})$ 。这仍可能包含首距 $\leq 5$  的少量点, 属于保守放松。
280     """
281     if circle_sides < 8 or int(circle_sides) != circle_sides:
282         raise ValueError("circle_sides 必须是至少为 8 的整数")
283     if not prior_nonempty(station, bearing_deg, error_deg):
284         raise ValueError(" 首读数与目标圆或接收距离范围不相容")
285     vertices = clip_bearing([(-1500., -1500.), (1500., -1500.)],

```

```

286         (1500., 1500.), (-1500., 1500.)], (0., 0.), 0., error_deg)
287     center = to_local((0., 0.), station, bearing_deg)
288     for c, radius in (((0., 0.), 1500.), (center, 1800.)):
289         for i in range(circle_sides):
290             t = 2*math.pi*i/circle_sides
291             n = (math.cos(t), math.sin(t))
292             vertices = clip_polygon(vertices, n, radius+n[0]*c[0]+n[1]*c[1])
293     e = math.radians(error_deg)
294     vertices = clip_polygon(vertices, (-1., 0.), -5*math.cos(e))
295     for sign in (-1., 1.):
296         vertices = clip_polygon(vertices, (0., sign), 1500*math.sin(e))
297     if not vertices:
298         raise ValueError(" 首读数可行区域为空")
299     return vertices
300
301
302 class ContinuousDiameterEvaluator:
303     """ 同一后验几何模型的全第二读数上界；支持 min(A,B) 的精确短路。
304
305     B 是共用外包几何增宽后的有限读数单元直径最大值；半格增宽覆盖连续读数。
306     geometric_max_lower_bound_m 只下界此离散 B，不能称为真实 J 或实际误差下界。
307     这是双精度外包计算，不是区间算术认证；没有用角噪声线性近似评分。
308     """
309
310     def __init__(self, polygon: list[Point], error_deg: float = 1.,
311                  angle_step_deg: float = .1, circle_sides: int = 360):
312         if not polygon:
313             raise ValueError(" 首读数外包不能为空")
314         if not math.isfinite(angle_step_deg) or not 0 < angle_step_deg <= 30:
315             raise ValueError(" 角度扫描步长须在 (0,30] 度")
316         self.count = math.ceil(360/angle_step_deg)
317         self.step = 360/self.count
318         self.error = error_deg
319         self.widened_error = error_deg+self.step/2
320         if not 0 < error_deg < 90 or self.widened_error >= 90:
321             raise ValueError(" 原始和增宽误差半角须在 (0,90) 度")
322         self.polygon = list(polygon)
323         if circle_sides < 8 or int(circle_sides) != circle_sides:
324             raise ValueError("circle_sides 必须是至少为 8 的整数")
325         self.circle_sides = circle_sides
326         self.normals = [(i*self.step, _observation_normals(i*self.step, self.widened_error))
327                         for i in range(self.count)]
328
329     def evaluate(self, q_local: Point, *, analytic_cap_m: float = math.inf,
330                early_stop: bool = True) -> dict:
331         """ 返回 C=min(A,B)；部分最大值达到已知解析上界 A 时可正确短路。
332
333         不传 cap (即 A==+inf) 或 early_stop=False 都完整扫描。短路后 B 未知，
334         仅 C=A 已确定；绝不把已扫描最大值当作完整几何上界。
335         """
336         if math.isnan(analytic_cap_m) or analytic_cap_m < 0:
337             raise ValueError(" 解析 cap 必须非负，允许正无穷")
338         if not isinstance(early_stop, bool):
339             raise ValueError("early_stop 必须为布尔值")
340         candidate = prepare_candidate_region_local(self.polygon, q_local, self.error, self.circle_sides)

```

```

341     width, near_projection = _observation_limits(candidate, self.widened_error)
342     maximum, worst_angle, worst_poly, scanned = 0., 0., [], 0
343     stopped = False
344     for angle, normals in self.normals:
345         scanned += 1
346         poly = _clip_candidate_observation(candidate, normals, width, near_projection)
347         value = polygon_diameter(poly) if poly else 0.
348         if value > maximum:
349             maximum, worst_angle, worst_poly = value, angle, poly
350         if early_stop and maximum >= analytic_cap_m and scanned < self.count:
351             stopped = True
352             break
353     observed_polygon = [list(v) for v in worst_poly]
354     return {"capped_diameter_upper_bound_m": min(analytic_cap_m, maximum),
355           "geometric_diameter_upper_bound_m": None if stopped else maximum,
356           "geometric_max_lower_bound_m": maximum,
357           "geometric_scan_status": "capped_early_stop" if stopped else "complete",
358           "geometric_scan_complete": not stopped,
359           "stop_reason": "analytic_cap_reached" if stopped else "all_cells_scanned",
360           "analytic_cap_m": analytic_cap_m if math.isfinite(analytic_cap_m) else None,
361           "scanned_cells": scanned, "total_cells": self.count,
362           "early_stopped": stopped, "score_exact_for_capped_objective": True,
363           "worst_cell_center_local_deg": None if stopped else worst_angle,
364           "worst_cell_outer_polygon_local_xy_m": None if stopped else observed_polygon,
365           "observed_max_cell_center_local_deg": worst_angle,
366           "observed_max_cell_outer_polygon_local_xy_m": observed_polygon,
367           "second_distance_upper_m": candidate["second_distance_upper_m"],
368           "outer_distance_upper_m": candidate["outer_distance_upper_m"]}
369
370
371 def direction_clearance(q_local: Point, error_deg: float = 1.) -> float:
372     """ 位于前向半平面的安全点，到首示向扇形的外侧垂距。"""
373     e = math.radians(error_deg)
374     return abs(q_local[1])*math.cos(e)-q_local[0]*math.sin(e)
375
376
377 def safe_radial_limit(beta_deg: float, error_deg: float = 1.,
378                      move_budget_m: float = 1100.) -> float:
379     """ 给定局部移动方位，四圆盘在该射线上的精确半径上限。"""
380     alpha = math.radians(abs(beta_deg)+error_deg)
381     if abs(beta_deg)+error_deg >= 90:
382         return 0.
383     return min(move_budget_m, 2000*math.cos(alpha),
384              5*math.cos(alpha)+math.sqrt(1000**2-25*math.sin(alpha)**2))
385
386
387 def _candidate_row(q_local: Point, station: Point, bearing_deg: float,
388                  evaluator: ContinuousDiameterEvaluator, *, early_stop: bool = True) -> dict:
389     # A 是同一后验模型的上界，只能作为 min(A,B) 的 cap，不能用于淘汰候选。
390     strip = diameter_bound(q_local, evaluator.error)
391     geometric = evaluator.evaluate(q_local, analytic_cap_m=strip, early_stop=early_stop)
392     d = math.hypot(*q_local)
393     return {"local_xy_m": list(q_local),
394           "xy_m": list(rotate_local(q_local, station, bearing_deg)),
395           "diameter_upper_bound_m": geometric["capped_diameter_upper_bound_m"],

```

```

396         "analytic_diameter_upper_bound_m": strip if math.isfinite(strip) else None,
397         **geometric, "move_distance_m": d,
398         "move_and_detect_time_s": d/5+5,
399         "direction_clearance_m": direction_clearance(q_local, evaluator.error)}
400
401
402 def _score_key(row: dict) -> tuple:
403     # 1e-7 米仅用于确定浮点平局顺序，不宣称更高精度的物理区别。
404     return (round(row["diameter_upper_bound_m"], 7), row["move_distance_m"],
405             -row["local_xy_m"][1], row["local_xy_m"][0])
406
407
408 def choose_second_refined(station: Point, bearing_deg: float, *,
409                           move_budget_m: float = 1100., tolerance: float = .10,
410                           error_deg: float = 1., circle_sides: int = 360,
411                           stages: tuple = ((100., 5., 1.), (20., 1., .5), (5., .25, .25)),
412                           final_angle_step_deg: float = .1,
413                           baseline_grid_step_m: float = 10.,
414                           early_stop: bool = True,
415                           progress: Callable[[str], None] | None = None) -> dict:
416     """ 四圆盘安全域内，合并解析保底和目标圆条件化的连续读数上界。
417
418     stages 每项为移动距离步长、移动方位步长、读数评分角步长。
419     所有访问候选最后统一精度重新评分，才进行精度/时间比较。
420     """
421     if not all(math.isfinite(v) for v in (*station, bearing_deg, move_budget_m,
422                                           tolerance, error_deg, baseline_grid_step_m)):
423         raise ValueError(" 输入必须为有限数值")
424     if move_budget_m <= 0 or tolerance < 0 or not 0 < error_deg < 10 or baseline_grid_step_m <= 0:
425         raise ValueError(" 移动预算和网格步长须为正，容差非负，误差半角位于 (0,10) 度")
426     if not stages or any(len(s) != 3 or any(not math.isfinite(v) or v <= 0 for v in s) for s in stages):
427         raise ValueError(" 至少需要一层，每层包含三个正有限步长")
428     if not isinstance(early_stop, bool):
429         raise ValueError("early_stop 必须为布尔值")
430     polygon = first_region_outer_local(station, bearing_deg, error_deg, circle_sides)
431     # 四圆盘蕴含 ||q|| <= 1005; 目标圆不参与此处的机器人位置筛选。
432     maximum_radius = min(move_budget_m, 1005.)
433     visited: dict[tuple, Point] = {}
434     history, best, last_r_step, last_beta_step = [], None, 0., 0.
435     # 原实现优选点和两个直观安全点都参加统一评分，保证合并不会遗漏原候选。
436     seeds = [(800., 600.), (800., -600.),
437              (1000/math.sqrt(2), 1000/math.sqrt(2)),
438              (1000/math.sqrt(2), -1000/math.sqrt(2))]
439     try:
440         baseline = choose_second(station, bearing_deg, grid_step_m=baseline_grid_step_m,
441                                  move_budget_m=move_budget_m, error_deg=error_deg)
442         q0 = baseline["selected"]["local_xy_m"]
443         seeds += [tuple(q0), (q0[0], -q0[1])]
444     except ValueError:
445         baseline = None
446
447     def allowed(q):
448         return (math.hypot(*q) <= maximum_radius+EPS and
449                 guaranteed_reception(q, error_deg) and direction_clearance(q, error_deg) > 5+EPS)
450

```

```

451 def grid(start, end, step):
452     return [start+i*step for i in range(max(0, math.floor((end-start)/step+1e-8)+1))]
453
454 def scan_counts(rows):
455     return {"candidate_evaluations": len(rows),
456            "scanned_cells": sum(row["scanned_cells"] for row in rows),
457            "total_cells": sum(row["total_cells"] for row in rows),
458            "early_stopped_candidates": sum(row["early_stopped"] for row in rows)}
459
460 scan_stages = []
461
462 for level, (r_step, beta_step, scan_step) in enumerate(stages):
463     if best is None:
464         radii = grid(r_step, maximum_radius, r_step)+[maximum_radius]
465         betas = grid(-90., 90., beta_step)
466     else:
467         a, b = best["local_xy_m"]
468         r0, beta0 = math.hypot(a, b), math.degrees(math.atan2(b, a))
469         radii = grid(max(r_step, r0-last_r_step), min(maximum_radius, r0+last_r_step), r_step)+[r0]
470         betas = []
471         for center in (beta0, -beta0):
472             betas += grid(center-last_beta_step, center+last_beta_step, beta_step)+[center]
473     current = {key: value for key, value in visited.items()} # 保留旧候选供下一层共同评分。
474     for q in seeds:
475         if allowed(q):
476             current[tuple(round(x, 8) for x in q)] = q
477     for beta in betas:
478         # 四圆盘比队友三圆盘多出约 0~5 米薄层；显式加入安全边界避免漏搜。
479         for r in radii+[safe_radial_limit(beta, error_deg, move_budget_m)]:
480             t = math.radians(beta)
481             q = (r*math.cos(t), r*math.sin(t))
482             if allowed(q):
483                 current[tuple(round(x, 8) for x in q)] = q
484     if not current:
485         raise ValueError(" 当前搜索没有保证正常示向度的候选；增加预算或减小网格")
486     evaluator = ContinuousDiameterEvaluator(polygon, error_deg, scan_step, circle_sides)
487     rows = [_candidate_row(q, station, bearing_deg, evaluator, early_stop=early_stop)
488            for q in current.values()]
489     best = min(rows, key=_score_key)
490     visited.update(current)
491     history.append({"level": level+1, "candidate_count": len(rows),
492                  "radial_step_m": r_step, "azimuth_step_deg": beta_step,
493                  "angle_step_deg": evaluator.step,
494                  "scan_statistics": scan_counts(rows),
495                  "selected_local_xy_m": best["local_xy_m"],
496                  "diameter_upper_bound_m": best["diameter_upper_bound_m"]})
497     if progress:
498         progress(f" 第 {level+1} 层 {len(rows)} 个候选, 上界 {best['diameter_upper_bound_m']:.3f} 米")
499     last_r_step, last_beta_step = r_step, beta_step
500     scan_stages.append(scan_counts(rows))
501 evaluator = ContinuousDiameterEvaluator(polygon, error_deg, final_angle_step_deg, circle_sides)
502 candidates = sorted(_candidate_row(q, station, bearing_deg, evaluator, early_stop=early_stop)
503                    for q in visited.values(), key=_score_key)
504 final_scan = scan_counts(candidates)
505 all_scans = scan_stages+[final_scan]

```



```

506     selected = candidates[0]
507     threshold = selected["diameter_upper_bound_m"]*(1+tolerance)
508     shortlist = [q for q in candidates if q["diameter_upper_bound_m"] <= threshold+EPS]
509     fastest = min(shortlist, key=lambda q: (q["move_distance_m"], _score_key(q)))
510     # 在同一有限集合和同一评分精度下给出非劣的时间/直径代表点。
511     pareto, best_bound = [], math.inf
512     for q in sorted(candidates, key=lambda row: (row["move_distance_m"], _score_key(row))):
513         if q["diameter_upper_bound_m"] < best_bound-1e-7:
514             pareto.append(q)
515             best_bound = q["diameter_upper_bound_m"]
516     if progress:
517         progress(f"统一精度复核 {len(candidates)} 个候选, 上界 {selected['diameter_upper_bound_m']:.3f} 米")
518     return {"method": "four_disk_target_conditioned_continuous_minimax_upper_bound",
519            "data_provenance": " 自建示例/用户输入; 不是官方模拟器测试数据",
520            "first_station_xy_m": list(station), "first_bearing_deg": bearing_deg % 360,
521            "error_deg": error_deg, "move_budget_m": move_budget_m,
522            "grid_step_m": baseline_grid_step_m, "near_best_tolerance": tolerance,
523            "guaranteed_reception": True, "guaranteed_direction": True,
524            "optimization_scope": " 已访问有限候选集、统一精度下的保证上界最小; 未证明连续域全局最优",
525            "continuous_candidate_region": "四圆盘交集  $\cap$  移动预算圆  $\cap$  {到首扇形垂距>5}  $\cap$ 
526             $\hookrightarrow$  {min(D_strip,B_geometry)<=threshold}",
527            "candidate_threshold_m": threshold, "selected": selected,
528            "fastest_near_best": fastest, "pareto_frontier": pareto,
529            "safe_finite_bound_grid_count": len(candidates), "near_best_grid_count": len(shortlist),
530            "shortlist": shortlist, "search_history": history,
531            "early_stop_enabled": early_stop,
532            "scan_statistics": {"stages": scan_stages, "final": final_scan,
533                               "overall": {key: sum(row[key] for row in all_scans) for key in
534                                $\hookrightarrow$  final_scan}},
535            "geometry_model": "shared_prior_reception_disk_bearing_near_projection_and_strips",
536            "circle_sides": circle_sides, "final_angle_step_deg": evaluator.step,
537            "widened_error_deg": evaluator.widened_error,
538            "first_outer_polygon_local_xy_m": [list(p) for p in polygon],
539            "baseline_analytic_selected": baseline["selected"] if baseline else None,
540            "numerical_note": " 圆外包和半步增宽保证连续集合包含; 双精度容差计算, 不是区间算术认证"}
541
542 def save_svg(plan: dict, path: Path) -> None:
543     """ 不用绘图库生成可缩放示意图: 图内坐标为第一点的局部坐标。"""
544     sx, sy = 470.0, 460.0
545     scale = 0.32
546     def xy(p):
547         return sx + p[0]*scale, sy-p[1]*scale
548     parts = ['<svg xmlns="http://www.w3.org/2000/svg" width="1120" height="860" viewBox="0 0 1120 860">',
549             '<rect width="1120" height="860" fill="#f8fafc"/>',
550             '<style>text{font-family:Arial,sans-serif;fill:#172554}.small{font-size:14px}</style>',
551             '<text x="40" y="42" font-size="26" font-weight="bold">Problem 2: guaranteed-reception
552              $\hookrightarrow$  candidate region</text>',
553             '<text x="40" y="71" font-size="16">Local coordinates: first measured bearing is the positive
554              $\hookrightarrow$  x-axis (metres).</text>']
555     # 灰色采样只用于显示连续安全区域, 不参与收信证明。
556     for a in range(0, 1011, 10):
557         for b in range(-1000, 1001, 10):
558             if guaranteed_reception((a, b), plan["error_deg"]):
559                 x, y = xy((a, b))

```

```

557         parts.append(f'<rect x="{x-1.7:.2f}" y="{y-1.7:.2f}" width="3.5" height="3.5"
↳ fill="#dbe3eb"/>')
558     for c in safe_centers(plan["error_deg"]):
559         x,y=xy(c)
560         parts.append(f'<circle cx="{x:.2f}" cy="{y:.2f}" r="320" fill="none" stroke="#94a3b8"
↳ stroke-width="1"/>')
561     for row in plan["shortlist"]:
562         x,y=xy(row["local_xy_m"])
563         parts.append(f'<circle cx="{x:.2f}" cy="{y:.2f}" r="2.2" fill="#2563eb" opacity="0.7"/>')
564     e=math.radians(plan["error_deg"])
565     prior=plan.get("first_outer_polygon_local_xy_m",
↳ [(0.,0.), (1500*math.cos(e), 1500*math.sin(e)), (1500*math.cos(e), -1500*math.sin(e))])
566     points=" ".join(f"{xy(p)[0]:.2f},{xy(p)[1]:.2f}" for p in prior)
567     parts.append(f'<polygon points="{points}" fill="#f59e0b" opacity="0.65"/>')
568     for y in (-1000,-500,0,500,1000):
569         xp,yp=xy((0,y))
570         parts.append(f'<text x="{xp-55:.2f}" y="{yp+5:.2f}" class="small">{y}</text>')
571     for x in (0,500,1000,1500):
572         xp,yp=xy((x,0))
573         parts.append(f'<text x="{xp-12:.2f}" y="{yp+28:.2f}" class="small">{x}</text>')
574     parts.append(f'<line x1="{sx}" y1="{sy}" x2="990" y2="{sy}" stroke="#64748b"/>')
575     x,y=xy(plan["selected"]["local_xy_m"])
576     parts.append(f'<line x1="{sx}" y1="{sy}" x2="{x:.2f}" y2="{y:.2f}" stroke="#dc2626"
↳ stroke-width="2"/>')
577     parts.append(f'<circle cx="{sx}" cy="{sy}" r="5" fill="#172554"/>')
578     parts.append(f'<circle cx="{x:.2f}" cy="{y:.2f}" r="7" fill="#dc2626" stroke="white"
↳ stroke-width="2"/>')
579     parts.append(f'<text x="{x+12:.2f}" y="{y-10:.2f}" class="small">Selected second station</text>')
580     a,b=plan["selected"]["local_xy_m"]
581     bound=plan["selected"]["diameter_upper_bound_m"]
582     parts.extend([
583         '<rect x="35" y="752" width="1050" height="82" rx="10" fill="white" stroke="#cbd5e1"/>',
584         '<text x="50" y="776" class="small">Grey: safe reception region. Blue: candidate grid points
↳ within the stated bound threshold. Orange: first bearing sector.</text>',
585         f'<text x="50" y="800" class="small">Selected: ({a:.2f}, {b:.2f}) m | uniform diameter bound:
↳ {bound:.2f} m | movement: {plan["selected"]["move_distance_m"]:.2f} m.</text>',
586         '<text x="50" y="822" class="small">Blue dots are visited candidates, not an exact region
↳ boundary. The 1800 m target disk constrains the source only.</text>',
587         '</svg>')]
588     path.write_text("\n".join(parts), encoding="utf-8")
589
590
591 def run_example(config: dict, output_dir: Path) -> dict:
592     first = tuple(map(float, config["first_station_xy_m"]))
593     first_bearing = float(config["first_bearing_deg"])
594     mode = config.get("selection_mode", "refined")
595     common = {"move_budget_m": float(config.get("move_budget_m", 1100)),
596               "tolerance": float(config.get("near_best_tolerance", .10)),
597               "error_deg": float(config.get("error_deg", 1.))}
598     if mode == "analytic":
599         plan = choose_second(first, first_bearing,
600                             grid_step_m=float(config.get("grid_step_m", 10)), **common)
601     elif mode == "refined":
602         plan = choose_second_refined(first, first_bearing, **common,
603                                     baseline_grid_step_m=float(config.get("grid_step_m", 10)),

```

```

604         circle_sides=int(config.get("circle_sides", 360)),
605         stages=tuple(tuple(s) for s in config.get("search_stages", ((100.,
        ↪ 5., 1.), (20., 1., .5), (5., .25, .25))))),
606         early_stop=config.get("early_stop", True),
607         final_angle_step_deg=float(config.get("final_angle_step_deg", .1)))
608     else:
609         raise ValueError("selection_mode 必须为 refined 或 analytic")
610     output_dir.mkdir(parents=True,exist_ok=True)
611     rows = plan.pop("shortlist")
612     with (output_dir/"second_point_candidates.csv").open("w",encoding="utf-8-sig",newline="") as f:
613         writer=csv.writer(f)
614         writer.writerow(["local_a_m", "local_b_m", "x_m", "y_m", "diameter_upper_bound_m", "move_distance_m", "
        ↪ move_and_detect_time_s"])
615         for q in rows:
616             writer.writerow([*q["local_xy_m"],*q["xy_m"],q["diameter_upper_bound_m"],q["move_distance_m"]
        ↪ ,q["move_and_detect_time_s"]])
617     save_svg({**plan,"shortlist":rows},output_dir/"candidate_region.svg")
618     if "synthetic_source_xy_m" in config:
619         source=tuple(map(float,config["synthetic_source_xy_m"]))
620         second=tuple(plan["selected"]["xy_m"])
621         radius=float(config.get("synthetic_reception_radius_m",1500))
622         err=float(config.get("synthetic_second_error_deg",0.4))
623         if not 1000 <= radius <= 1500 or distance(source,(0.,0.)) > 1800+EPS:
624             raise ValueError(" 自建源不符合题目位置/接收半径限制")
625         if not 5 < distance(source,first) <= radius:
626             raise ValueError(" 自建源在第一点不能产生 direction")
627         if abs(angular_error(first_bearing,bearing(source,first))) > common["error_deg"]+EPS or abs(err) >
        ↪ common["error_deg"]:
628             raise ValueError(" 自建读数误差超出 ±1 度")
629         if distance(source,second) > radius+EPS:
630             raise AssertionError(" 保证收信失败")
631         if distance(source,second) <= 5:
632             plan["synthetic_observation"]={"status":"near","source_xy_m":list(source),"action":"直接光学定
        ↪ 位与清除"}
633     else:
634         second_bearing=(bearing(source,second)+err)%360
635         poly=posterior_outer_polygon(first,first_bearing,second,second_bearing,
636                                     error_deg=common["error_deg"],
        ↪ circle_sides=plan.get("circle_sides",720))
637     plan["synthetic_observation"]={
638         "status":"direction","source_xy_m":list(source),"reception_radius_m":radius,
639         "second_bearing_deg":second_bearing,"second_fixed_error_deg":err,
640         "source_distance_from_second_m":distance(source,second),
641         "posterior_outer_polygon_xy_m":[list(p) for p in poly],
642         "posterior_outer_polygon_diameter_m":polygon_diameter(poly),
643         "polygon_note":f"与评分共用完整几何构造: 首验、第二距离上界圆、第二扇区、
        ↪ 两次5米近区的有效投影下界和两条有效条带;
        ↪ 圆用{plan.get('circle_sides',720)}个局部相切半平面; 实际读数用原半角,
        ↪ 扫描单元同步增宽扇区/条带/近区投影"}
644     (output_dir/"selection.json").write_text(json.dumps(plan,ensure_ascii=False,indent=2,allow_nan=False)
        ↪ +"\n",encoding="utf-8")
645     return plan
646
647
648 def main() -> None:

```

```

649     parser=argparse.ArgumentParser(description=__doc__)
650     parser.add_argument("--input",type=Path,default=ROOT/"examples"/"synthetic_case.json")
651     parser.add_argument("--output",type=Path,default=ROOT/"results")
652     args=parser.parse_args()
653     # 自定义相对路径也相对本脚本目录，避免启动目录改变含义。
654     input_path=args.input if args.input.is_absolute() else ROOT/args.input
655     output_path=args.output if args.output.is_absolute() else ROOT/args.output
656     plan=run_example(json.loads(input_path.read_text(encoding="utf-8-sig")),output_path)
657     print(json.dumps({"selected":plan["selected"],"candidate_count":plan["near_best_grid_count"],
658                     "output_dir":str(output_path.resolve())},ensure_ascii=False,indent=2))
659
660
661 if __name__ == "__main__":
662     main()

```

程序 8: problem2/tests/test_solve.py

```

1  """ 问题 2 的几何保证、噪声边界和路径约定验证；不访问官方模拟器。"""
2  import importlib.util
3  import json
4  import math
5  import random
6  from pathlib import Path
7  import tempfile
8  import unittest
9
10 MODULE_PATH = Path(__file__).resolve().parents[1] / "solve.py"
11 spec = importlib.util.spec_from_file_location("problem2_solver", MODULE_PATH)
12 solve = importlib.util.module_from_spec(spec)
13 spec.loader.exec_module(solve)
14
15
16 class SecondPointTests(unittest.TestCase):
17     def assert_polygon_contains(self, polygon, point, tolerance=1e-6):
18         self.assertTrue(polygon)
19         for a,b in zip(polygon,polygon[1:]+polygon[:1]):
20             length=solve.distance(a,b)
21             if length > 1e-10:
22                 cross=(b[0]-a[0])*(point[1]-a[1])-(b[1]-a[1])*(point[0]-a[0])
23                 self.assertGreaterEqual(cross/length,-tolerance)
24
25     def test_pure_transverse_move_can_lose_signal(self):
26         self.assertFalse(solve.guaranteed_reception((0, 100)))
27         self.assertGreater(solve.distance((0, 100), (1000, 0)), 1000)
28
29     def test_degenerate_crossing_has_no_finite_bound(self):
30         self.assertTrue(solve.guaranteed_reception((500, 0)))
31         self.assertTrue(math.isinf(solve.diameter_bound((500, 0))))
32
33     def test_four_disk_receipt_on_dense_annular_sector(self):
34         # 覆盖 5、1000、1500 米与 ±1 度边界；半径取与第一次收信相容的最小值。
35         for q in ((0, 0), (500, 800), (650, 700), (1000, 50)):
36             self.assertTrue(solve.guaranteed_reception(q))
37         for r in (5, 5.001, 50, 500, 999.999, 1000, 1000.001, 1200, 1500):
38             for i in range(101):

```

```

39         t=math.radians(-1+0.02*i)
40         g=(r*math.cos(t), r*math.sin(t))
41         self.assertEqual(solve.distance(q,g),max(1000,r)+1e-8)
42
43     def test_choice_and_rotation_equivariance(self):
44         base=solve.choose_second((0,0),0,grid_step_m=25)
45         shifted=solve.choose_second((100,-200),123,grid_step_m=25)
46         self.assertEqual(base["selected"]["local_xy_m"],shifted["selected"]["local_xy_m"])
47         q=base["selected"]["local_xy_m"]
48         expected=solve.rotate_local(q,(100,-200),123)
49         self.assertLess(solve.distance(expected,shifted["selected"]["xy_m"]),1e-8)
50
51     def test_source_circle_does_not_constrain_robot(self):
52         plan=solve.choose_second((2200,0),180,grid_step_m=25)
53         self.assertTrue(plan["guaranteed_reception"])
54
55     def test_impossible_first_observation_and_bad_inputs(self):
56         with self.assertRaises(ValueError):
57             solve.choose_second((2200,0),0)
58         for kwargs in ({'grid_step_m':0},{ 'move_budget_m':-1},{ 'error_deg':0}):
59             with self.assertRaises(ValueError):
60                 solve.choose_second((0,0),0,**kwargs)
61
62     def test_near_best_grid_respects_budget_and_bound(self):
63         p=solve.choose_second((0,0),359.5,grid_step_m=25,move_budget_m=850)
64         for row in p["shortlist"]:
65             self.assertLessEqual(row["move_distance_m"],850+1e-8)
66             self.assertLessEqual(row["diameter_upper_bound_m"],p["candidate_threshold_m"]+1e-8)
67             self.assertTrue(solve.guaranteed_reception(row["local_xy_m"]))
68
69     def test_uniform_bound_against_exact_wedges_at_noise_extremes(self):
70         # 对宽范围真实源与第二点固定误差，直接构造保守交会多边形核验。
71         q=tuple(solve.choose_second((0,0),0,grid_step_m=25)["selected"]["xy_m"])
72         bound=solve.diameter_bound(q)
73         for r in (5.01,100,500,1000,1500):
74             for first_angle in (-1,0,1):
75                 angle=math.radians(first_angle)
76                 source=(r*math.cos(angle),r*math.sin(angle))
77                 for noise in (-1,0,1):
78                     theta=(solve.bearing(source,q)+noise)%360
79                     poly=solve.posterior_outer_polygon((0,0),0,q,theta,circle_sides=720)
80                     self.assertGreaterEqual(len(poly),3)
81                     # 外包圆的微小放松也计入 0.1 米数值余量。
82                     self.assertLessEqual(solve.polygon_diameter(poly),bound+0.1)
83                     for a,b in zip(poly,poly[1:]+poly[:1]):
84                         cross=(b[0]-a[0])*(source[1]-a[1])-(b[1]-a[1])*(source[0]-a[0])
85                         self.assertGreaterEqual(cross,-1e-5)
86
87     def test_bound_is_symmetric(self):
88         for a,b in ((200,400),(500,800),(750,500)):
89             self.assertEqual(solve.diameter_bound((a,b)),solve.diameter_bound((a,-b)))
90
91     def test_translated_rotated_posterior_respects_bound(self):
92         first,theta=(500.0,-400.0),123.0
93         plan=solve.choose_second(first,theta,grid_step_m=25)

```

```

94     q=tuple(plan["selected"]["xy_m"])
95     source=solve.rotate_local((800.0,5.0),first,theta)
96     bound=plan["selected"]["diameter_upper_bound_m"]
97     for noise in (-1,0,1):
98         theta2=(solve.bearing(source,q)+noise)%360
99         poly=solve.posterior_outer_polygon(first,theta,q,theta2)
100         self.assertGreaterEqual(len(poly),3)
101         self.assertLessEqual(solve.polygon_diameter(poly),bound+1e-7)
102         for a,b in zip(poly,poly[1:]+poly[:1]):
103             cross=(b[0]-a[0])*(source[1]-a[1])-(b[1]-a[1])*(source[0]-a[0])
104             self.assertGreaterEqual(cross,-1e-5)
105
106     def test_example_is_reproducible_and_writes_parseable_outputs(self):
107         config=json.loads((MODULE_PATH.parent/"examples"/"synthetic_case.json").read_text(encoding="utf-8")
108         ↪ ")
109         config["grid_step_m"]=25
110         config["search_stages"]=[[250,15,5]]
111         config["final_angle_step_deg"]=2
112         with tempfile.TemporaryDirectory() as tmp:
113             result=solve.run_example(config,Path(tmp))
114             reread=json.loads((Path(tmp)/"selection.json").read_text(encoding="utf-8"))
115             self.assertEqual(result,reread)
116             obs=result["synthetic_observation"]
117             self.assertLess(obs["posterior_outer_polygon_diameter_m"],result["selected"]["diameter_upper_
118             ↪ bound_m"])
119             self.assertTrue((Path(tmp)/"second_point_candidates.csv").is_file())
120             import xml.etree.ElementTree as ET
121             ET.parse(Path(tmp)/"candidate_region.svg")
122
123     def test_four_disk_boundary_extends_teammate_three_disk_region(self):
124         for beta in (10,33,45,59):
125             r=solve.safe_radial_limit(beta)
126             q=(r*math.cos(math.radians(beta)),r*math.sin(math.radians(beta)))
127             self.assertTrue(solve.guaranteed_reception(q))
128             self.assertFalse(solve.guaranteed_reception((q[0]*1.000001,q[1]*1.000001)))
129             r=solve.safe_radial_limit(33)
130             self.assertGreater(r,1000)
131             self.assertAlmostEqual(solve.safe_radial_limit(33,move_budget_m=600),600)
132
133     def test_conditioned_outer_prior_contains_physical_sources(self):
134         # 特别覆盖目标圆边界，世界坐标不能误作局部坐标。
135         first, theta=(1500.,0.),0.
136         poly=solve.first_region_outer_local(first,theta,circle_sides=720)
137         self.assertLessEqual(max(p[0] for p in poly),300+1e-7)
138         self.assertGreater(min(p[0] for p in poly),4.99)
139         for r in (5.001,25,150,299.9):
140             for angle in (-1,0,1):
141                 t=math.radians(angle)
142                 source=(r*math.cos(t),r*math.sin(t))
143                 for a,b in zip(poly,poly[1:]+poly[:1]):
144                     self.assertGreaterEqual((b[0]-a[0])*(source[1]-a[1])-(b[1]-a[1])*(source[0]-a[0]),-1e
145                     ↪ -6)
146
147     def test_widened_cells_cover_non_grid_observations(self):
148         poly=solve.first_region_outer_local((0,0),0)

```

```

146         q=(800.,600.)
147         evaluator=solve.ContinuousDiameterEvaluator(poly,angle_step_deg=2)
148         bound=evaluator.evaluate(q)["geometric_diameter_upper_bound_m"]
149         candidate=solve.prepare_candidate_region_local(poly,q)
150         rng=random.Random(71023)
151         for _ in range(250):
152             # 任意第二读数而不只取离散噪声端点。
153             theta=rng.uniform(0,360)
154             exact=solve.clip_candidate_observation_local(candidate,theta)
155             self.assertEqual(solve.polygon_diameter(exact),bound+1e-7)
156             nearest=round(theta/evaluator.step)*evaluator.step
157             widened=solve.clip_candidate_observation_local(candidate,nearest,evaluator.widened_error)
158             self.assertEqual(solve.polygon_diameter(exact),solve.polygon_diameter(widened)+1e-7)
159             for point in exact:
160                 self.assert_polygon_contains(widened,point)
161
162     def test_refined_selection_retains_baseline_and_shorter_option(self):
163         p=solve.choose_second_refined((0,0),0,stages=((250,15,5),),final_angle_step_deg=2,
164                                     baseline_grid_step_m=25)
165         q=p["selected"]["local_xy_m"]
166         self.assertTrue(solve.guaranteed_reception(q))
167         self.assertGreater(solve.direction_clearance(q),5)
168         evaluator=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0,0),0),angle_step_de
↵ g=2)
169         baseline=solve._candidate_row(tuple(p["baseline_analytic_selected"]["local_xy_m"]), (0,0),0,evalua
↵ tor)
170         self.assertEqual(p["selected"]["diameter_upper_bound_m"],baseline["diameter_upper_bound_m"]+1
↵ e-7)
171         self.assertEqual(p["fastest_near_best"]["diameter_upper_bound_m"],p["candidate_threshold_m"]+
↵ 1e-7)
172         self.assertEqual(p["fastest_near_best"]["move_distance_m"],p["selected"]["move_distance_m"]+1
↵ e-7)
173         self.assertGreater(len(p["pareto_frontier"]),1)
174
175     def test_boundary_conditioning_changes_selection_and_allows_robot_outside(self):
176         p=solve.choose_second_refined((1500,0),0,stages=((250,15,5),(50,3,2)),final_angle_step_deg=1)
177         evaluator=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((1500,0),0),angle_step
↵ _deg=1)
178         baseline=evaluator.evaluate((750.,660.))["geometric_diameter_upper_bound_m"]
179         self.assertLess(p["selected"]["diameter_upper_bound_m"],baseline)
180         self.assertLess(p["selected"]["move_distance_m"],600)
181         outside=solve.choose_second_refined((2200,0),180,stages=((250,15,5),),final_angle_step_deg=2)
182         self.assertTrue(outside["guaranteed_reception"])
183
184     def test_refined_input_rejects_invalid_precision_and_stages(self):
185         for kwargs in ({ "circle_sides":3},{ "stages":()},{ "stages":((1,-1,1),)},
186                       { "final_angle_step_deg":0},{ "move_budget_m":float("nan")}):
187             with self.assertRaises(ValueError):
188                 solve.choose_second_refined((0,0),0,**kwargs)
189
190     def test_clipping_tolerance_does_not_extrapolate_vertices(self):
191         poly=[(0,9e-10),(1,1.1e-9),(1,1),(0,1)]
192         clipped=solve.clip_polygon(poly,(0,1),0)
193         self.assertTrue(clipped)
194         for x,y in clipped:

```

```

195         self.assertGreaterEqual(x,0)
196         self.assertLessEqual(x,1)
197         self.assertLessEqual(y,solve.EPS+1e-16)
198
199     def test_thin_boundary_posterior_uses_same_first_distance_lower_bound(self):
200         for x,source_x in ((1794.,1799.5),(1794.9,1799.95)):
201             first=(x,0.)
202             p=solve.choose_second_refined(first,0,stages=((100,30,10),),
203                                           final_angle_step_deg=1,baseline_grid_step_m=50)
204             q=tuple(p["selected"] ["xy_m"])
205             theta=solve.bearing((source_x,0.),q)
206             poly=solve.posterior_outer_polygon(first,0,q,theta,circle_sides=p["circle_sides"])
207             self.assertLessEqual(solve.polygon_diameter(poly),p["selected"] ["diameter_upper_bound_m"]+1e-7)
208             self.assertGreaterEqual(min(v[0] for v in poly),x+5*math.cos(math.radians(1))-1e-7)
209
210     def test_analytic_cap_short_circuit_equals_full_capped_score(self):
211         ev=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0,0),0),angle_step_deg=5)
212         q=(750.,660.)
213         capped=solve._candidate_row(q,(0,0),0,ev,early_stop=True)
214         full=solve._candidate_row(q,(0,0),0,ev,early_stop=False)
215         self.assertTrue(capped["early_stopped"])
216         self.assertEqual(capped["diameter_upper_bound_m"],full["diameter_upper_bound_m"])
217         self.assertEqual(capped["diameter_upper_bound_m"],solve.diameter_bound(q))
218         self.assertLess(capped["scanned_cells"],capped["total_cells"])
219         self.assertIsNone(capped["geometric_diameter_upper_bound_m"])
220         self.assertIsNone(capped["worst_cell_center_local_deg"])
221         self.assertIsNone(capped["worst_cell_outer_polygon_local_xy_m"])
222         self.assertFalse(capped["geometric_scan_complete"])
223         self.assertEqual(capped["stop_reason"],"analytic_cap_reached")
224         self.assertTrue(capped["score_exact_for_capped_objective"])
225         self.assertGreaterEqual(capped["geometric_max_lower_bound_m"],capped["analytic_diameter_upper_bou
↳ nd_m"])
226         self.assertLessEqual(capped["geometric_max_lower_bound_m"],full["geometric_diameter_upper_bound_m
↳ "]
227
228     def test_large_or_infinite_analytic_cap_does_not_stop(self):
229         ev=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0,0),0),angle_step_deg=5)
230         for q in ((100.,100.), (500.,0.)):
231             result=solve._candidate_row(q,(0,0),0,ev)
232             self.assertFalse(result["early_stopped"])
233             self.assertEqual(result["scanned_cells"],result["total_cells"])
234             self.assertEqual(result["diameter_upper_bound_m"],result["geometric_diameter_upper_bound_m"])
235             self.assertIsNone(solve._candidate_row((500.,0.), (0,0),0,ev) ["analytic_diameter_upper_bound_m"])
236
237     def test_cap_equality_boundary_stops_without_tolerance_shift(self):
238         ev=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0,0),0),angle_step_deg=5)
239         q=(750.,660.)
240         full=ev.evaluate(q,early_stop=False)
241         cap=full["geometric_diameter_upper_bound_m"]
242         equal=ev.evaluate(q,analytic_cap_m=cap)
243         self.assertTrue(equal["early_stopped"])
244         self.assertEqual(equal["geometric_max_lower_bound_m"],cap)
245         self.assertEqual(equal["capped_diameter_upper_bound_m"],cap)
246         above=ev.evaluate(q,analytic_cap_m=math.nextafter(cap,math.inf))
247         self.assertFalse(above["early_stopped"])

```



```

248         self.assertEqual(above["capped_diameter_upper_bound_m"], cap)
249         zero=ev.evaluate(q, analytic_cap_m=0.)
250         self.assertTrue(zero["early_stopped"])
251         self.assertEqual(zero["capped_diameter_upper_bound_m"], 0.)
252
253     def test_search_recommendation_and_candidate_order_match_full_scan(self):
254         options={"stages":((250,15,5),(100,5,2)), "final_angle_step_deg":2,
255                  "baseline_grid_step_m":25}
256         for first in ((0.,0.), (1500.,0.)):
257             short=solve.choose_second_refined(first,0,early_stop=True,**options)
258             full=solve.choose_second_refined(first,0,early_stop=False,**options)
259             self.assertEqual(short["selected"]["local_xy_m"], full["selected"]["local_xy_m"])
260             self.assertEqual(short["selected"]["diameter_upper_bound_m"], full["selected"]["diameter_upper_
↳ _bound_m"])
261             self.assertEqual(short["safe_finite_bound_grid_count"], full["safe_finite_bound_grid_count"])
262             rows=lambda p: [(r["local_xy_m"], r["diameter_upper_bound_m"]) for r in p["shortlist"]]
263             self.assertEqual(rows(short), rows(full))
264             self.assertEqual(short["fastest_near_best"]["local_xy_m"], full["fastest_near_best"]["local_xy_
↳ _m"])
265             self.assertEqual(full["scan_statistics"]["overall"]["early_stopped_candidates"], 0)
266             self.assertGreater(solve.choose_second_refined((0,0),0,**options)["scan_statistics"]["overall"]["
↳ early_stopped_candidates"], 0)
267
268     def test_actual_posterior_uses_shared_candidate_geometry(self):
269         first, theta=(500., -400.), 123.
270         q_local=(750., 660.)
271         second=solve.rotate_local(q_local, first, theta)
272         source=solve.rotate_local((800., 5.), first, theta)
273         theta2=solve.bearing(source, second)+.4
274         prior=solve.first_region_outer_local(first, theta, circle_sides=180)
275         candidate=solve.prepare_candidate_region_local(prior, q_local, circle_sides=180)
276         local=solve.clip_candidate_observation_local(candidate, theta2-theta)
277         actual=solve.posterior_outer_polygon(first, theta, second, theta2, circle_sides=180)
278         self.assertEqual(len(local), len(actual))
279         for a, b in zip(local, actual):
280             self.assertLess(solve.distance(solve.rotate_local(a, first, theta), b), 1e-8)
281         self.assert_polygon_contains(actual, source)
282
283     def test_circle_outer_relaxation_and_half_cell_endpoints_are_covered(self):
284         # 极粗外切圆放松必须进入条带增宽，不能仅替换 sin(error)。
285         q=(500., 400.)
286         prior=solve.first_region_outer_local((0,0),0,circle_sides=8)
287         candidate=solve.prepare_candidate_region_local(prior, q, circle_sides=8)
288         theta=337.917140935712
289         exact=solve.clip_candidate_observation_local(candidate, theta)
290         widened=solve.clip_candidate_observation_local(candidate, theta+1., 2.)
291         self.assertTrue(exact)
292         for p in exact:
293             self.assert_polygon_contains(widened, p)
294         # 包括跨零和不整除 360° 的步长，使用实际 h 而不是输入近似。
295         ev=solve.ContinuousDiameterEvaluator(prior, angle_step_deg=7., circle_sides=8)
296         bound=ev.evaluate(q)["geometric_diameter_upper_bound_m"]
297         for center in (0., ev.step, 51*ev.step):
298             for sign in (-1,1):
299                 angle=center+sign*ev.step/2

```

```

300         actual=solve.clip_candidate_observation_local(candidate,angle)
301         cell=solve.clip_candidate_observation_local(candidate,center,ev.widened_error)
302         self.assertLessEqual(solve.polygon_diameter(actual),bound+1e-7)
303         for p in actual:
304             self.assert_polygon_contains(cell,p)
305
306     def test_near_projection_lower_bound_widens_with_angle(self):
307         q=(500.,0.)
308         candidate=solve.prepare_candidate_region_local(solve.first_region_outer_local((0,0),0),q)
309         # 真距恰为 5 的闭包边界, 实际读数 +1 度, 增宽 cell 中心再 +1 度。
310         source=(505.,0.)
311         exact=solve.clip_candidate_observation_local(candidate,1.)
312         cell=solve.clip_candidate_observation_local(candidate,2.,2.)
313         self.assert_polygon_contains(exact,source)
314         self.assert_polygon_contains(cell,source)
315         for p in exact:
316             self.assert_polygon_contains(cell,p)
317
318     def test_early_stop_input_validation_and_example_switch(self):
319         ev=solve.ContinuousDiameterEvaluator(solve.first_region_outer_local((0,0),0),angle_step_deg=5)
320         for cap in (-1.,float("nan"),-math.inf):
321             with self.assertRaises(ValueError):
322                 ev.evaluate((750,660),analytic_cap_m=cap)
323         with self.assertRaises(ValueError):
324             solve.choose_second_refined((0,0),0,early_stop="false")
325         config={"first_station_xy_m": [0,0], "first_bearing_deg": 0,
326               "search_stages": [[250,15,5]], "final_angle_step_deg": 5, "early_stop": False}
327         with tempfile.TemporaryDirectory() as tmp:
328             result=solve.run_example(config,Path(tmp))
329             self.assertFalse(result["early_stop_enabled"])
330             self.assertEqual(result["scan_statistics"]["overall"]["early_stopped_candidates"],0)
331
332
333 if __name__ == "__main__":
334     unittest.main()

```

程序 9: problem3/__init__.py

```

1 """Problem 3 and reusable robot simulation components."""

```

程序 10: problem3/client.py

```

1 """Serial HTTP client for attachment 2; construction never contacts the simulator."""
2
3 from __future__ import annotations
4
5 import http.client
6 import json
7 import math
8 import threading
9 import time
10 import unicodedata
11 import uuid
12 from datetime import datetime, timezone
13 from pathlib import Path

```

```

14 from urllib.error import HTTPError, URLError
15 from urllib.parse import urlsplit
16 from urllib.request import ProxyHandler, Request, build_opener
17
18
19 class ClientError(RuntimeError):
20     """The client cannot safely treat an action as successful."""
21
22
23 class BudgetExceeded(ClientError):
24     """The remaining time must be reserved for leaving the simulator."""
25
26
27 class ProtocolError(ClientError):
28     """HTTP or JSON does not satisfy the published protocol."""
29
30
31 class TransportError(ClientError):
32     """An action has an unknown outcome; only the identical action may be retried."""
33
34
35 def _number(value):
36     return isinstance(value, (int, float)) and not isinstance(value, bool) and math.isfinite(value)
37
38
39 def _json_object(pairs):
40     result = {}
41     for key, value in pairs:
42         if key in result:
43             raise ValueError(f"Duplicate JSON key: {key}")
44         result[key] = value
45     return result
46
47
48 def _read_body(response):
49     body = response.read(1048577)
50     length = response.headers.get("Content-Length")
51     if length is not None and length.isdigit() and len(body) < min(int(length), 1048577):
52         raise http.client.IncompleteRead(body, int(length) - len(body))
53     return body
54
55
56 class HttpClient:
57     """Send one action at a time and preserve a separate, non-overwritten JSONL log.
58
59     Accepted business responses are returned unchanged. A normal rejected response
60     is also returned unchanged: callers must require ``accepted is True``. Errors
61     raise ClientError. If a response is lost, retries reuse the exact request bytes.
62     After an unresolved outcome, retry_pending() is the only allowed network action.
63     """
64
65     def __init__(self, base_url, robot_id, log_path, *, timeout_s=5.0,
66                 max_retries=2, retry_delay_s=0.2, exit_reserve_s=5.0,
67                 clock=time.monotonic, sleep=time.sleep):
68         parsed = urlsplit(base_url)

```

```

69         if (parsed.scheme != "http" or parsed.hostname not in {"127.0.0.1", "localhost", "::1"}
70             or parsed.path not in {"", "/"}) or parsed.query or parsed.fragment
71             or parsed.username or parsed.password):
72             raise ValueError("Simulator base_url must be a local HTTP origin, e.g. http://127.0.0.1:2026")
73         if (not isinstance(robot_id, str) or not 1 <= len(robot_id.encode("utf-8")) <= 64
74             or any(unicodedata.category(c) in {"Cc", "Cf"} for c in robot_id)):
75             raise ValueError("robot_id must be the exact team ID, 1..64 UTF-8 bytes, without
76                 ↪ control/format characters")
77         if (not _number(timeout_s) or timeout_s <= 0 or not _number(exit_reserve_s)
78             or exit_reserve_s < 0 or not _number(retry_delay_s) or retry_delay_s < 0
79             or not isinstance(max_retries, int) or isinstance(max_retries, bool) or max_retries < 0):
80             raise ValueError("Invalid timeout, retry count, delay, or exit reserve")
81         self.base_url = base_url.rstrip("/")
82         self.robot_id = robot_id
83         self.log_path = Path(log_path)
84         self.log_path.parent.mkdir(parents=True, exist_ok=True)
85         with self.log_path.open("x", encoding="utf-8", newline="\n") as stream:
86             stream.write(json.dumps({"event": "client_created", "base_url": self.base_url,
87                 "created_utc": datetime.now(timezone.utc).isoformat()},
88                 ↪ ensure_ascii=False) + "\n")
89         self.timeout_s = float(timeout_s)
90         self.max_retries = max_retries
91         self.retry_delay_s = float(retry_delay_s)
92         self.exit_reserve_s = float(exit_reserve_s)
93         self._clock, self._sleep = clock, sleep
94         # The official simulator is local-only. System proxy settings must not
95         # forward team IDs or robot actions through an unrelated proxy server.
96         self._opener = build_opener(ProxyHandler({}))
97         self._lock = threading.RLock()
98         self._pending = None
99         self._deadline = None
100        self._entered = False
101        self._exited = False
102        self.position = (0.0, 0.0)
103        self.current_channel = 1
104        self.virtual_time_s = 0.0
105        self.max_virtual_duration_s = 360000.0
106        self.remaining_real_duration_s = None
107
108    @property
109    def pending_request(self):
110        """A copy of an unresolved action for diagnostics; never mutate/reissue it."""
111        with self._lock:
112            return None if self._pending is None else json.loads(self._pending[2].decode("utf-8"))
113
114    @property
115    def can_exit(self):
116        with self._lock:
117            return (self._entered and not self._exited and self._pending is None
118                and self._deadline is not None and self._clock() < self._deadline)
119
120    def check_budget(self, virtual_cost_s=0.0):
121        """Return usable real seconds, raising before spending the exit reserve.
122
123        Use the /enter remaining time, conservatively counted from the first send

```

```

122         (including lost-response retry time), rather than assuming 1200 seconds.
123         """
124         with self._lock:
125             self._require_active()
126             if not _number(virtual_cost_s) or virtual_cost_s < 0:
127                 raise ValueError("virtual_cost_s must be finite and nonnegative")
128             usable = self._deadline - self._clock() - self.exit_reserve_s
129             if usable <= 0:
130                 raise BudgetExceeded("Real-time budget reached; reserve remaining time for /exit")
131             if self.virtual_time_s + virtual_cost_s >= self.max_virtual_duration_s:
132                 raise BudgetExceeded("Virtual-time budget reached")
133             return usable
134
135     def enter(self):
136         with self._lock:
137             if self._entered or self._exited:
138                 raise ClientError("This client already entered a test; do not start it again")
139             return self._new_action("/enter")
140
141     def measure(self, point, channel):
142         return self._located_action("/measure", point, channel)
143
144     def clear(self, point, channel):
145         return self._located_action("/clear", point, channel)
146
147     def exit(self):
148         with self._lock:
149             self._require_active()
150             if self._clock() >= self._deadline:
151                 raise BudgetExceeded("Test deadline has passed; inspect the simulator, do not probe /exit")
152             return self._new_action("/exit")
153
154     def retry_pending(self):
155         """Retry an unresolved request with its original path, ID and exact bytes."""
156         with self._lock:
157             if self._pending is None:
158                 raise ClientError("There is no unresolved request")
159             return self._send_pending()
160
161     def _require_active(self):
162         if not self._entered or self._exited:
163             raise ClientError("Call /enter successfully before actions; ended tests cannot resume")
164
165     def _located_action(self, path, point, channel):
166         with self._lock:
167             self._require_active()
168             if (not isinstance(point, (tuple, list)) or len(point) != 2
169                 or any(not _number(x) or abs(x) > 2000000 for x in point)):
170                 raise ValueError("position must have two finite coordinates within +/-2000000 m")
171             if not isinstance(channel, int) or isinstance(channel, bool) or not 1 <= channel <= 20:
172                 raise ValueError("channel must be an integer in 1..20")
173             cost = math.dist(self.position, point) / 5 + 5
174             if path == "/measure" and channel != self.current_channel:
175                 cost += 1
176             self.check_budget(cost)

```

```

177         return self._new_action(path, {"position": {"x": point[0], "y": point[1]}, "channel": channel})
178
179     def _new_action(self, path, fields=None):
180         if self._pending is not None:
181             raise TransportError("Previous outcome unresolved: use retry_pending(), do not send another
↳ action")
182         payload = {"arena_id": "default", "robot_id": self.robot_id,
183                   "request_id": path[1:] + "-" + uuid.uuid4().hex}
184         payload.update(fields or {})
185         raw = json.dumps(payload, ensure_ascii=False, separators=(",", ":"),
↳ allow_nan=False).encode("utf-8")
186         self._pending = (path, payload, raw, self._clock())
187         return self._send_pending()
188
189     def _log(self, event, **fields):
190         record = {"event": event, "recorded_utc": datetime.now(timezone.utc).isoformat(), **fields}
191         with self.log_path.open("a", encoding="utf-8", newline="\n") as stream:
192             stream.write(json.dumps(record, ensure_ascii=False, allow_nan=False) + "\n")
193
194     def _send_pending(self):
195         path, payload, raw, started = self._pending
196         for attempt in range(self.max_retries + 1):
197             timeout = self.timeout_s
198             if self._deadline is not None:
199                 # Retry the same possibly executed action even inside the reserve,
200                 # but never intentionally send after the actual test deadline.
201                 timeout = min(timeout, self._deadline - self._clock())
202                 if timeout <= 0:
203                     raise BudgetExceeded("Deadline passed with an unresolved action; inspect simulator
↳ status")
204             self._log("request", path=path, payload=payload, attempt=attempt)
205             request = Request(self.base_url + path, data=raw,
206                              headers={"Content-Type": "application/json; charset=utf-8"}, method="POST")
207             try:
208                 try:
209                     with self._opener.open(request, timeout=timeout) as response:
210                         status, body = response.status, _read_body(response)
211                     except HTTPError as error:
212                         with error:
213                             status, body = error.code, _read_body(error)
214                     except (URLError, OSError, http.client.HTTPException) as error:
215                         self._log("transport_error", path=path, request_id=payload["request_id"], error=str(error))
216                         if attempt == self.max_retries:
217                             raise TransportError("No complete response; action outcome unknown. Use
↳ retry_pending() with the same client") from error
218                         delay = self.retry_delay_s * (2 ** attempt)
219                         if self._deadline is not None:
220                             delay = min(delay, max(0, self._deadline - self._clock()))
221                         self._sleep(delay)
222                         continue
223                 try:
224                     if len(body) > 1048576:
225                         raise ValueError("Response exceeds 1 MiB")
226                     result = json.loads(body.decode("utf-8"), object_pairs_hook=_json_object)
227                     self._validate_response(path, result)

```

```

228     except (ValueError, TypeError, KeyError) as error:
229         self._log("protocol_error", path=path, http_status=status, error=str(error))
230         raise ProtocolError("Malformed/unknown response; action outcome unresolved") from error
231     self._log("response", path=path, request_id=payload["request_id"], http_status=status,
    ↪ response=result)
232     if status != 200:
233         if result["accepted"] is False:
234             self._pending = None
235             raise ProtocolError(f"HTTP {status}; action not confirmed. Response: {result}")
236     if result["accepted"] is True:
237         self.virtual_time_s = float(result["virtual_time_s"])
238         if path == "/enter":
239             self._entered = True
240             self.position, self.current_channel = (0.0, 0.0), 1
241             self.max_virtual_duration_s = float(result["max_virtual_duration_s"])
242             self.remaining_real_duration_s = int(result["remaining_real_duration_s"])
243             self._deadline = started + self.remaining_real_duration_s
244         elif path in {"/measure", "/clear"}:
245             self.position = (float(payload["position"]["x"]), float(payload["position"]["y"]))
246             if path == "/measure":
247                 self.current_channel = payload["channel"]
248         elif path == "/exit":
249             self._exited = True
250     self._pending = None
251     return result
252
253 def _validate_response(self, path, result):
254     if not isinstance(result, dict) or type(result.get("accepted")) is not bool:
255         raise ValueError("accepted must be boolean")
256     for name in ("real_timestamp_ms", "virtual_time_s"):
257         if not _number(result.get(name)) or result[name] < 0:
258             raise ValueError(f"Invalid {name}")
259     if result["accepted"] is False:
260         return
261     if result["virtual_time_s"] < self.virtual_time_s:
262         raise ValueError("Accepted virtual time moved backwards")
263     if path == "/enter":
264         for name in ("max_virtual_duration_s", "max_real_duration_s", "remaining_real_duration_s"):
265             if not _number(result.get(name)) or result[name] < 0:
266                 raise ValueError(f"Missing/invalid {name}")
267         remaining = result["remaining_real_duration_s"]
268         if remaining != int(remaining) or remaining > 1200 or result["max_virtual_duration_s"] <= 0:
269             raise ValueError("Invalid time limits")
270     elif path == "/measure":
271         if result.get("measure_result") not in {"no_signal", "near", "direction"}:
272             raise ValueError("Unknown measure_result")
273         if result["measure_result"] == "direction" and (
274             not _number(result.get("svd_deg")) or not 0 <= result["svd_deg"] < 360):
275             raise ValueError("Invalid svd_deg")
276     elif path == "/clear" and result.get("clear_result") not in {"success", "no_target_in_range"}:
277         raise ValueError("Unknown clear_result")
278     elif path == "/exit" and result.get("exit_reason") != "user_exit":
279         raise ValueError("Unknown exit_reason")

```

程序 11: problem3/geometry.py

```
1 """Conservative floating-point geometry shared by problems 3 and 4.
2
3 All disks used for intersection are OUTER regular polygons. Optical decisions
4 use an actual enclosing radius, never half of the polygon diameter.
5 """
6 from __future__ import annotations
7
8 import math
9 import random
10
11 Point = tuple[float, float]
12 Polygon = list[Point]
13 GEOMETRY_MARGIN_M = 1e-6
14
15
16 def distance(a: Point, b: Point) -> float:
17     return math.hypot(a[0] - b[0], a[1] - b[1])
18
19
20 def clip_halfplane(polygon: Polygon, a: float, b: float, c: float) -> Polygon:
21     """Clip by  $a \cdot x + b \cdot y \leq c$ , with a small outward numerical margin."""
22     if not polygon:
23         return []
24     c += GEOMETRY_MARGIN_M * math.hypot(a, b)
25     output: Polygon = []
26     previous = polygon[-1]
27     fp = a * previous[0] + b * previous[1] - c
28     for current in polygon:
29         fc = a * current[0] + b * current[1] - c
30         if (fc <= 0) != (fp <= 0):
31             t = fp / (fp - fc)
32             output.append((previous[0] + t * (current[0] - previous[0]),
33                             previous[1] + t * (current[1] - previous[1])))
34         if fc <= 0:
35             output.append(current)
36     previous, fp = current, fc
37     return output
38
39
40 def initial_region(radius: float = 1800.0, sides: int = 96) -> Polygon:
41     outer = radius / math.cos(math.pi / sides) + GEOMETRY_MARGIN_M
42     return [(outer * math.cos((2 * k + 1) * math.pi / sides),
43             outer * math.sin((2 * k + 1) * math.pi / sides)) for k in range(sides)]
44
45
46 def intersect_bearing(polygon: Polygon, point: Point, bearing: float,
47                       error_deg: float = 1.005, receive_radius: float = 1500.0,
48                       disk_sides: int = 48) -> Polygon:
49     """Intersect an angular sector and a containing disk; retain true source."""
50     result = list(polygon)
51     low, high = map(math.radians, (bearing - error_deg, bearing + error_deg))
52     # cross(low, x-point) >= 0; cross(high, x-point) <= 0.
53     for a, b in ((math.sin(low), -math.cos(low)),
```



```

54         (-math.sin(high), math.cos(high))):
55     result = clip_halfplane(result, a, b, a * point[0] + b * point[1])
56     for k in range(disk_sides):
57         angle = 2 * math.pi * k / disk_sides
58         a, b = math.cos(angle), math.sin(angle)
59         result = clip_halfplane(result, a, b, receive_radius + a * point[0] + b * point[1])
60     return result
61
62
63 def polygon_centroid(polygon: Polygon) -> Point:
64     if not polygon:
65         raise ValueError("Empty feasible polygon")
66     area2 = sx = sy = 0.0
67     for p, q in zip(polygon, polygon[1:] + polygon[:1]):
68         cross = p[0] * q[1] - q[0] * p[1]
69         area2 += cross
70         sx += (p[0] + q[0]) * cross
71         sy += (p[1] + q[1]) * cross
72     if abs(area2) < 1e-8:
73         return (sum(p[0] for p in polygon) / len(polygon),
74               sum(p[1] for p in polygon) / len(polygon))
75     return sx / (3 * area2), sy / (3 * area2)
76
77
78 def _circle_three(a: Point, b: Point, c: Point):
79     bx, by, cx, cy = b[0] - a[0], b[1] - a[1], c[0] - a[0], c[1] - a[1]
80     determinant = 2 * (bx * cy - by * cx)
81     if abs(determinant) < 1e-12:
82         return None
83     b2, c2 = bx * bx + by * by, cx * cx + cy * cy
84     center = (a[0] + (cy * b2 - by * c2) / determinant,
85             a[1] + (bx * c2 - cx * b2) / determinant)
86     return center, distance(center, a)
87
88
89 def enclosing_circle(polygon: Polygon) -> tuple[Point, float]:
90     """Randomized incremental minimum enclosing circle, verified outwards.
91
92     The fixed shuffle makes experiments reproducible. The final radius is
93     recomputed against ALL vertices; any rounding can only loosen the bound.
94     """
95     if not polygon:
96         raise ValueError("Empty feasible polygon")
97     points = list(polygon)
98     random.Random(20260911).shuffle(points)
99     center, radius = points[0], 0.0
100    for i, a in enumerate(points):
101        if distance(center, a) <= radius + 1e-9:
102            continue
103        center, radius = a, 0.0
104        for j, b in enumerate(points[:i]):
105            if distance(center, b) <= radius + 1e-9:
106                continue
107            center = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
108            radius = distance(a, b) / 2

```

```

109         for c in points[:j]:
110             if distance(center, c) <= radius + 1e-9:
111                 continue
112             circle = _circle_three(a, b, c)
113             if circle is not None:
114                 center, radius = circle
115 radius = max(distance(center, p) for p in polygon) + GEOMETRY_MARGIN_M
116 return center, radius
117
118
119 def contains(polygon: Polygon, point: Point, tolerance: float = 1e-5) -> bool:
120     if not polygon:
121         return False
122     for a, b in zip(polygon, polygon[1:] + polygon[:1]):
123         cross = (b[0] - a[0]) * (point[1] - a[1]) - (b[1] - a[1]) * (point[0] - a[0])
124         if cross < -tolerance * max(1.0, distance(a, b)):
125             return False
126     return True
127
128
129 def optical_cover(polygon: Polygon, spacing: float = 26.0) -> list[Point]:
130     """Finite 20m optical cover: rotated square cells, side <= 20*sqrt(2).
131
132     Every cell intersecting the feasible polygon contributes its center, even
133     when the center lies outside the polygon. Omitting such boundary centers
134     would invalidate coverage. Rotation follows the longest polygon chord.
135     """
136     if not 0 < spacing <= 20 * math.sqrt(2) - 1e-3:
137         raise ValueError("Optical square spacing must safely fit in radius 20")
138     if not polygon:
139         raise ValueError("Cannot cover an empty region")
140     a, b = max(((a, b) for a in polygon for b in polygon),
141               key=lambda pair: distance(*pair))
142     chord = distance(a, b)
143     ux, uy = ((b[0] - a[0]) / chord, (b[1] - a[1]) / chord) if chord else (1.0, 0.0)
144     rotated = [(p[0] * ux + p[1] * uy, -p[0] * uy + p[1] * ux) for p in polygon]
145     min_x, max_x = min(p[0] for p in rotated), max(p[0] for p in rotated)
146     min_y, max_y = min(p[1] for p in rotated), max(p[1] for p in rotated)
147     answer = []
148     for ix in range(math.floor(min_x / spacing), math.floor(max_x / spacing) + 1):
149         rows = range(math.floor(min_y / spacing), math.floor(max_y / spacing) + 1)
150         if ix % 2:
151             rows = reversed(rows)
152         for iy in rows:
153             cell = rotated
154             for aa, bb, cc in ((1, 0, (ix + 1) * spacing), (-1, 0, -ix * spacing),
155                               (0, 1, (iy + 1) * spacing), (0, -1, -iy * spacing)):
156                 cell = clip_halfplane(cell, aa, bb, cc)
157             if cell:
158                 x, y = (ix + .5) * spacing, (iy + .5) * spacing
159                 answer.append((x * ux - y * uy, x * uy + y * ux))
160     return answer
161
162
163 def certified_covering_radius(points: list[Point], radius: float = 1800.0) -> float:

```

```

164     """Upper-bound distance from the target disk to its nearest scan point.
165
166     Clip an outer target polygon into nearest-site Voronoi cells. Euclidean
167     distance to the cell's site is convex, hence its maximum over the polygon
168     occurs at a vertex. An upper bound  $\leq 1000$  certifies omnidirectional cover.
169     """
170     if not points:
171         return math.inf
172     maximum = 0.0
173     for point in points:
174         cell = initial_region(radius)
175         for other in points:
176             if other == point:
177                 continue
178             a, b = 2 * (other[0] - point[0]), 2 * (other[1] - point[1])
179             c = other[0]**2 + other[1]**2 - point[0]**2 - point[1]**2
180             cell = clip_halfplane(cell, a, b, c)
181         if cell:
182             maximum = max(maximum, max(distance(point, vertex) for vertex in cell))
183     return maximum + GEOMETRY_MARGIN_M

```

程序 12: problem3/policy.py

```

1  """Black-box search, bounded-error localization and optical clearance policy.
2
3  The client contract deliberately exposes no source positions, count, receiver
4  radii or emission directions. run() never enters or exits the official arena.
5  """
6  from __future__ import annotations
7
8  import math
9  from dataclasses import asdict, dataclass
10
11  from problem3.geometry import (Point, Polygon, certified_covering_radius, distance, enclosing_circle,
12     initial_region, intersect_bearing, optical_cover, polygon_centroid)
13
14
15  @dataclass
16  class PolicyConfig:
17     ring_radius: float = 1150.0
18     bearing_error_deg: float = 1.005
19     max_probe_count: int = 5
20     lateral_fraction: float = 0.12
21     lateral_min_m: float = 45.0
22     lateral_max_m: float = 140.0
23     opportunistic_radius_m: float = 0.0
24     optical_spacing_m: float = 26.0
25     clear_guarantee_radius_m: float = 19.99
26     exploratory_clear_radius_m: float = 55.0
27     neighbor_updates: bool = True
28     neighbor_max_distance_m: float = 1450.0
29     neighbor_min_parallax_deg: float = 12.0
30     adaptive_discovery: bool = False
31     discovery_min_spacing_m: float = 700.0
32     discovery_replacement_only: bool = True

```

```

33     target_uncertainty_weight: float = 0.5
34
35
36 class SearchPolicy:
37     """Seven-point certified omnidirectional search with adaptive goal visits.
38
39     Override coverage_points() for a different certified discovery design.
40     Negative readings do not shrink source polygons, making localization safe
41     to reuse for a 180-degree directional emitter in problem 4.
42     """
43     def __init__(self, problem: int = 3, strategy: str = "adaptive", config=None):
44         if strategy not in ("adaptive", "baseline", "optical"):
45             raise ValueError("strategy must be adaptive, baseline, or optical")
46         self.problem = problem
47         self.strategy = strategy
48         self.config = config if isinstance(config, PolicyConfig) else PolicyConfig(**(config or {}))
49         if self.config.bearing_error_deg < 1.005:
50             raise ValueError("Official two-decimal bearings require at least 1.005 degrees")
51         if not 0 < self.config.clear_guarantee_radius_m < 20:
52             raise ValueError("Clear guarantee radius must lie below 20 metres")
53         self.regions: dict[int, Polygon] = {}
54         self.observations: dict[int, list[tuple[Point, float]]] = {}
55         self.cleared: set[int] = set()
56         self.coverage_visited: list[Point] = []
57         self.stats = {"measure_actions": 0, "clear_actions": 0,
58                     "failed_clears": 0, "optical_fallbacks": 0,
59                     "negative_localization_reads": 0}
60
61     def coverage_points(self) -> list[Point]:
62         radius = self.config.ring_radius
63         # Closed analytical certificate for the center-plus-six-ring design.
64         outer = math.sqrt(1800**2 + radius**2 - 2 * 1800 * radius * math.cos(math.pi / 6))
65         if radius / math.sqrt(3) > 1000 or outer > 1000:
66             raise ValueError("The chosen ring radius does not certify 1000m coverage")
67         return [(0.0, 0.0)] + [(radius * math.cos(k * math.pi / 3),
68                                radius * math.sin(k * math.pi / 3)) for k in range(6)]
69
70     def _clear(self, client, point: Point, channel: int) -> bool:
71         client.check_budget()
72         response = client.clear(point, channel)
73         if response.get("accepted") is not True:
74             raise RuntimeError("Clear action was not accepted")
75         self.stats["clear_actions"] += 1
76         if response.get("clear_result") == "success":
77             self.cleared.add(channel)
78             return True
79         if response.get("clear_result") != "no_target_in_range":
80             raise RuntimeError(f"Invalid clear response: {response!r}")
81         self.stats["failed_clears"] += 1
82         return False
83
84     def _measure(self, client, point: Point, channel: int) -> str:
85         client.check_budget()
86         response = client.measure(point, channel)
87         if response.get("accepted") is not True:

```

```

88         raise RuntimeError("Measure action was not accepted")
89     self.stats["measure_actions"] += 1
90     result = response.get("measure_result")
91     if result == "near":
92         if not self._clear(client, point, channel):
93             raise RuntimeError("A near reading was followed by a failed optical clear")
94     elif result == "direction":
95         angle = float(response["svd_deg"])
96         previous = self.regions.get(channel, initial_region())
97         updated = intersect_bearing(previous, point, angle,
98                                   error_deg=self.config.bearing_error_deg)
99         if not updated:
100             raise RuntimeError("Bearing intersection is empty: model/protocol inconsistency")
101         self.regions[channel] = updated
102         self.observations.setdefault(channel, []).append((point, angle))
103     elif result != "no_signal":
104         raise RuntimeError(f"Invalid measurement response: {response!r}")
105     return result
106
107 def _scan(self, client, point: Point) -> None:
108     unknown = [ch for ch in range(1, 21) if ch not in self.regions and ch not in self.cleared]
109     if client.current_channel in unknown:
110         unknown.remove(client.current_channel)
111         unknown.insert(0, client.current_channel)
112     for channel in unknown:
113         self._measure(client, point, channel)
114     self.coverage_visited.append(point)
115
116 def _update_neighbors(self, client) -> None:
117     """Collect useful second bearings at a stop already paid for."""
118     if self.strategy != "adaptive" or not self.config.neighbor_updates:
119         return
120     point = client.position
121     for channel in list(self.regions):
122         if channel in self.cleared:
123             continue
124         polygon = self.regions[channel]
125         center, radius = enclosing_circle(polygon)
126         if radius < 35 or distance(point, center) > self.config.neighbor_max_distance_m:
127             continue
128         origin, _ = self.observations[channel][-1]
129         a = (origin[0] - center[0], origin[1] - center[1])
130         b = (point[0] - center[0], point[1] - center[1])
131         denominator = math.hypot(*a) * math.hypot(*b)
132         sine = abs(a[0] * b[1] - a[1] * b[0]) / denominator if denominator else 0
133         if sine >= math.sin(math.radians(self.config.neighbor_min_parallax_deg)):
134             self._measure(client, point, channel)
135
136 def _probe_point(self, channel: int, client, probe_index: int) -> Point:
137     polygon = self.regions[channel]
138     center = polygon_centroid(polygon)
139     origin, bearing = self.observations[channel][-1]
140     theta = math.radians(bearing)
141     perpendicular = (-math.sin(theta), math.cos(theta))
142     if self.strategy == "baseline" and len(self.observations[channel]) == 1:

```

```

143         return (origin[0] + 200 * perpendicular[0], origin[1] + 200 * perpendicular[1])
144     # Move towards the source while creating parallax. An estimate drives
145     # movement only: correctness comes from retained sectors and fallback.
146     _, radius = enclosing_circle(polygon)
147     if len(self.observations[channel]) > 1 and radius < 100:
148         return center
149     lateral = min(self.config.lateral_max_m, max(self.config.lateral_min_m,
150         distance(origin, center) * self.config.lateral_fraction))
151     if probe_index % 2:
152         lateral = -lateral
153     return (center[0] + lateral * perpendicular[0],
154         center[1] + lateral * perpendicular[1])
155
156 def _localize(self, channel: int, client) -> bool:
157     attempted_measure: set[tuple[float, float]] = set()
158     attempted_clear: list[Point] = []
159     if self.strategy != "optical":
160         for probe_index in range(self.config.max_probe_count):
161             polygon = self.regions[channel]
162             center, radius = enclosing_circle(polygon)
163             if radius <= self.config.clear_guarantee_radius_m:
164                 if self._clear(client, center, channel):
165                     return True
166                 # Retain the region; unexpected failure must not certify success.
167                 attempted_clear.append(center)
168             elif radius <= self.config.exploratory_clear_radius_m and not attempted_clear:
169                 if self._clear(client, polygon_centroid(polygon), channel):
170                     return True
171                 attempted_clear.append(polygon_centroid(polygon))
172             point = self._probe_point(channel, client, probe_index)
173             key = tuple(round(value, 6) for value in point)
174             if key in attempted_measure:
175                 break
176             attempted_measure.add(key)
177             result = self._measure(client, point, channel)
178             if channel in self.cleared:
179                 return True
180             if result == "no_signal":
181                 self.stats["negative_localization_reads"] += 1
182             # Fixed error at one point: never average repeated readings.
183         self.stats["optical_fallbacks"] += 1
184         points = optical_cover(self.regions[channel], self.config.optical_spacing_m)
185         # Nearest-neighbour ordering changes cost only, never the finite cover.
186         while points:
187             index = min(range(len(points)), key=lambda i: distance(client.position, points[i]))
188             if self._clear(client, points.pop(index), channel):
189                 return True
190         raise RuntimeError("Optical cover exhausted without clearance: model/protocol inconsistency")
191
192 def run(self, client) -> dict:
193     pending = self.coverage_points()
194     while pending or any(channel not in self.cleared for channel in self.regions):
195         client.check_budget()
196         known = [channel for channel in self.regions if channel not in self.cleared]
197         def target_cost(ch):

```

```

198         return distance(client.position, polygon_centroid(self.regions[ch])) +
        ↪ self.config.target_uncertainty_weight * enclosing_circle(self.regions[ch])[1]
199     channel = min(known, key=target_cost) if known else None
200     waypoint_index = min(range(len(pending)), key=lambda i: distance(client.position, pending[i]))
    ↪ if pending else None
201     visit_target = channel is not None and (waypoint_index is None or self.strategy != "baseline"
    ↪ and
202         target_cost(channel) <=
203         distance(client.position, pending[waypoint_index]) + 180)
204     if visit_target:
205         self._localize(channel, client)
206         if self.problem == 3 and self.strategy == "adaptive" and self.config.adaptive_discovery:
207             if self.config.discovery_replacement_only:
208                 for index in sorted(range(len(pending)), key=lambda i: distance(client.position,
    ↪ pending[i])):
209                     replacement = self.coverage_visited + pending[:index] + pending[index + 1:] +
    ↪ [client.position]
210                     if certified_covering_radius(replacement) <= 1000:
211                         self._scan(client, client.position)
212                         pending.pop(index)
213                         break
214                     elif not self.coverage_visited or min(distance(client.position, point) for point in
    ↪ self.coverage_visited) >= self.config.discovery_min_spacing_m:
215                         self._scan(client, client.position)
216                     # Remove only waypoints made redundant by actual full scans.
217                     for index in reversed(range(len(pending))):
218                         remaining = self.coverage_visited + pending[:index] + pending[index + 1:]
219                         if certified_covering_radius(remaining) <= 1000:
220                             pending.pop(index)
221             elif waypoint_index is not None:
222                 self._scan(client, pending.pop(waypoint_index))
223             else:
224                 self._localize(channel, client)
225                 self._update_neighbors(client)
226                 # Known upper count permits an earlier proof of completion.
227                 if len(self.cleared) == 16:
228                     return self._result("maximum_source_count_cleared", not pending)
229     return self._result("certified_coverage_and_all_detected_cleared", True)
230
231     def _result(self, reason: str, complete: bool) -> dict:
232         return {"problem": self.problem, "strategy": self.strategy,
233                 "cleared_channels": sorted(self.cleared), "coverage_complete": complete,
234                 "completion_certified": complete or len(self.cleared) == 16,
235                 "termination_reason": reason, "coverage_points_visited": len(self.coverage_visited),
236                 "detected_channels": sorted(set(self.regions) | self.cleared),
237                 "config": asdict(self.config), **self.stats}

```

程序 13: problem3/run.py

```

1  """Shared local/online command line; local is the safe reproducible default."""
2  from __future__ import annotations
3
4  import argparse
5  import html
6  import json

```

```

7 import math
8 import sys
9 import time
10 import uuid
11 from datetime import datetime
12 from pathlib import Path
13
14 ROOT = Path(__file__).resolve().parents[1]
15
16
17 def draw_route(case, events, destination):
18     points = [tuple(e['position']) for e in events]
19     points = [(0, 0)] + [p for i, p in enumerate(points) if i == 0 or p != points[i-1]]
20     limit = max([2300] + [max(abs(x), abs(y)) + 150 for x, y in points])
21     def xy(p):
22         return (410 + 350 * p[0] / limit, 400 - 350 * p[1] / limit)
23     def circ(p, r, color, extra=''):
24         x, y = xy(p)
25         return f'<circle cx="{x:.2f}" cy="{y:.2f}" r="{r:.2f}" fill="{color}" {extra}/>'
26     coords = ' '.join(f'{x:.2f},{y:.2f}' for x, y in map(xy, points))
27     lines = ['<svg xmlns="http://www.w3.org/2000/svg" width="820" height="840" viewBox="0 0 820 840">',
28             '<rect width="820" height="840" fill="#fafafa"/>',
29             f'<text x="28" y="32" font-family="sans-serif"
30             ↪ font-size="20">{html.escape(case["case_id"])}</text>',
31             '<text x="28" y="58" font-family="sans-serif" font-size="13">Synthetic local case; blue:
32             ↪ path, red: sources, arrows: emission orientation</text>',
33             circ((0, 0), 1800 * 350 / limit, 'none', 'stroke="#aaa" stroke-dasharray="5 4"'),
34             f'<polyline points="{coords}" fill="none" stroke="#3977bc" stroke-width="1.3"
35             ↪ opacity=".65"/>']
36     for s in case['sources']:
37         p = (s['x'], s['y']); x, y = xy(p)
38         lines.append(circ(p, 4, '#c64343'))
39         lines.append(f'<text x="{x+6:.2f}" y="{y-6:.2f}" font-family="sans-serif"
40         ↪ font-size="11">{s["channel"]}</text>')
41         if s['orientation_deg'] is not None:
42             a = math.radians(s['orientation_deg'])
43             dx, dy = 20 * math.cos(a), -20 * math.sin(a)
44             lines.append(f'<path d="M{x:.2f},{y:.2f} l{dx:.2f},{dy:.2f}" stroke="#c64343"
45             ↪ stroke-width="2"/>')
46     lines += [circ((0, 0), 5, '#111'), '<text x="28" y="815" font-family="sans-serif" font-size="13">Black:
47     ↪ start (0,0). Dashed circle: 1800 m target boundary. Truth is shown only after simulation.</text>',
48     ↪ '</svg>']
49     destination.write_text('\n'.join(lines) + '\n', encoding='utf-8')
50
51
52 def main(problem):
53     parser = argparse.ArgumentParser(description=f'问题{problem}: 默认运行本地自建样本;
54     ↪ --online连接已就绪的平台')
55     parser.add_argument('--online', action='store_true')
56     parser.add_argument('--robot-id')
57     parser.add_argument('--base-url', default='http://127.0.0.1:2026')
58     strategies = ('adaptive', 'legacy', 'baseline', 'optical') if problem == 4 else
59     ↪ ('adaptive', 'baseline', 'optical')
60     parser.add_argument('--strategy', choices=strategies, default='adaptive')
61     parser.add_argument('--case', type=Path, help='Local case JSON; relative to this problem directory')

```



```

53     parser.add_argument('--index',type=int,default=3,help='Built-in development case index')
54     parser.add_argument('--config',type=Path,help='Fixed policy config JSON; relative to this problem
    ↳ directory')
55     parser.add_argument('--output-dir',type=Path,help='Output directory; relative to this problem
    ↳ directory')
56     args=parser.parse_args()
57     here=ROOT/f'problem{problem}'
58     def resolve(p):
59         return p if p.is_absolute() else here/p
60     config=json.loads(resolve(args.config).read_text(encoding='utf-8')) if args.config else None
61     output=resolve(args.output_dir) if args.output_dir else here/'results'
62     output.mkdir(parents=True,exist_ok=True)
63     if not args.online:
64         if args.robot_id:
65             parser.error('--robot-id only applies with --online')
66             from problem3.scenarios import generate_case
67             from scripts.benchmark_search import run_case
68             case=json.loads(resolve(args.case).read_text(encoding='utf-8')) if args.case else
    ↳ generate_case(problem,args.index)
69             if case.get('problem') != problem:
70                 parser.error('The selected case belongs to a different problem')
71             if not args.case:
72                 examples=here/'examples'; examples.mkdir(parents=True,exist_ok=True)
73                 (examples/'demo_case.json').write_text(json.dumps(case,ensure_ascii=False,indent=2)+'\n',enco
    ↳ ding='utf-8')
74             row,events=run_case(case,args.strategy,config,record=True)
75             (output/'demo_result.json').write_text(json.dumps(row,ensure_ascii=False,indent=2)+'\n',encoding=
    ↳ 'utf-8')
76             (output/'demo_trace.jsonl').write_text(''.join(json.dumps(e,ensure_ascii=False)+'\n' for e in
    ↳ events),encoding='utf-8')
77             draw_route(case,events,output/'demo_route.svg')
78             average = ('无定义' if row['average_clear_time_s'] is None else
79                 f"{row['average_clear_time_s']:.1f} 秒/源")
80             print(f" 本地自建案例 {case['case_id']}: 清除 {row['cleared_count']}/{row['source_count']}, "
81                 f" 总虚拟耗时 {row['virtual_time_s']:.1f} 秒, 平均 {average}, "
82                 f" 完成确认 ={row['completion_certified']}.")
83             print(f" 结果: {output/'demo_result.json'}")
84             if row['error']:
85                 print(row['error'],file=sys.stderr)
86             return 0 if row['certified_full_clear'] else 1
87     if not args.robot_id:
88         parser.error('--online requires --robot-id with the logged-in team ID')
89     if args.case:
90         parser.error('--case cannot be combined with --online')
91     from problem3.client import HttpClient, BudgetExceeded
92     from scripts.benchmark_search import get_policy
93     stamp=datetime.now().strftime('%Y%m%d_%H%M%S')+ '_' +uuid.uuid4().hex[:8]
94     live=output/'online'; live.mkdir(parents=True,exist_ok=True)
95     client=HttpClient(args.base_url,args.robot_id,live/f'{stamp}_client_trace.jsonl')
96     policy=get_policy(problem,args.strategy,config)
97     started=time.perf_counter()
98     result={}
99     failure=None
100     exited=False
101     entered=False

```

```

102     try:
103         response=client.enter()
104         if response.get('accepted') is not True:
105             raise RuntimeError('平台拒绝进入: 检查所选模块、接口就绪状态和参赛队号')
106         entered=True
107         print(f" 已进入, 实际剩余运行时间 {response['remaining_real_duration_s']} 秒。",flush=True)
108         result=policy.run(client)
109         response=client.exit()
110         exited=response.get('accepted') is True
111         if not exited:
112             raise RuntimeError('平台未接受退出请求')
113     except BudgetExceeded as exc:
114         failure=str(exc)
115         if client.can_exit:
116             try:
117                 exited=client.exit().get('accepted') is True
118             except Exception as exit_error:
119                 failure+=f'; exit: {exit_error}'
120     except (Exception, KeyboardInterrupt) as exc:
121         # Unknown request outcomes must not trigger a different action.
122         failure=f'{type(exc).__name__}: {exc}'
123     count=len(policy.cleared)
124     pending = client.pending_request
125     # Without a confirmed /exit, later accepted-but-unobserved actions or an
126     # automatic timeout may make the platform's final totals differ from these.
127     statistics_final = exited and pending is None
128     observed_average = client.virtual_time_s/count if count else None
129     summary=dict(problem=problem,strategy=args.strategy,provenance='official_platform_client_observations',
130                 platform_case_code=None,test_module='fill_from_platform_ui',source_count=None,
131                 cleared_count=count,cleared_fraction=None,virtual_time_s=client.virtual_time_s,
132                 average_clear_time_s=observed_average if statistics_final else None,
133                 observed_average_clear_time_s=observed_average,
134                 statistics_final=statistics_final,statistics_scope='accepted_responses_only',
135                 pending_request_id=pending.get('request_id') if pending else None,
136                 local_program_runtime_s=time.perf_counter()-started,
137                 platform_program_runtime_s=None,entered=entered,exit_accepted=exited,
138                 policy=result,error=failure,
139                 note='平台案例编码/正式运行时间从界面填写; 演练源总数仅在结束后可见; 正式测试不公开总数。
140                 ↪ 未确认退出时, 虚拟时间与清除数仅是已确认观测, 不能当作平台最终成绩。')
141     dest=live/f'{stamp}_summary.json'
142     dest.write_text(json.dumps(summary,ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
143     print(f'记录已保存: {dest}')
144     print('请在平台确认结束状态, 并导出平台加密日志; 本地记录不能替代官方日志。')
145     if failure:
146         print(failure,file=sys.stderr)
147     return 0 if not failure and exited and
148     ↪ result.get('completion_certified',result.get('coverage_complete')) else 1

```

程序 14: problem3/scenarios.py

```

1  """Diverse deterministic synthetic cases; official hidden data are never used."""
2  from __future__ import annotations
3
4  import math
5  import random

```

```

6
7 FAMILIES = ('uniform', 'clusters', 'boundary', 'outward', 'corridor',
8             'close_pairs', 'grazing', 'short_radius')
9 SPLITS = {'development': (170000, 96), 'holdout': (290000, 192), 'stress': (430000, 96),
10           'development_v2': (610000, 384), 'holdout_v2': (790000, 512),
11           'stress_v2': (930000, 256),
12           'development_v3': (1510000, 384), 'holdout_v3': (1690000, 512),
13           'stress_v3': (1830000, 256)}
14
15
16 def generate_case(problem: int, index: int = 0, split: str = 'development') -> dict:
17     seed = SPLITS[split][0] + index + 1000000*problem
18     rng = random.Random(seed)
19     family = FAMILIES[index % len(FAMILIES)]
20     noise = ('hash_uniform', 'spatial', 'extreme', 'bias')[index//8 % 4]
21     n = 10 + rng.randrange(7)
22     stress = split.startswith('stress')
23     if stress:
24         # Cross each family with BOTH counts and BOTH noise modes, rather than
25         # accidentally coupling a family's index parity to its difficulty.
26         n = 16 if (index // 8) % 2 else 10
27         noise = ('extreme', 'bias')[index // 16 % 2]
28     rotation = rng.uniform(0, 2*math.pi)
29     cluster_centers = [(rng.uniform(600, 1550)*math.cos(a),
30                          rng.uniform(600, 1550)*math.sin(a)) for a in
31                        (rotation, rotation+2.3, rotation+4.5)]
32     channels = rng.sample(range(1, 21), n)
33     directional_count = rng.randint(1, n-1) if problem == 4 else 0
34     if problem == 4 and stress:
35         directional_count = n-1
36     sources = []
37     for j, ch in enumerate(channels):
38         a = rng.uniform(0, 2*math.pi)
39         r = 1800*math.sqrt(rng.random())
40         x, y = r*math.cos(a), r*math.sin(a)
41         if family in ('boundary', 'outward'):
42             a = rotation + 2*math.pi*j/n + rng.uniform(-.06, .06)
43             r = 1800 if stress else rng.uniform(1690, 1800)
44             x, y = r*math.cos(a), r*math.sin(a)
45         elif family == 'clusters':
46             cx, cy = cluster_centers[j % 3]
47             x, y = cx+rng.gauss(0, 70), cy+rng.gauss(0, 70)
48         elif family == 'corridor':
49             length, offset = rng.uniform(-1770, 1770), rng.uniform(-12, 12)
50             x = length*math.cos(rotation)-offset*math.sin(rotation)
51             y = length*math.sin(rotation)+offset*math.cos(rotation)
52         elif family == 'close_pairs' and j % 2:
53             x = sources[-1]['x'] + rng.uniform(-8, 8)
54             y = sources[-1]['y'] + rng.uniform(-8, 8)
55         elif family == 'grazing':
56             r = rng.choice((5.001, 19.999, 999.999, 1499.999, 1799.999))
57             a = rotation + j*2*math.pi/n
58             x, y = r*math.cos(a), r*math.sin(a)
59         length = math.hypot(x, y)
60         if length > 1800:

```

```

61         x, y = x*1800/length, y*1800/length
62         radius = rng.uniform(1000, 1500)
63         if family in ('short_radius', 'outward', 'boundary') or stress:
64             radius = 1000 if stress else rng.uniform(1000, 1030)
65         orient = None
66         if j < directional_count:
67             orient = rng.uniform(0, 360)
68             if family in ('outward', 'boundary'):
69                 orient = math.degrees(math.atan2(y, x)) + rng.uniform(-4, 4)
70             elif family == 'grazing':
71                 orient = math.degrees(math.atan2(-y, -x)) + rng.choice((-90, 90)) + rng.uniform(-.01, .01)
72             orient %= 360
73         sources.append(dict(channel=ch, x=x, y=y, radius=radius, orientation_deg=orient))
74     return dict(case_id=f'local_p{problem}_{split}_{index:04d}', problem=problem,
75               provenance='self_constructed_not_official', seed=seed, split=split,
76               family=family, noise=noise, sources=sources)
77
78
79 def generate_suite(problem: int, split: str, count: int | None = None):
80     return [generate_case(problem, i, split) for i in range(count or SPLITS[split][1])]

```

程序 15: problem3/simulator.py

```

1  """Local synthetic environment, NOT the official simulator.
2
3  Policies receive an ObservationClient facade, never source coordinates/counts.
4  All elapsed times below are virtual seconds except explicitly real_* fields.
5  """
6  from __future__ import annotations
7
8  import hashlib
9  import json
10 import math
11 import time
12 from pathlib import Path
13
14
15 class SimulationLimit(RuntimeError):
16     pass
17
18
19 class LocalSimulator:
20     def __init__(self, case: dict, *, record: bool = False, action_limit: int = 20000):
21         self.case = case
22         self.sources = {s['channel']: dict(s) for s in case['sources']}
23         if len(self.sources) != len(case['sources']):
24             raise ValueError('Channels must be unique')
25         for ch, source in self.sources.items():
26             if type(ch) is not int or not 1 <= ch <= 20:
27                 raise ValueError('Invalid source channel')
28             values = [source['x'], source['y'], source['radius']]
29             if not all(isinstance(v, (int, float)) and not isinstance(v, bool)
30                       and math.isfinite(v) for v in values):
31                 raise ValueError('Invalid source number')
32             orientation = source.get('orientation_deg')

```

```

33         if orientation is not None and (not isinstance(orientation, (int, float))
34             or isinstance(orientation, bool) or not math.isfinite(orientation)):
35             raise ValueError('Invalid source orientation')
36         if math.hypot(source['x'], source['y']) > 1800.000001:
37             raise ValueError('Source outside target disk')
38         if not 1000 <= source['radius'] <= 1500:
39             raise ValueError('Invalid receiving radius')
40         self.position = (0.0, 0.0)
41         self.current_channel = 1
42         self.virtual_time_s = 0.0
43         self.active = False
44         self.started = 0.0
45         self.cleared: set[int] = set()
46         self.clear_times: list[float] = []
47         self.actions = 0
48         self.action_limit = action_limit
49         self.counts = dict(measure=0, clear=0, failed_clear=0, switch=0, no_signal=0)
50         self.movement_m = 0.0
51         self.record = record
52         self.events: list[dict] = []
53
54     def check_budget(self):
55         if not self.active:
56             raise SimulationLimit('Simulation not active')
57         if self.actions >= self.action_limit or self.virtual_time_s >= 360000:
58             raise SimulationLimit('Local action or virtual-time limit reached')
59         if time.perf_counter() - self.started > 1180:
60             raise SimulationLimit('Local wall-clock limit reached')
61
62     def _response(self, **fields):
63         return dict(accepted=True, virtual_time_s=round(self.virtual_time_s, 6),
64             real_timestamp_ms=int(time.time() * 1000), **fields)
65
66     def enter(self):
67         if self.active or self.started:
68             raise SimulationLimit('Enter once per case')
69         self.active = True
70         self.started = time.perf_counter()
71         return self._response(max_virtual_duration_s=360000, max_real_duration_s=1200,
72             remaining_real_duration_s=1200)
73
74     def exit(self):
75         if not self.active:
76             raise SimulationLimit('Simulation not active')
77         self.active = False
78         return self._response(exit_reason='user_exit')
79
80     def _move(self, point, channel):
81         self.check_budget()
82         if type(channel) is not int or not 1 <= channel <= 20:
83             raise ValueError('Channel must be an integer in 1..20')
84         if len(point) != 2 or any(isinstance(v, bool) or not math.isfinite(v) or
85             abs(v) > 2000000 for v in point):
86             raise ValueError('Invalid position')
87         distance = math.dist(self.position, point)

```

```

88         self.movement_m += distance
89         self.virtual_time_s += distance / 5.0
90         self.position = tuple(float(v) for v in point)
91         self.actions += 1
92
93     def _error(self, channel):
94         x, y = self.position
95         key = f"{self.case['seed']}|{channel}|{x:.9f}|{y:.9f}".encode()
96         u = int.from_bytes(hashlib.blake2b(key, digest_size=8).digest(), 'big') / (2**64-1)
97         mode = self.case.get('noise', 'hash_uniform')
98         if mode == 'extreme':
99             return 1.0 if u >= .5 else -1.0
100         if mode == 'bias':
101             return (1.0 if channel % 2 else -1.0) * (.8 + .2*u)
102         if mode == 'spatial':
103             return math.sin(x/145.0 + y/217.0 + self.case['seed'] + channel)
104         return 2*u - 1
105
106     def _log(self, action, channel, response):
107         if self.record:
108             self.events.append(dict(action=action, position=self.position,
109                                     channel=channel, response=response))
110         return response
111
112     def measure(self, point, channel):
113         self._move(point, channel)
114         switched = channel != self.current_channel
115         self.counts['switch'] += int(switched)
116         self.current_channel = channel
117         self.virtual_time_s += 5 + int(switched)
118         self.counts['measure'] += 1
119         result = 'no_signal'
120         source = self.sources.get(channel)
121         fields = {}
122         if source and channel not in self.cleared:
123             dx, dy = self.position[0] - source['x'], self.position[1] - source['y']
124             dist = math.hypot(dx, dy)
125             orient = source.get('orientation_deg')
126             visible = orient is None or (dx*math.cos(math.radians(orient)) +
127                                         dy*math.sin(math.radians(orient))) >= -1e-9
128             if dist <= source['radius'] + 1e-9 and visible:
129                 if dist <= 5:
130                     result = 'near'
131                 else:
132                     result = 'direction'
133                     bearing = math.degrees(math.atan2(-dy, -dx))
134                     fields['svd_deg'] = round((bearing + self._error(channel)) % 360, 2) % 360
135             self.counts['no_signal'] += int(result == 'no_signal')
136             return self._log('measure', channel, self._response(measure_result=result, **fields))
137
138     def clear(self, point, channel):
139         self._move(point, channel)
140         source = self.sources.get(channel)
141         success = bool(source and channel not in self.cleared and
142                        math.dist(self.position, (source['x'], source['y'])) <= 20 + 1e-9)

```

```

143         self.virtual_time_s += 5 if success else 3
144         self.counts['clear'] += 1
145         self.counts['failed_clear'] += int(not success)
146         if success:
147             self.cleared.add(channel)
148             self.clear_times.append(self.virtual_time_s)
149         return self._log('clear', channel, self._response(
150             clear_result='success' if success else 'no_target_in_range'))
151
152     def statistics(self):
153         n, k = len(self.sources), len(self.cleared)
154         return dict(source_count=n, cleared_count=k, cleared_fraction=k/n if n else 1.0,
155                     virtual_time_s=self.virtual_time_s,
156                     average_clear_time_s=self.virtual_time_s/k if k else None,
157                     last_clear_time_s=self.clear_times[-1] if k else None,
158                     time_to_90_percent_s=(self.clear_times[math.ceil(.9*n)-1]
159                                             if k >= math.ceil(.9*n) and n else None),
160                     movement_m=self.movement_m, actions=self.actions, **self.counts)
161
162
163 class ObservationClient:
164     """Deliberate API boundary; strategy code only sees public observations."""
165     __slots__ = ('__sim',)
166
167     def __init__(self, simulator):
168         self.__sim = simulator
169
170     @property
171     def position(self):
172         return self.__sim.position
173
174     @property
175     def current_channel(self):
176         return self.__sim.current_channel
177
178     @property
179     def virtual_time_s(self):
180         return self.__sim.virtual_time_s
181
182     def check_budget(self):
183         self.__sim.check_budget()
184
185     def measure(self, point, channel):
186         return self.__sim.measure(point, channel)
187
188     def clear(self, point, channel):
189         return self.__sim.clear(point, channel)

```

程序 16: problem3/solve.py

```

1 """Problem 3 entry point, independent of the caller's working directory."""
2 from pathlib import Path
3 import sys
4
5 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

```

```
6 from problem3.run import main
7
8 if __name__ == '__main__':
9     raise SystemExit(main(3))
```

程序 17: problem3/tests/test client.py

```

1  """Protocol tests use an in-process mock; they never contact the official platform."""
2
3  import json
4  import math
5  import socket
6  import tempfile
7  import threading
8  import unittest
9  from contextlib import contextmanager
10 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
11 from pathlib import Path
12
13 from problem3.client import BudgetExceeded, ClientError, HttpClient, ProtocolError, TransportError
14
15
16 class FakeClock:
17     def __init__(self):
18         self.now = 100.0
19
20     def __call__(self):
21         return self.now
22
23
24 @contextmanager
25 def mock_simulator(*, remaining=1200, drop_path=None, reject_path=None,
26                    unknown_path=None, http_error_path=None, on_drop=None, partial_path=None):
27     state = {"requests": [], "cache": {}, "executed": [], "position": (0, 0),
28             "channel": 1, "virtual_time": 0.0, "dropped": False}
29
30     class Handler(BaseHTTPRequestHandler):
31         def log_message(self, *args):
32             pass
33
34         def do_POST(self):
35             raw = self.rfile.read(int(self.headers["Content-Length"]))
36             payload = json.loads(raw)
37             key = payload["request_id"]
38             state["requests"].append((self.path, raw))
39             status = 200
40             if key in state["cache"]:
41                 response = state["cache"][key]
42             elif self.path == reject_path or self.path == http_error_path:
43                 response = {"accepted": False, "real_timestamp_ms": 1000, "virtual_time_s": 0}
44                 status = 400 if self.path == http_error_path else 200
45             else:
46                 state["executed"].append(self.path)
47                 response = {"accepted": True, "real_timestamp_ms": 1000, "virtual_time_s":
48                             ↪ state["virtual_time"]}

```

```

3 import json
4 import math
5 import socket
6 import tempfile
7 import threading
8 import unittest
9 from contextlib import contextmanager
10 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
11 from pathlib import Path
12
13 from problem3.client import BudgetExceeded, ClientError, HttpClient, ProtocolError, TransportError
14
15
16 class FakeClock:
17     def __init__(self):
18         self.now = 100.0
19
20     def __call__(self):
21         return self.now
22
23
24 @contextmanager
25 def mock_simulator(*, remaining=1200, drop_path=None, reject_path=None,
26                    unknown_path=None, http_error_path=None, on_drop=None, partial_path=None):
27     state = {"requests": [], "cache": {}, "executed": [], "position": (0, 0),
28             "channel": 1, "virtual_time": 0.0, "dropped": False}
29
30     class Handler(BaseHTTPRequestHandler):
31         def log_message(self, *args):
32             pass
33
34         def do_POST(self):
35             raw = self.rfile.read(int(self.headers["Content-Length"]))
36             payload = json.loads(raw)
37             key = payload["request_id"]
38             state["requests"].append((self.path, raw))
39             status = 200
40             if key in state["cache"]:
41                 response = state["cache"][key]
42             elif self.path == reject_path or self.path == http_error_path:
43                 response = {"accepted": False, "real_timestamp_ms": 1000, "virtual_time_s": 0}
44                 status = 400 if self.path == http_error_path else 200
45             else:
46                 state["executed"].append(self.path)
47                 response = {"accepted": True, "real_timestamp_ms": 1000, "virtual_time_s":
48                             ↪ state["virtual_time"]}

```

```
5 import socket
```

```
7 import threading
```

```
9 from contextlib import contextmanager
```

```
10 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

```
11 from pathlib import Path
```

```
13 from problem3.client import BudgetExceeded, ClientError, HttpClient, ProtocolError, TransportError
```

15

```
16 class FakeClock:
```

```
17     def __init__(self):
```

19

```
20     def __call__(self):
```

22

24 @contextmanager

```
25 def mock_simulator(*, remaining=1200, drop_path=None, reject_path=None,
```

```
26         unknown_path=None, http_error_path=None, on_drop=None, partial_path=None):
```

```
27     state = {"requests": [], "cache": {}, "executed": [], "position": (0, 0),
```

```
30 class Handler(BaseHTTPRequestHandler):
```

```
31         def log_message(self, *args):
```

32 pass

```
34     def do_POST(self):
```

```
35     raw = self.rfile.read(int(self.headers["Content-Length"]))
```

```
36     payload = json.loads(raw)
```

```
37         key = payload["request_id"]
```

```
38 state["requests"].append((self.path, raw))
```

```
39         status = 200
```

```
40         if key in state["cache"]:
```

```
41         response = state["cache"][key]
```

```
42         elif self.path == reject_path or self.path == http_error_path:
```

```
43     response = {"accepted": False, "real_timestamp_ms": 1000, "virtual_time_s": 0}
```

```
44         status = 400 if self.path == http_error_path else 200
```

```
45         else:
```

```
46         state["executed"].append(self.path)
```

```
47 response = {"accepted": True, "real_timestamp_ms": 1000, "virtual_time_s":
```

```
    ↪ state["virtual time"]}
```



```

48         if self.path == "/enter":
49             response.update(max_virtual_duration_s=360000, max_real_duration_s=1200,
50                             remaining_real_duration_s=remaining)
51         elif self.path in {"/measure", "/clear"}:
52             point = (payload["position"]["x"], payload["position"]["y"])
53             state["virtual_time"] += math.dist(point, state["position"]) / 5
54             state["position"] = point
55             if self.path == "/measure":
56                 state["virtual_time"] += 5 + (payload["channel"] != state["channel"])
57                 state["channel"] = payload["channel"]
58                 response.update(measure_result="direction", svd_deg=123.45)
59             else:
60                 state["virtual_time"] += 3
61                 response.update(clear_result="no_target_in_range")
62                 response["virtual_time_s"] = state["virtual_time"]
63         elif self.path == "/exit":
64             response["exit_reason"] = "user_exit"
65         if self.path == unknown_path:
66             response["measure_result"] = "unexpected"
67             state["cache"][key] = response
68     if self.path == drop_path and not state["dropped"]:
69         state["dropped"] = True
70         if on_drop:
71             on_drop()
72         self.connection.shutdown(socket.SHUT_RDWR)
73         self.connection.close()
74         return
75     data = json.dumps(response).encode("utf-8")
76     self.send_response(status)
77     self.send_header("Content-Type", "application/json")
78     self.send_header("Content-Length", str(len(data)))
79     self.end_headers()
80     if self.path == partial_path and not state["dropped"]:
81         state["dropped"] = True
82         self.wfile.write(data[:len(data) // 2])
83         self.wfile.flush()
84         self.connection.shutdown(socket.SHUT_RDWR)
85         self.connection.close()
86         return
87     self.wfile.write(data)
88
89     server = ThreadingHTTPServer(("127.0.0.1", 0), Handler)
90     thread = threading.Thread(target=server.serve_forever, kwargs={"poll_interval": 0.01}, daemon=True)
91     thread.start()
92     try:
93         yield f"http://127.0.0.1:{server.server_port}", state
94     finally:
95         server.shutdown()
96         server.server_close()
97         thread.join(timeout=2)
98
99
100 class ClientTests(unittest.TestCase):
101     def setUp(self):
102         self.directory = tempfile.TemporaryDirectory()

```

```

103         self.addCleanup(self.directory.cleanup)
104         self.log = Path(self.directory.name) / "session.jsonl"
105
106     def make_client(self, url, **kwargs):
107         return HttpClient(url, "team-123", self.log, retry_delay_s=0, **kwargs)
108
109     def test_protocol_timing_example_and_clear_does_not_switch_channel(self):
110         with mock_simulator() as (url, _):
111             client = self.make_client(url)
112             self.assertTrue(client.enter()["accepted"])
113             self.assertEqual(client.measure((300, 400), 1)["virtual_time_s"], 105)
114             self.assertEqual(client.measure((300, 400), 2)["virtual_time_s"], 111)
115             self.assertEqual(client.clear((300, 0), 3)["virtual_time_s"], 194)
116             self.assertEqual(client.current_channel, 2)
117             self.assertEqual(client.position, (300.0, 0.0))
118             self.assertEqual(client.measure((300, 0), 2)["virtual_time_s"], 199)
119             self.assertEqual(client.exit()["exit_reason"], "user_exit")
120             self.assertFalse(client.can_exit)
121
122     def test_accepted_but_lost_response_reuses_identical_request(self):
123         with mock_simulator(drop_path="/measure") as (url, state):
124             client = self.make_client(url)
125             client.enter()
126             result = client.measure((300, 400), 2)
127             self.assertEqual(result["virtual_time_s"], 106)
128             requests = [r for r in state["requests"] if r[0] == "/measure"]
129             self.assertEqual(len(requests), 2)
130             self.assertEqual(requests[0], requests[1])
131             self.assertEqual(state["executed"].count("/measure"), 1)
132             self.assertEqual(client.virtual_time_s, 106.0)
133
134     def test_exhausted_retry_keeps_pending_and_blocks_new_action(self):
135         with mock_simulator(drop_path="/measure") as (url, state):
136             client = self.make_client(url, max_retries=0)
137             client.enter()
138             with self.assertRaises(TransportError):
139                 client.measure((30, 0), 1)
140             original = client.pending_request
141             self.assertFalse(client.can_exit)
142             with self.assertRaises(TransportError):
143                 client.clear((30, 0), 1)
144             with self.assertRaises(TransportError):
145                 client.exit()
146             self.assertEqual(client.retry_pending()["virtual_time_s"], 11)
147             self.assertIsNone(client.pending_request)
148             self.assertEqual(json.loads(state["requests"])[-1][1], original)
149             self.assertEqual(state["executed"].count("/measure"), 1)
150
151     def test_truncated_response_is_retried_as_the_same_action(self):
152         with mock_simulator(partial_path="/clear") as (url, state):
153             client = self.make_client(url)
154             client.enter()
155             self.assertEqual(client.clear((30, 0), 1)["clear_result"], "no_target_in_range")
156             self.assertEqual(client.virtual_time_s, 9)
157             self.assertEqual(state["executed"].count("/clear"), 1)

```

```

158         requests = [r for r in state["requests"] if r[0] == "/clear"]
159         self.assertEqual(requests[0], requests[1])
160
161     def test_rejection_zero_does_not_reset_accepted_state(self):
162         with mock_simulator(reject_path="/clear") as (url, _):
163             client = self.make_client(url)
164             client.enter()
165             client.measure((12, 1), 2)
166             before = (client.virtual_time_s, client.position, client.current_channel)
167             self.assertFalse(client.clear((300, 400), 5)["accepted"])
168             self.assertEqual((client.virtual_time_s, client.position, client.current_channel), before)
169
170     def test_http_error_is_not_an_accepted_action(self):
171         with mock_simulator(http_error_path="/measure") as (url, _):
172             client = self.make_client(url)
173             client.enter()
174             with self.assertRaises(ProtocolError):
175                 client.measure((100, 100), 2)
176             self.assertEqual(client.virtual_time_s, 0)
177             self.assertEqual(client.position, (0, 0))
178             self.assertIsNone(client.pending_request)
179
180     def test_unknown_result_is_not_silently_no_signal(self):
181         with mock_simulator(unknown_path="/measure") as (url, _):
182             client = self.make_client(url)
183             client.enter()
184             with self.assertRaises(ProtocolError):
185                 client.measure((0, 0), 1)
186             self.assertEqual(client.virtual_time_s, 0)
187             self.assertIsNotNone(client.pending_request)
188
189     def test_remaining_duration_and_exit_reserve(self):
190         clock = FakeClock()
191         with mock_simulator(remaining=13) as (url, state):
192             client = self.make_client(url, clock=clock, exit_reserve_s=5)
193             client.enter()
194             self.assertEqual(client.check_budget(), 8)
195             clock.now += 8
196             with self.assertRaises(BudgetExceeded):
197                 client.measure((0, 0), 1)
198             self.assertEqual(state["executed"], ["/enter"])
199             self.assertTrue(client.can_exit)
200             self.assertTrue(client.exit()["accepted"])
201
202     def test_enter_lost_response_does_not_extend_deadline(self):
203         clock = FakeClock()
204         with mock_simulator(remaining=20, drop_path="/enter",
205                             on_drop=lambda: setattr(clock, "now", clock.now + 7)) as (url, _):
206             client = self.make_client(url, clock=clock)
207             client.enter()
208             self.assertEqual(client.check_budget(), 8)
209
210     def test_zero_budget_does_not_probe_exit(self):
211         with mock_simulator(remaining=0) as (url, state):
212             client = self.make_client(url)

```

```

213         client.enter()
214         self.assertFalse(client.can_exit)
215         with self.assertRaises(BudgetExceeded):
216             client.exit()
217         self.assertEqual(state["executed"], ["/enter"])
218
219     def test_no_overwrite_of_existing_log(self):
220         self.log.write_text("original\n", encoding="utf-8")
221         with self.assertRaises(FileExistsError):
222             self.make_client("http://127.0.0.1:2026")
223         self.assertEqual(self.log.read_text(encoding="utf-8"), "original\n")
224
225     def test_invalid_input_is_rejected_before_network(self):
226         with mock_simulator() as (url, state):
227             client = self.make_client(url)
228             with self.assertRaises(ClientError):
229                 client.measure((0, 0), 1)
230             client.enter()
231             for point, channel in [((float("nan"), 0), 1), ((2000001, 0), 1), ((0, 0), True), ((0, 0),
↵ 21)]]:
232                 with self.assertRaises(ValueError):
233                     client.measure(point, channel)
234             self.assertEqual(state["executed"], ["/enter"])
235
236     def test_concurrent_callers_are_serialized_and_ids_are_new(self):
237         with mock_simulator() as (url, state):
238             client = self.make_client(url)
239             client.enter()
240             errors = []
241             def worker(channel):
242                 try:
243                     client.measure((0, 0), channel)
244                 except Exception as error:
245                     errors.append(error)
246             threads = [threading.Thread(target=worker, args=(channel,)) for channel in range(1, 6)]
247             for thread in threads:
248                 thread.start()
249             for thread in threads:
250                 thread.join(timeout=3)
251             self.assertEqual(errors, [])
252             self.assertTrue(all(not thread.is_alive() for thread in threads))
253             ids = [json.loads(raw)["request_id"] for _, raw in state["requests"]]
254             self.assertEqual(len(ids), len(set(ids)))
255             self.assertEqual(client.virtual_time_s, state["virtual_time"])
256
257
258 if __name__ == "__main__":
259     unittest.main()

```

程序 18: problem3/tests/test_http_integration.py

```

1 """Full policy + CLI + HTTP regression on isolated synthetic development cases.
2
3 The server below is a small localhost test mirror, not the official simulator.
4 It exposes only the four public response shapes and never reveals source truth.

```

```

5 All child-process outputs are isolated in temporary directories.
6 """
7
8 from __future__ import annotations
9
10 import json
11 import os
12 import subprocess
13 import sys
14 import tempfile
15 import threading
16 import unittest
17 from contextlib import contextmanager
18 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
19 from pathlib import Path
20
21 from problem3.scenarios import generate_case
22 from problem3.simulator import LocalSimulator, SimulationLimit
23
24
25 ROOT = Path(__file__).resolve().parents[2]
26
27
28 @contextmanager
29 def public_http_mirror(case):
30     simulator = LocalSimulator(case)
31     state = {'requests': [], 'responses': [], 'cache': {}, 'active': 0,
32             'maximum_active': 0, 'handler_errors': []}
33     state_lock = threading.Lock()
34
35     class Handler(BaseHTTPRequestHandler):
36         def log_message(self, *args):
37             pass
38
39         def do_POST(self):
40             counting = True
41             with state_lock:
42                 state['active'] += 1
43                 state['maximum_active'] = max(state['maximum_active'], state['active'])
44             try:
45                 raw = self.rfile.read(int(self.headers['Content-Length']))
46                 payload = json.loads(raw)
47                 state['requests'].append((self.path, payload))
48                 request_id = payload['request_id']
49                 status = 200
50                 rejected = dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
51                 if self.path not in {'/enter', '/measure', '/clear', '/exit'}:
52                     status, result = 404, rejected
53                 elif payload.get('robot_id') != 'local-test' or payload.get('arena_id') != 'default':
54                     result = rejected
55                 elif request_id in state['cache']:
56                     prior_path, prior_raw, prior_result = state['cache'][request_id]
57                     if (self.path, raw) != (prior_path, prior_raw):
58                         status, result = 409, rejected
59                 else:

```

```

60         result = prior_result
61     else:
62         try:
63             if self.path in {'/enter', '/exit'}:
64                 result = getattr(simulator, self.path[1:])()
65             else:
66                 point = payload['position']
67                 result = getattr(simulator, self.path[1:])((point['x'], point['y']),
68                     ↪ payload['channel'])
69             except SimulationLimit:
70                 result = rejected
71             if result['accepted'] is True:
72                 state['cache'][request_id] = self.path, raw, result
73                 state['responses'].append((self.path, result))
74                 # A next serial request may arrive as soon as its predecessor's
75                 # response body reaches the child process, before this handler
76                 # returns. Count overlapping action execution, not that harmless
77                 # handler cleanup interval; the JSONL assertions verify the full
78                 # request/response order independently below.
79                 with state_lock:
80                     state['active'] -= 1
81                     counting = False
82                 data = json.dumps(result).encode('utf-8')
83                 self.send_response(status)
84                 self.send_header('Content-Type', 'application/json')
85                 self.send_header('Content-Length', str(len(data)))
86                 self.end_headers()
87                 self.wfile.write(data)
88             except Exception as error:
89                 state['handler_errors'].append(repr(error))
90                 raise
91         finally:
92             if counting:
93                 with state_lock:
94                     state['active'] -= 1
95
96     server = ThreadingHTTPServer(('127.0.0.1', 0), Handler)
97     thread = threading.Thread(target=server.serve_forever, kwargs={'poll_interval': .01}, daemon=True)
98     thread.start()
99     try:
100         yield f'http://127.0.0.1:{server.server_port}', simulator, state
101     finally:
102         server.shutdown()
103         server.server_close()
104         thread.join(timeout=3)
105
106 class HttpEndToEndTests(unittest.TestCase):
107     def run_problem(self, problem, index):
108         # Short reception radii and fixed extreme errors; P4 additionally has
109         # outward-facing boundary sources. These are development cases only.
110         case = generate_case(problem, index, 'development')
111         self.assertTrue(10 <= len(case['sources']) <= 16)
112         with tempfile.TemporaryDirectory(prefix=f'p{problem}_http_test_') as folder:
113             destination = Path(folder)

```

```

114         with public_http_mirror(case) as (base_url, simulator, state):
115             environment = dict(os.environ, PYTHONIOENCODING='utf-8')
116             process = subprocess.run(
117                 [sys.executable, str(ROOT/f'problem{problem}/solve.py'), '--online',
118                  '--robot-id', 'local-test', '--base-url', base_url,
119                  '--output-dir', str(destination/'outputs')],
120                 cwd=destination, capture_output=True, text=True, encoding='utf-8',
121                 env=environment, timeout=60)
122             self.assertEqual(process.returncode, 0, process.stdout+'\n'+process.stderr)
123             self.assertEqual(state['handler_errors'], [])
124             self.assertEqual(state['maximum_active'], 1)
125             self.assertEqual(len(simulator.cleared), len(case['sources']))
126             self.assertFalse(simulator.active)
127             self.assertEqual(state['requests'][0][0], '/enter')
128             self.assertEqual(state['requests'][-1][0], '/exit')
129             ids = [payload['request_id'] for _, payload in state['requests']]
130             self.assertEqual(len(ids), len(set(ids)), 'Stable local transport should need no retries')
131             for path, response in state['responses']:
132                 self.assertTrue(response['accepted'], (path, response))
133                 self.assertFalse({'sources', 'source_count', 'radius', 'orientation_deg'} & set(response))
134             summaries = list((destination/'outputs/online').glob('*_summary.json'))
135             self.assertEqual(len(summaries), 1)
136             summary = json.loads(summaries[0].read_text(encoding='utf-8'))
137             self.assertTrue(summary['entered'])
138             self.assertTrue(summary['exit_accepted'])
139             self.assertTrue(summary['statistics_final'])
140             self.assertIsNone(summary['error'])
141             self.assertIsNone(summary['source_count'])
142             self.assertIsNone(summary['cleared_fraction'])
143             self.assertIsNone(summary['platform_case_code'])
144             self.assertIsNone(summary['platform_program_runtime_s'])
145             self.assertEqual(summary['cleared_count'], len(case['sources']))
146             self.assertAlmostEqual(summary['virtual_time_s'], simulator.virtual_time_s, places=5)
147             self.assertTrue(summary['policy']['completion_certified'])
148             logs = list((destination/'outputs/online').glob('*_client_trace.jsonl'))
149             self.assertEqual(len(logs), 1)
150             records = [json.loads(line) for line in logs[0].read_text(encoding='utf-8').splitlines()]
151             flow = [record for record in records if record['event'] != 'client_created']
152             self.assertEqual(len(flow), 2*len(state['requests']))
153             for request, response in zip(flow[:2], flow[1:2]):
154                 self.assertEqual(request['event'], 'request')
155                 self.assertEqual(response['event'], 'response')
156                 self.assertEqual(request['payload']['request_id'], response['request_id'])
157                 self.assertEqual(request['path'], response['path'])
158
159     def test_problem3_full_online_cli_on_local_synthetic_http(self):
160         self.run_problem(3, 18)
161
162     def test_problem4_full_online_cli_on_local_synthetic_http(self):
163         self.run_problem(4, 19)
164
165
166 if __name__ == '__main__':
167     unittest.main()

```

程序 19: problem3/tests/test_policy.py

```
1 """Independent geometric and black-box clearance checks."""
2 import math
3 import random
4 import unittest
5
6 from problem3.geometry import (certified_covering_radius, contains, distance, enclosing_circle,
7     ↪ initial_region,
8     intersect_bearing, optical_cover)
9
10 from problem3.policy import SearchPolicy
11
12 class GeometryTests(unittest.TestCase):
13     def test_quantized_adversarial_bearings_retain_truth(self):
14         rng = random.Random(49016)
15         for _ in range(400):
16             angle = rng.uniform(0, 2 * math.pi)
17             radius = rng.uniform(0, 1800)
18             truth = (radius * math.cos(angle), radius * math.sin(angle))
19             polygon = initial_region()
20             for _ in range(5):
21                 a = rng.uniform(0, 2 * math.pi)
22                 d = rng.uniform(5.001, 1500)
23                 receiver = (truth[0] - d * math.cos(a), truth[1] - d * math.sin(a))
24                 bearing = round((math.degrees(a) + rng.choice((-1.0, 1.0))) % 360, 2)
25                 polygon = intersect_bearing(polygon, receiver, bearing)
26                 self.assertTrue(contains(polygon, truth))
27                 center, bound = enclosing_circle(polygon)
28                 self.assertLessEqual(distance(center, truth), bound + 1e-5)
29
30     def test_ring_coverage_certificate_and_dense_boundary(self):
31         points = SearchPolicy().coverage_points()
32         self.assertEqual(len(points), 7)
33         for radial in range(0, 1801, 60):
34             for step in range(720):
35                 angle = step * math.pi / 360
36                 source = radial * math.cos(angle), radial * math.sin(angle)
37                 self.assertLessEqual(min(distance(source, p) for p in points), 1000)
38
39     def test_enclosing_circle_equilateral_not_half_diameter(self):
40         vertices = [(0.0, 0.0), (40.0, 0.0), (20.0, 20 * math.sqrt(3))]
41         _, radius = enclosing_circle(vertices)
42         self.assertAlmostEqual(radius, 40 / math.sqrt(3), places=5)
43         self.assertGreater(radius, 20)
44
45     def test_voronoi_cover_certificate(self):
46         ring = SearchPolicy().coverage_points()
47         self.assertLess(certified_covering_radius(ring), 1000)
48         self.assertGreater(certified_covering_radius([(0, 0)]), 1800)
49         self.assertGreater(certified_covering_radius(ring[:-1]), 1000)
50
51     def test_rotated_optical_cover_contains_every_sample(self):
52         rng = random.Random(499)
53         for bearing in (0, 0.005, 42.37, 90, 180, 359.995):
```



```

53         polygon = intersect_bearing(initial_region(), (300, -100), bearing)
54         cover = optical_cover(polygon)
55         for _ in range(250):
56             weights = [rng.expovariate(1) for _ in polygon]
57             total = sum(weights)
58             point = (sum(p[0] * w for p, w in zip(polygon, weights)) / total,
59                     sum(p[1] * w for p, w in zip(polygon, weights)) / total)
60             self.assertLessEqual(min(distance(point, p) for p in cover), 20)
61         for point in polygon:
62             self.assertLessEqual(min(distance(point, p) for p in cover), 20)
63
64
65 class ProtocolOnlyClient:
66     """A one-source fake with no public true-position API."""
67     def __init__(self, point, bearing_error=1.0, directional=False):
68         self.__point = point
69         self.__error = bearing_error
70         self.__directional = directional
71         self.__cleared = False
72         self.position = (0., 0.)
73         self.current_channel = 1
74         self.virtual_time_s = 0.
75         self.actions = 0
76
77     def check_budget(self):
78         if self.actions > 1500:
79             raise RuntimeError("Unexpected policy loop")
80
81     def measure(self, point, channel):
82         self.virtual_time_s += distance(self.position, point) / 5 + 5 + (channel != self.current_channel)
83         self.position, self.current_channel = point, channel
84         self.actions += 1
85         d = distance(point, self.__point)
86         if self.__cleared or channel != 7 or d > 1000 or self.__directional and point[1] > self.__point[1]:
87             return {"accepted": True, "measure_result": "no_signal"}
88         if d <= 5:
89             return {"accepted": True, "measure_result": "near"}
90         angle = math.degrees(math.atan2(self.__point[1] - point[1], self.__point[0] - point[0]))
91         return {"accepted": True, "measure_result": "direction", "svd_deg": round((angle + self.__error) %
92             ↪ 360, 2)}
93
94     def clear(self, point, channel):
95         success = not self.__cleared and channel == 7 and distance(point, self.__point) <= 20
96         self.virtual_time_s += distance(self.position, point) / 5 + (5 if success else 3)
97         self.position = point
98         self.actions += 1
99         self.__cleared |= success
100         return {"accepted": True, "clear_result": "success" if success else "no_target_in_range"}
101
102 class PolicyTests(unittest.TestCase):
103     def test_end_to_end_adversarial_error_and_edge_positions(self):
104         for strategy in ("baseline", "adaptive", "optical"):
105             for point in ((0, 0), (1799, 0), (-900, 1558), (650, -500)):
106                 for error in (-1, 1):

```

```

107         result = SearchPolicy(strategy=strategy).run(ProtocolOnlyClient(point, error))
108         self.assertEqual(result["cleared_channels"], [7])
109         self.assertTrue(result["coverage_complete"])
110
111     def test_directional_localization_remains_finite(self):
112         client = ProtocolOnlyClient((650, 0), directional=True)
113         policy = SearchPolicy()
114         policy._measure(client, (0., 0.), 7)
115         self.assertTrue(policy._localize(7, client))
116         self.assertIn(7, policy.cleared)
117
118     def test_near_clears_without_bearing(self):
119         client = ProtocolOnlyClient((3, 0))
120         policy = SearchPolicy()
121         self.assertEqual(policy._measure(client, (0, 0), 7), "near")
122         self.assertIn(7, policy.cleared)
123
124     def test_directional_negative_does_not_discard_true_region(self):
125         client = ProtocolOnlyClient((650, 0), directional=True)
126         policy = SearchPolicy()
127         policy._measure(client, (0, 0), 7)
128         original = list(policy.regions[7])
129         self.assertEqual(policy._measure(client, (600, 100), 7), "no_signal")
130         self.assertEqual(policy.regions[7], original)
131
132     def test_rejected_action_never_becomes_a_reading(self):
133         class RejectedClient(ProtocolOnlyClient):
134             def measure(self, point, channel):
135                 return {"accepted": False, "virtual_time_s": 0}
136         with self.assertRaisesRegex(RuntimeError, "not accepted"):
137             SearchPolicy().run(RejectedClient((0, 0)))
138
139     def test_sixteen_clearances_certify_early_stop(self):
140         class NearClient(ProtocolOnlyClient):
141             def measure(self, point, channel):
142                 return {"accepted": True, "measure_result": "near" if channel <= 16 else "no_signal"}
143             def clear(self, point, channel):
144                 return {"accepted": True, "clear_result": "success"}
145         report = SearchPolicy().run(NearClient((0, 0)))
146         self.assertEqual(len(report["cleared_channels"]), 16)
147         self.assertTrue(report["completion_certified"])
148         self.assertFalse(report["coverage_complete"])
149
150
151 if __name__ == "__main__":
152     unittest.main()

```

程序 20: problem3/tests/test_runner.py

```

1 """Local/online runner integration and failure-reporting boundaries."""
2
3 import contextlib
4 import io
5 import json
6 import os

```

```

7 import subprocess
8 import sys
9 import tempfile
10 import unittest
11 from pathlib import Path
12 from unittest.mock import Mock, patch
13
14 from problem3 import run
15 from problem3.client import BudgetExceeded, TransportError
16 from problem3.scenarios import generate_case
17 from problem3.simulator import LocalSimulator
18 from scripts import benchmark_search
19
20
21 class RunnerTests(unittest.TestCase):
22     def setUp(self):
23         self.directory = tempfile.TemporaryDirectory()
24         self.addCleanup(self.directory.cleanup)
25         self.destination = Path(self.directory.name)
26
27     def online(self, *, failure=None, enter_accepted=True, exit_accepted=True,
28               pending=None, can_exit=True):
29         client = Mock()
30         client.virtual_time_s = 0.0
31         client.pending_request = None
32         client.can_exit = can_exit
33         client.enter.return_value = dict(accepted=enter_accepted, remaining_real_duration_s=37)
34         client.exit.return_value = dict(accepted=exit_accepted, exit_reason='user_exit')
35         policy = Mock()
36         policy.cleared = set()
37
38     def policy_run(observations):
39         self.assertIs(observations, client)
40         client.virtual_time_s = 46.25
41         client.pending_request = pending
42         policy.cleared.add(7)
43         if failure:
44             raise failure
45         return dict(completion_certified=True, coverage_complete=True)
46
47     policy.run.side_effect = policy_run
48     args = ['solve.py', '--online', '--robot-id', 'test-team', '--output-dir', str(self.destination)]
49     with patch.object(sys, 'argv', args), patch('problem3.client.HttpClient', return_value=client), \
50         patch.object(benchmark_search, 'get_policy', return_value=policy), \
51         contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):
52         code = run.main(3)
53         files = list((self.destination/'online').glob('*_summary.json'))
54         self.assertEqual(len(files), 1)
55         return code, json.loads(files[0].read_text(encoding='utf-8')), client, policy
56
57     def test_success_keeps_platform_truth_and_runtime_unknown(self):
58         code, result, client, _ = self.online()
59         self.assertEqual(code, 0)
60         self.assertTrue(result['statistics_final'])
61         self.assertEqual(result['cleared_count'], 1)

```

```

62         self.assertEqual(result['average_clear_time_s'], 46.25)
63         self.assertIsNone(result['source_count'])
64         self.assertIsNone(result['cleared_fraction'])
65         self.assertIsNone(result['platform_program_runtime_s'])
66         self.assertIsNone(result['platform_case_code'])
67         client.exit.assert_called_once()
68
69     def test_unknown_outcome_does_not_exit_or_publish_final_average(self):
70         code, result, client, _ = self.online(failure=TransportError('lost response'),
71                                               pending={'request_id': 'measure-uncertain'}, can_exit=False)
72         self.assertEqual(code, 1)
73         client.exit.assert_not_called()
74         self.assertFalse(result['statistics_final'])
75         self.assertIsNone(result['average_clear_time_s'])
76         self.assertEqual(result['observed_average_clear_time_s'], 46.25)
77         self.assertEqual(result['pending_request_id'], 'measure-uncertain')
78
79     def test_budget_uses_exit_reserve_when_exit_is_safe(self):
80         code, result, client, _ = self.online(failure=BudgetExceeded('reserve reached'))
81         self.assertEqual(code, 1)
82         client.exit.assert_called_once()
83         self.assertTrue(result['statistics_final'])
84         self.assertTrue(result['exit_accepted'])
85         self.assertIn('reserve reached', result['error'])
86
87     def test_rejected_enter_does_not_run_policy_or_probe_exit(self):
88         code, result, client, policy = self.online(enter_accepted=False)
89         self.assertEqual(code, 1)
90         self.assertFalse(result['entered'])
91         self.assertEqual(result['cleared_count'], 0)
92         policy.run.assert_not_called()
93         client.exit.assert_not_called()
94
95     def test_rejected_exit_is_not_reissued_as_a_new_request(self):
96         code, result, client, _ = self.online(exit_accepted=False)
97         self.assertEqual(code, 1)
98         self.assertFalse(result['statistics_final'])
99         self.assertIsNone(result['average_clear_time_s'])
100        client.exit.assert_called_once()
101
102     def test_relative_local_paths_are_based_on_problem_directory_from_outside(self):
103         root = self.destination/'repository'
104         problem = root/'problem3'
105         examples = problem/'examples'
106         examples.mkdir(parents=True)
107         case = generate_case(3)
108         (examples/'case.json').write_text(json.dumps(case), encoding='utf-8')
109         row = dict(cleared_count=0, source_count=10, virtual_time_s=10.0,
110                  average_clear_time_s=None, completion_certified=False,
111                  certified_full_clear=False, error='injected failure')
112         output = io.StringIO()
113         original = Path.cwd()
114         try:
115             os.chdir(self.destination)
116             with patch.object(sys, 'argv', ['solve.py', '--case', 'examples/case.json'],

```

```

117         '--output-dir', 'results/check']], \
118         patch.object(run, 'ROOT', root), \
119         patch.object(benchmark_search, 'run_case', return_value=(row, [])) as local_run, \
120         patch('problem3.client.HttpClient', side_effect=AssertionError('Local mode contacted
↪ network')), \
121         contextlib.redirect_stdout(output), contextlib.redirect_stderr(io.StringIO()):
122     self.assertEqual(run.main(3), 1)
123     finally:
124         os.chdir(original)
125     self.assertEqual(local_run.call_args.args[0]['case_id'], case['case_id'])
126     self.assertTrue((problem/'results/check/demo_result.json').exists())
127     self.assertTrue((problem/'results/check/demo_route.svg').exists())
128     self.assertFalse((self.destination/'results').exists())
129     self.assertIn('无定义', output.getvalue())
130
131     def test_solver_wrappers_import_from_outside_repository(self):
132         for problem in (3, 4):
133             process = subprocess.run([sys.executable, str(run.ROOT/f'problem{problem}/solve.py'),
↪ '--help'],
134                                     cwd=self.destination, capture_output=True, timeout=15)
135             self.assertEqual(process.returncode, 0, process.stderr.decode('utf-8', errors='replace'))
136             self.assertIn(b'--online', process.stdout)
137
138     def test_total_failure_is_retained_in_aggregate(self):
139         case = generate_case(3)
140         with patch.object(benchmark_search, 'get_policy', side_effect=RuntimeError('injected failure')):
141             row, _ = benchmark_search.run_case(case)
142             summary = benchmark_search.aggregate([row])
143             self.assertEqual(summary['all_clear_cases'], 0)
144             self.assertEqual(summary['missed_sources'], len(case['sources']))
145             self.assertIsNone(summary['mean_average_clear_time_s'])
146             self.assertEqual(summary['failures'], [case['case_id']])
147             self.assertEqual(summary['threshold_pass_rate'], 0)
148             self.assertLess(summary['mean_reward'], -1000000)
149
150     def test_nonfinite_orientation_is_rejected_as_bad_case(self):
151         for invalid in (float('nan'), float('inf'), True, 'east'):
152             case = generate_case(4)
153             case['sources'][0]['orientation_deg'] = invalid
154             with self.assertRaises(ValueError):
155                 LocalSimulator(case)
156
157
158 if __name__ == '__main__':
159     unittest.main()

```

程序 21: problem3/tests/test_simulator.py

```

1 import math
2 import unittest
3
4 from problem3.simulator import LocalSimulator, ObservationClient
5 from problem3.scenarios import generate_case
6
7

```

```

8 def scene(sources, noise='extreme'):
9     return dict(seed=77, noise=noise, sources=sources)
10
11
12 class SimulatorTests(unittest.TestCase):
13     def test_attachment_time_example(self):
14         s = LocalSimulator(scene([])); s.enter()
15         self.assertEqual(s.measure((300, 400), 1)['virtual_time_s'], 105)
16         self.assertEqual(s.measure((300, 400), 2)['virtual_time_s'], 111)
17         self.assertEqual(s.clear((300, 0), 3)['virtual_time_s'], 194)
18         self.assertEqual(s.measure((300, 0), 2)['virtual_time_s'], 199)
19         self.assertEqual(s.exit()['virtual_time_s'], 199)
20
21     def test_directional_backside_and_optical(self):
22         s = LocalSimulator(scene([dict(channel=1, x=0, y=0, radius=1000, orientation_deg=0)]))
23         s.enter()
24         self.assertEqual(s.measure((-5, 0), 1)['measure_result'], 'no_signal')
25         self.assertEqual(s.measure((0, 5), 1)['measure_result'], 'near')
26         self.assertEqual(s.clear((-20, 0), 1)['clear_result'], 'success')
27         self.assertEqual(s.clear((-20, 0), 1)['clear_result'], 'no_target_in_range')
28
29     def test_fixed_local_error_rounding_and_boundary(self):
30         s = LocalSimulator(scene([dict(channel=2, x=800, y=.0001, radius=1000)])); s.enter()
31         a = s.measure((0, 0), 2)['svd_deg']
32         self.assertEqual(a, s.measure((0, 0), 2)['svd_deg'])
33         truth = math.degrees(math.atan2(.0001, 800))
34         self.assertEqual(abs((a-truth+180)%360-180), 1.0050001)
35         self.assertEqual(s.measure((-201, 0), 2)['measure_result'], 'no_signal')
36
37     def test_invalid_action_does_not_move(self):
38         s = LocalSimulator(scene([])); s.enter()
39         for point, channel in [((float('nan'), 0), 1), ((0, 0), True), ((2000001, 0), 1)]:
40             with self.assertRaises(ValueError):
41                 s.measure(point, channel)
42         self.assertEqual(s.position, (0, 0)); self.assertEqual(s.virtual_time_s, 0)
43
44     def test_facade_exposes_no_case_fields(self):
45         c = ObservationClient(LocalSimulator(generate_case(4)))
46         self.assertFalse(hasattr(c, 'sources')); self.assertFalse(hasattr(c, 'case'))
47
48     def test_generator_reproducible_and_disjoint(self):
49         seen = set()
50         for problem in (3, 4):
51             for split in ('development', 'holdout', 'stress'):
52                 for i in range(32):
53                     c = generate_case(problem, i, split)
54                     self.assertEqual(c, generate_case(problem, i, split))
55                     self.assertNotIn(c['seed'], seen); seen.add(c['seed'])
56                     self.assertTrue(10 <= len(c['sources']) <= 16)
57                     LocalSimulator(c)
58                     if problem == 4:
59                         count = sum(s['orientation_deg'] is not None for s in c['sources'])
60                         self.assertTrue(0 < count < len(c['sources']))
61
62

```

```

63 if __name__ == '__main__':
64     unittest.main()

```

程序 22: problem4/__init__.py

```

1 """Problem 4: mixed omnidirectional and directional sources."""

```

程序 23: problem4/coverage.py

```

1 """Finite directional-coverage certificates, using only the standard library.
2
3 For a hypothetical source g, let N(g) be the observation points within 1000 m.
4 Every closed emission half-plane through g contains a point of N(g) exactly
5 when g is in conv(N(g)). If not, a supporting pair line separates g from N(g).
6 For every oriented observation-point pair, we therefore cover the target disk
7 on the left with reception disks centred strictly on the left of that line.
8
9 The 0/1/2-neighbour cases also have a separating pair: with one neighbour a,
10 choose a second observation off the line ag; with collinear neighbours and g
11 beyond their segment, use the endpoint nearest g and any off-line observation.
12 For no neighbours use any noncollinear pair. The full observation hull must
13 strictly contain the target disk, so such off-line observations exist. A source
14 coinciding with its sole nearby observation is already in the neighbour hull.
15
16 Each pair check is ordinary disk coverage on a circular segment. Clip every
17 eligible site's Voronoi cell by that segment. The maximum distance on the cell
18 is attained at a vertex, an edge/circle intersection, or a farthest point of a
19 circular arc; all are enumerated. A sample grid is never used for acceptance.
20 Numerical clipping is outward and acceptance requires a separate metre margin.
21 """
22 from __future__ import annotations
23
24 import hashlib
25 import itertools
26 import json
27 import math
28 import time
29
30 Point = tuple[float, float]
31
32 # Frozen after geometric optimization, independently certified again after
33 # rounding to six decimals. These constants never depend on simulator cases.
34 STATIC23_POINTS: tuple[Point, ...] = (
35     (0.000000, 0.000000),
36     (1864.500000, 0.000000),
37     (1614.704365, 932.250000),
38     (932.250000, 1614.704365),
39     (0.000000, 1864.500000),
40     (-932.250000, 1614.704365),
41     (-1614.704365, 932.250000),
42     (-1864.500000, 0.000000),
43     (-1614.704365, -932.250000),
44     (-932.250000, -1614.704365),
45     (0.000000, -1864.500000),
46     (932.250000, -1614.704365),

```

```

47     (1614.704365, -932.250000),
48     (1076.467178, 0.001385),
49     (673.445446, 707.456053),
50     (35.090167, 1039.476106),
51     (-652.783202, 727.398374),
52     (-1076.469577, 0.000000),
53     (-804.167008, -363.249827),
54     (-538.234788, -932.250000),
55     (93.843216, -877.978803),
56     (538.234788, -932.250000),
57     (955.618224, -253.126730),
58 )
59
60
61 def static_coverage_points() -> list[Point]:
62     return list(STATIC23_POINTS)
63
64
65 def legacy_coverage_points() -> list[Point]:
66     return [(0.0, 0.0)] + [(950 * math.cos((i + .5) * math.tau / 12), 950 * math.sin((i + .5) * math.tau /
    ↪ 12)) for i in range(12)] + [(1870 * math.cos(i * math.tau / 12), 1870 * math.sin(i * math.tau /
    ↪ 12)) for i in range(12)]
67
68
69 def _cross(a, b, c):
70     return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])
71
72
73 def convex_hull(points):
74     points = sorted(set(points))
75     if len(points) <= 1:
76         return points
77     lower, upper = [], []
78     for point in points:
79         while len(lower) >= 2 and _cross(lower[-2], lower[-1], point) <= 0:
80             lower.pop()
81         lower.append(point)
82     for point in reversed(points):
83         while len(upper) >= 2 and _cross(upper[-2], upper[-1], point) <= 0:
84             upper.pop()
85         upper.append(point)
86     return lower[:-1] + upper[:-1]
87
88
89 def _clip(polygon, nx, ny, offset):
90     if not polygon:
91         return []
92     norm = math.hypot(nx, ny)
93     if norm == 0:
94         return polygon if offset >= 0 else []
95     nx, ny, offset = nx / norm, ny / norm, offset / norm + 1e-8
96     answer = []
97     a = polygon[-1]
98     fa = nx * a[0] + ny * a[1] - offset
99     for b in polygon:

```



```

100         fb = nx * b[0] + ny * b[1] - offset
101         if (fa <= 0) != (fb <= 0):
102             t = fa / (fa - fb)
103             answer.append((a[0] + t * (b[0] - a[0]), a[1] + t * (b[1] - a[1])))
104         if fb <= 0:
105             answer.append(b)
106         a, fa = b, fb
107     return answer
108
109
110 def _contains(polygon, point):
111     return bool(polygon) and all(_cross(a, b, point) >= -1e-6 * max(1.0, math.dist(a, b)) for a, b in
112         ↪ zip(polygon, polygon[1:] + polygon[:1]))
113
114 def _disk_intersection_extrema(polygon, site, radius):
115     candidates = [p for p in polygon if math.hypot(*p) <= radius + 1e-7]
116     for a, b in zip(polygon, polygon[1:] + polygon[:1]):
117         dx, dy = b[0] - a[0], b[1] - a[1]
118         length2 = dx * dx + dy * dy
119         if length2 < 1e-16:
120             continue
121         midpoint_t = -(a[0] * dx + a[1] * dy) / length2
122         foot = (a[0] + midpoint_t * dx, a[1] + midpoint_t * dy)
123         radial2 = radius * radius - foot[0] * foot[0] - foot[1] * foot[1]
124         if radial2 < -1e-6:
125             continue
126         half_t = math.sqrt(max(0.0, radial2) / length2)
127         for t in (midpoint_t - half_t, midpoint_t + half_t):
128             if -1e-9 <= t <= 1 + 1e-9:
129                 candidates.append((a[0] + t * dx, a[1] + t * dy))
130     norm = math.hypot(*site)
131     far = (-radius * site[0] / norm, -radius * site[1] / norm) if norm else (radius, 0.0)
132     if _contains(polygon, far):
133         candidates.append(far)
134     return candidates
135
136
137 def segment_cover_radius(sites, normal, offset, radius=1800.0, full_report=False):
138     """Analytic maximum nearest-site distance on disk intersect  $n \cdot x \geq \text{offset}$ ."""
139     nx, ny = normal
140     norm = math.hypot(nx, ny)
141     if norm == 0:
142         raise ValueError("A circular-segment normal must be nonzero")
143     if norm * radius < offset - 1e-7:
144         return {"radius_bound_m": 0.0, "witness": None, "cells": []}
145     if not sites:
146         return {"radius_bound_m": math.inf, "witness": (radius * nx / norm, radius * ny / norm), "cells":
147             ↪ []}
148     box = [(-radius, -radius), (radius, -radius), (radius, radius), (-radius, radius)]
149     segment = _clip(box, -nx, -ny, -offset)
150     worst, witness, cells = 0.0, None, []
151     for site_index, site in enumerate(sites):
152         cell = segment

```

```

153         if other == site:
154             continue
155         nx2, ny2 = other[0] - site[0], other[1] - site[1]
156         # Difference of squared norms as a dot product avoids cancellation
157         # for nearly coincident sites and keeps reversed constraints
158         # exactly antisymmetric before normalization. Otherwise two cells
159         # can develop a numerical gap and omit part of the target disk.
160         bound = (nx2 * (other[0] + site[0]) + ny2 * (other[1] + site[1])) / 2
161         cell = _clip(cell, nx2, ny2, bound)
162         if not cell:
163             break
164         candidates = _disk_intersection_extrema(cell, site, radius)
165         if not candidates:
166             continue
167         point = max(candidates, key=lambda p: math.dist(p, site))
168         value = math.dist(point, site)
169         if value > worst:
170             worst, witness = value, point
171         if full_report:
172             cells.append({"site_index": site_index, "radius_bound_m": value, "witness": point})
173     return {"radius_bound_m": worst, "witness": witness, "cells": cells}
174
175
176 def directional_cover_certificate(points, target_radius=1800.0, receive_radius=1000.0, margin_m=1e-3,
↪ full_report=False, early_exit=True):
177     """Return a conservative finite coverage report; no SciPy is required."""
178     started = time.perf_counter()
179     points = [tuple(map(float, p)) for p in points]
180     if any(len(p) != 2 or not all(math.isfinite(x) for x in p) for p in points):
181         raise ValueError("Coverage points must be finite coordinate pairs")
182     if not all(math.isfinite(x) and x > 0 for x in (target_radius, receive_radius, margin_m)) or margin_m <
↪ 1e-4:
183         raise ValueError("Radii must be positive; numerical margin must be at least 1e-4 m")
184     fingerprint = hashlib.sha256(json.dumps(points, separators=(",", ":"),
↪ allow_nan=False).encode()).hexdigest()
185     base = {"point_count": len(points), "points_sha256": fingerprint, "target_radius_m": target_radius,
186            "receive_radius_m": receive_radius, "numerical_margin_m": margin_m}
187     hull = convex_hull(points)
188     if len(hull) < 3:
189         return {**base, "certified": False, "reason": "degenerate_observation_hull"}
190     hull_inradius = min((a[0] * b[1] - a[1] * b[0]) / math.dist(a, b) for a, b in zip(hull, hull[1:] +
↪ hull[:1]))
191     if hull_inradius < target_radius + margin_m:
192         return {**base, "certified": False, "reason": "target_disk_not_strictly_inside_hull",
↪ "hull_inradius_m": hull_inradius}
193     worst, witness, worst_pair, checked, rows = 0.0, None, None, 0, []
194     for i, j in itertools.permutations(range(len(points)), 2):
195         a, b = points[i], points[j]
196         dx, dy = b[0] - a[0], b[1] - a[1]
197         length = math.hypot(dx, dy)
198         if length == 0:
199             continue
200         normal = (-dy / length, dx / length)
201         offset = normal[0] * a[0] + normal[1] * a[1]
202         if offset >= target_radius:

```

```

203         continue
204         indices = [k for k, point in enumerate(points) if normal[0] * point[0] + normal[1] * point[1] >
↳ offset + 1e-7]
205         result = segment_cover_radius([points[k] for k in indices], normal, offset, target_radius,
↳ full_report)
206         value, point = result["radius_bound_m"], result["witness"]
207         checked += 1
208         if value > worst:
209             worst, witness, worst_pair = value, point, [i, j]
210         if full_report:
211             rows.append({"pair": [i, j], "left_normal": normal, "line_offset": offset, "eligible_indices":
↳ indices, **result})
212         if early_exit and worst >= receive_radius - margin_m:
213             break
214         answer = {**base, "certified": worst < receive_radius - margin_m,
215                 "hull_inradius_m": hull_inradius, "max_directional_radius_bound_m": worst,
216                 "witness": witness, "pair": worst_pair, "pair_checks": checked,
217                 "program_runtime_s": time.perf_counter() - started}
218         if full_report:
219             answer["points"] = points
220             answer["pair_details"] = rows
221         return answer

```

程序 24: problem4/legacy_policy.py

```

1  """Mixed omnidirectional/directional search with a finite coverage certificate.
2
3  Only the public robot client is used. In particular, source count, positions,
4  receive radii and transmitting directions are never read from the client.
5  """
6  from __future__ import annotations
7
8  import math
9
10 from problem3.policy import SearchPolicy as OmnidirectionalPolicy
11 from problem3.geometry import clip_halfplane, distance, enclosing_circle, polygon_centroid
12
13 Point = tuple[float, float]
14 Triangle = tuple[Point, Point, Point]
15
16
17 def _cross(a: Point, b: Point, c: Point) -> float:
18     return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])
19
20
21 def _segment_distance(point: Point, a: Point, b: Point) -> float:
22     dx, dy = b[0] - a[0], b[1] - a[1]
23     denominator = dx * dx + dy * dy
24     t = 0.0 if denominator == 0 else max(0.0, min(1.0, ((point[0] - a[0]) * dx + (point[1] - a[1]) * dy) /
↳ denominator))
25     return math.hypot(point[0] - a[0] - t * dx, point[1] - a[1] - t * dy)
26
27
28 def triangle_contains(triangle: Triangle, point: Point, tolerance: float = 1e-7) -> bool:
29     """Inclusive test; tolerance has square-metre units."""

```

```

30     cross = [_cross(triangle[i], triangle[(i + 1) % 3], point) for i in range(3)]
31     return min(cross) >= -tolerance or max(cross) <= tolerance
32
33
34 def _triangle_intersects_disk(triangle: Triangle, radius: float) -> bool:
35     if triangle_contains(triangle, (0.0, 0.0)):
36         return True
37     return any(_segment_distance((0.0, 0.0), triangle[i], triangle[(i + 1) % 3]) <= radius + 1e-8 for i in
    ↪ range(3))
38
39
40 def covering_triangles(radius: float = 1800.0, spacing: float = 990.0) -> list[Triangle]:
41     """All triangles of the equilateral tiling that meet the target disk.
42
43     Each such triangle has diameter ``spacing < 1000``. A source in the
44     triangle is therefore within reception distance of all its vertices.
45     Since it is in their convex hull, every closed half-plane through the
46     source contains at least one vertex. This handles every unknown beam
47     orientation, including sources on the target boundary radiating outward.
48     """
49     if not math.isfinite(radius) or radius <= 0:
50         raise ValueError("radius must be positive and finite")
51     if not math.isfinite(spacing) or not 0 < spacing < 1000:
52         raise ValueError("directional coverage requires 0 < spacing < 1000")
53     height = spacing * math.sqrt(3.0) / 2.0
54     bound = math.ceil(2.0 * (radius + spacing) / spacing) + 2
55
56     def vertex(i: int, j: int) -> Point:
57         return (spacing * (i + j / 2.0), height * j)
58
59     result = []
60     for j in range(-bound, bound + 1):
61         for i in range(-bound, bound + 1):
62             lower = (vertex(i, j), vertex(i + 1, j), vertex(i, j + 1))
63             upper = (vertex(i + 1, j + 1), vertex(i, j + 1), vertex(i + 1, j))
64             for triangle in (lower, upper):
65                 if _triangle_intersects_disk(triangle, radius):
66                     result.append(triangle)
67     return result
68
69
70 def directional_coverage_points(radius: float = 1800.0, spacing: float = 990.0) -> list[Point]:
71     """Return a deterministic finite set, starting at the initial position."""
72     vertices = {vertex for triangle in covering_triangles(radius, spacing) for vertex in triangle}
73     # The base policy decides the visit order. This sorted form also provides
74     # a reproducible serpentine baseline when fixed ordering is requested.
75     height = spacing * math.sqrt(3.0) / 2.0
76     points = sorted(vertices, key=lambda p: (round(p[1] / height), p[0] if round(p[1] / height) % 2 == 0
    ↪ else -p[0]))
77     origin = (0.0, 0.0)
78     points.remove(origin)
79     return [origin, *points]
80
81
82 def polar_covering_triangles() -> list[Triangle]:

```

```

83     """A 25-vertex triangulated polygon containing the entire target disk.
84
85     The outer dodecagon has inradius  $1870 \cdot \cos(15 \text{ degrees}) > 1800$ . Every
86     triangle edge is at most 983.599 m, so the same convex-hull certificate
87     applies with fewer search stops than the regular triangular tiling.
88     """
89     outer = [(1870 * math.cos(i * math.tau / 12), 1870 * math.sin(i * math.tau / 12)) for i in range(12)]
90     inner = [(950 * math.cos((i + .5) * math.tau / 12), 950 * math.sin((i + .5) * math.tau / 12)) for i in
91             ↪ range(12)]
92     triangles = []
93     for i in range(12):
94         nxt = (i + 1) % 12
95         triangles.append(((0.0, 0.0), inner[i], inner[nxt]))
96         triangles.append((inner[i], outer[i], outer[nxt]))
97         triangles.append((inner[i], outer[nxt], inner[nxt]))
98     return triangles
99
100 def polar_coverage_points() -> list[Point]:
101     vertices = {vertex for triangle in polar_covering_triangles() for vertex in triangle}
102     return sorted(vertices, key=lambda p: (round(math.hypot(*p), 5), math.atan2(p[1], p[0])))
103
104
105 class SearchPolicy(OmnidirectionalPolicy):
106     """Q4 reuses bounded-bearing localization with a stronger global cover."""
107
108     route_refinements = 40
109
110     def __init__(self, problem: int = 4, strategy: str = "adaptive", config=None):
111         if problem != 4:
112             raise ValueError("problem4.SearchPolicy requires problem=4")
113         self._directional_strategy = strategy
114         super().__init__(problem=problem, strategy=strategy, config=config)
115
116     def coverage_points(self) -> list[Point]:
117         return directional_coverage_points() if self._directional_strategy == "baseline" else
118             ↪ polar_coverage_points()
119
120     def _route_goal(self, client, pending):
121         goals = [(point, "scan", index) for index, point in enumerate(pending)]
122         goals += [(polygon_centroid(polygon), "clear", channel) for channel, polygon in
123             ↪ self.regions.items() if channel not in self.cleared]
124         route = []
125         origin = client.position
126         while goals:
127             index = min(range(len(goals)), key=lambda i: distance(origin, goals[i][0]))
128             goal = goals.pop(index)
129             route.append(goal)
130             origin = goal[0]
131         for _ in range(self.route_refinements):
132             changed = False
133             for i in range(len(route) - 1):
134                 a = client.position if i == 0 else route[i - 1][0]
135                 b = route[i][0]
136                 for j in range(i + 1, len(route)):

```

```

135         c = route[j][0]
136         d = route[j + 1][0] if j + 1 < len(route) else None
137         old = distance(a, b) + (distance(c, d) if d is not None else 0)
138         new = distance(a, c) + (distance(b, d) if d is not None else 0)
139         if new + .01 < old:
140             route[i:j + 1] = reversed(route[i:j + 1])
141             changed = True
142             break
143         if changed:
144             break
145         if not changed:
146             break
147     return route[0]
148
149 def run(self, client) -> dict:
150     if self.strategy != "adaptive":
151         return super().run(client)
152     pending = self.coverage_points()
153     while pending or any(channel not in self.cleared for channel in self.regions):
154         client.check_budget()
155         _, action, identifier = self._route_goal(client, pending)
156         if action == "scan":
157             self._scan(client, pending.pop(identifier))
158         else:
159             self._localize(identifier, client)
160             self._update_neighbors(client)
161             if len(self.cleared) == 16:
162                 return self._result("maximum_source_count_cleared", not pending)
163     return self._result("certified_coverage_and_all_detected_cleared", True)
164
165 def _localize(self, channel: int, client) -> bool:
166     if self.strategy != "adaptive":
167         return super()._localize(channel, client)
168     attempted_clear = False
169     attempted_measure = set()
170     # Paired transverse probes handle the unknown beam side. A single
171     # negative result never removes a reception disk from the source set.
172     for _ in range(5):
173         polygon = self.regions[channel]
174         center, radius = enclosing_circle(polygon)
175         if radius <= self.config.clear_guarantee_radius_m:
176             if self._clear(client, center, channel):
177                 return True
178         elif radius <= self.config.exploratory_clear_radius_m and not attempted_clear:
179             attempted_clear = True
180             if self._clear(client, polygon.centroid(polygon), channel):
181                 return True
182     anchor, bearing = self.observations[channel][-1]
183     theta = math.radians(bearing)
184     forward = (math.cos(theta), math.sin(theta))
185     sideways = (-forward[1], forward[0])
186     longitudinal = [(point[0] - anchor[0]) * forward[0] + (point[1] - anchor[1]) * forward[1] for
187                     ↪ point in polygon]
188     step = (min(longitudinal) + max(longitudinal)) / 2
189     width = max(35.0, min(140.0, .18 * step))

```

```

189         width = max(width, step * math.tan(math.radians(self.config.bearing_error_deg)) + .01)
190         midpoint = (anchor[0] + step * forward[0], anchor[1] + step * forward[1])
191         probes = [(midpoint[0] + sign * width * sideways[0], midpoint[1] + sign * width * sideways[1])
192                  ↪ for sign in (-1, 1)]
193         probes.sort(key=lambda point: distance(client.position, point))
194         all_in_range = all(distance(point, vertex) < 999.999 for point in probes for vertex in polygon)
195         negative = 0
196         for point in probes:
197             key = (round(point[0], 7), round(point[1], 7))
198             if key in attempted_measure:
199                 continue
200             attempted_measure.add(key)
201             result = self._measure(client, point, channel)
202             self.stats["paired_probe_actions"] = self.stats.get("paired_probe_actions", 0) + 1
203             if channel in self.cleared:
204                 return True
205             if result == "direction":
206                 break
207             negative += 1
208             self.stats["negative_localization_reads"] += 1
209         if negative == 2 and all_in_range and step > 0:
210             # Every feasible source beyond this cross-section has a point
211             # of its anchor-to-source segment between the two probes.
212             # That segment stays inside its closed emitting half-plane.
213             # Two absent signals therefore certify that the source lies
214             # before the cross-section, because distance was ruled out.
215             constant = step + anchor[0] * forward[0] + anchor[1] * forward[1]
216             updated = clip_halfplane(polygon, forward[0], forward[1], constant)
217             if not updated:
218                 raise RuntimeError("Paired negative observations contradict the feasible source set")
219             self.regions[channel] = updated
220             self.stats["paired_negative_clips"] = self.stats.get("paired_negative_clips", 0) + 1
221         # Reuse the complete optical cell cover, without restarting radio
222         # probes. The ordinary fallback never relies on emission direction.
223         from problem3.geometry import optical_cover
224         self.stats["optical_fallbacks"] += 1
225         points = optical_cover(self.regions[channel], self.config.optical_spacing_m)
226         while points:
227             index = min(range(len(points)), key=lambda i: distance(client.position, points[i]))
228             if self._clear(client, points.pop(index), channel):
229                 return True
230         raise RuntimeError("Optical cover exhausted without clearance: model/protocol inconsistency")

```

程序 25: problem4/localization.py

```

1  """Conservative directional localization from positive and negative readings.
2
3  The mixin uses only the public client interface. Movement proposals affect
4  cost, while retained bearing polygons and verified negative cuts establish
5  correctness. Optical coverage remains the final, direction-independent path.
6  """
7  from __future__ import annotations
8
9  import math
10

```

```

11 from problem3.geometry import (
12     clip_halfplane, distance, enclosing_circle, optical_cover, polygon_centroid,
13 )
14
15
16 def _convex_hull(points):
17     """Convex outer merge; preserve singleton and segment degeneracies."""
18     ordered = sorted(set(points))
19     if len(ordered) <= 2:
20         return ordered
21
22     def cross(o, a, b):
23         return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
24
25     lower, upper = [], []
26     for point in ordered:
27         while len(lower) > 1 and cross(lower[-2], lower[-1], point) <= 0:
28             lower.pop()
29         lower.append(point)
30     for point in reversed(ordered):
31         while len(upper) > 1 and cross(upper[-2], upper[-1], point) <= 0:
32             upper.pop()
33         upper.append(point)
34     return lower[:-1] + upper[:-1]
35
36
37 class LocalizationMixin:
38     localization_quantile = .2
39     localization_width_fraction = .04
40     localization_width_min = 15.
41     localization_width_max = 140.
42     localization_rounds = 5
43     localization_history_slices = 12
44
45     def _history_point_in_range(self, point, cell, channel):
46         """Prove Q is inside the unknown receiving radius for every g in cell.
47
48         Either  $|Q-g| < 1000$ , or a previous positive P guarantees
49          $|Q-g| < |P-g| \leq R$ . The latter squared-distance difference is affine,
50         so checking all vertices is sufficient. The two negative points of
51         a pair may use different positive observations for this proof.
52         """
53         if max(distance(point, vertex) for vertex in cell) < 999.999:
54             return True
55         qx, qy = point
56         for positive, _ in self.observations[channel]:
57             px, py = positive
58             a, b = 2 * (px - qx), 2 * (py - qy)
59             c = px * px + py * py - qx * qx - qy * qy
60             if max(a * v[0] + b * v[1] - c for v in cell) < -.002:
61                 return True
62         return False
63
64     def _measure(self, client, point, channel):
65         result = super()._measure(client, point, channel)

```



```

66     if self.strategy != "adaptive":
67         return result
68     if not hasattr(self, "_negative_history"):
69         self._negative_history = {}
70     if result == "no_signal":
71         history = self._negative_history.setdefault(channel, [])
72         if point not in history:
73             history.append(point)
74     elif result == "direction":
75         self._clip_history(channel)
76     return result
77
78 def _clip_history(self, channel):
79     """Remove only shadow cones with a reception-distance certificate.
80
81     A received anchor A and two absent points B,C rule out sources beyond
82     segment BC in A's viewing cone when B,C are both in range. Otherwise
83     the emitting half-plane would contain an A-to-source segment point
84     on BC while excluding all of BC, a contradiction. For an omni source,
85     an in-range negative is already impossible. Splitting into narrow
86     cells makes the distance proof useful without assuming a beam side.
87     """
88     negatives = self._negative_history.get(channel, [])
89     if len(negatives) < 2:
90         return
91     polygon = self.regions[channel]
92     _, bearing = self.observations[channel][-1]
93     theta = math.radians(bearing)
94     forward = (math.cos(theta), math.sin(theta))
95     projections = [p[0] * forward[0] + p[1] * forward[1] for p in polygon]
96     lower, upper = min(projections), max(projections)
97     cells = []
98     if upper - lower < 1e-9:
99         cells = [list(polygon)]
100     else:
101         count = self.localization_history_slices
102         for i in range(count):
103             lo = lower + (upper - lower) * i / count
104             hi = lower + (upper - lower) * (i + 1) / count
105             cell = clip_halfplane(polygon, forward[0], forward[1], hi)
106             cell = clip_halfplane(cell, -forward[0], -forward[1], -lo)
107             if cell:
108                 cells.append(cell)
109     for a, _ in self.observations[channel]:
110         for i, b_point in enumerate(negatives):
111             for c_point in negatives[i + 1:]:
112                 b, c = b_point, c_point
113                 cross = ((b[0] - a[0]) * (c[1] - a[1])
114                         - (b[1] - a[1]) * (c[0] - a[0]))
115                 if abs(cross) < 1e-5:
116                     continue
117                 if cross < 0:
118                     b, c = c, b
119                 dx1, dy1 = b[0] - a[0], b[1] - a[1]
120                 dx2, dy2 = c[0] - a[0], c[1] - a[1]

```

```

121         dx3, dy3 = c[0] - b[0], c[1] - b[1]
122         # cross(B-A,g-A)>=0, cross(C-A,g-A)<=0,
123         # and the side of BC opposite A.
124         forbidden = [
125             (dy1, -dx1, dy1 * a[0] - dx1 * a[1]),
126             (-dy2, dx2, -dy2 * a[0] + dx2 * a[1]),
127             (-dy3, dx3, -dy3 * b[0] + dx3 * b[1]),
128         ]
129         next_cells = []
130         for cell in cells:
131             if any(min(aa * v[0] + bb * v[1] - cc for v in cell) >= 1e-4
132                   for aa, bb, cc in forbidden):
133                 next_cells.append(cell)
134                 continue
135             if not all(self._history_point_in_range(p, cell, channel) for p in (b, c)):
136                 next_cells.append(cell)
137                 continue
138             inside = cell
139             # Keep the union of the cone's three complement
140             # half-planes. Outward clipping retains boundaries.
141             for aa, bb, cc in forbidden:
142                 outside = clip_halfplane(inside, -aa, -bb, -cc)
143                 if outside:
144                     next_cells.append(outside)
145                 inside = clip_halfplane(inside, aa, bb, cc)
146                 if not inside:
147                     break
148         cells = next_cells
149         if not cells:
150             raise RuntimeError("Historical negatives exhausted feasible cells")
151         if len(cells) > 128:
152             # Discard this optional refinement and retain the
153             # original region if its partition grows too large.
154             return
155         self.regions[channel] = _convex_hull(p for cell in cells for p in cell)
156         self.stats["history_clips"] = self.stats.get("history_clips", 0) + 1
157
158     def _localize(self, channel, client):
159         if self.strategy != "adaptive":
160             return super()._localize(channel, client)
161         attempted_clear = False
162         attempted_measure = set()
163         for _ in range(self.localization_rounds):
164             polygon = self.regions[channel]
165             center, radius = enclosing_circle(polygon)
166             if radius <= self.config.clear_guarantee_radius_m:
167                 if self._clear(client, center, channel):
168                     return True
169             elif radius <= self.config.exploratory_clear_radius_m and not attempted_clear:
170                 attempted_clear = True
171                 if self._clear(client, polygon_centroid(polygon), channel):
172                     return True
173             anchor, bearing = self.observations[channel][-1]
174             theta = math.radians(bearing)
175             forward = (math.cos(theta), math.sin(theta))

```

```

176     sideways = (-forward[1], forward[0])
177     longitudinal = [(point[0] - anchor[0]) * forward[0]
178                     + (point[1] - anchor[1]) * forward[1]] for point in polygon]
179     low, high = min(longitudinal), max(longitudinal)
180     step = low + self.localization_quantile * (high - low)
181     width = max(self.localization_width_min, min(
182         self.localization_width_max, self.localization_width_fraction * step))
183     width = max(width, step * math.tan(math.radians(self.config.bearing_error_deg)) + .01)
184     # Move the pair toward the interval midpoint until it is safely
185     # in range. The final explicit check below controls any cut even
186     # if this bounded search fails to find such a position.
187     for _range_step in range(24):
188         midpoint = (anchor[0] + step * forward[0], anchor[1] + step * forward[1])
189         pair = [(midpoint[0] + sign * width * sideways[0],
190                 midpoint[1] + sign * width * sideways[1]) for sign in (-1, 1)]
191         if max(distance(point, vertex) for point in pair for vertex in polygon) < 999.99:
192             break
193         step = .5 * step + .25 * (low + high)
194         width = max(width, step * math.tan(math.radians(self.config.bearing_error_deg)) + .01)
195     midpoint = (anchor[0] + step * forward[0], anchor[1] + step * forward[1])
196     probes = [(midpoint[0] + sign * width * sideways[0],
197                midpoint[1] + sign * width * sideways[1]) for sign in (-1, 1)]
198     probes.sort(key=lambda point: distance(client.position, point))
199     all_in_range = all(distance(point, vertex) < 999.999
200                       for point in probes for vertex in polygon)
201     negative = 0
202     for point in probes:
203         key = (round(point[0], 7), round(point[1], 7))
204         if key in attempted_measure:
205             continue
206         attempted_measure.add(key)
207         result = self._measure(client, point, channel)
208         self.stats["paired_probe_actions"] = self.stats.get("paired_probe_actions", 0) + 1
209         if channel in self.cleared:
210             return True
211         if result == "direction":
212             break
213         negative += 1
214         self.stats["negative_localization_reads"] += 1
215     if negative == 2 and all_in_range and step > 0:
216         constant = step + anchor[0] * forward[0] + anchor[1] * forward[1]
217         updated = clip_halfplane(polygon, forward[0], forward[1], constant)
218         if not updated:
219             raise RuntimeError("Paired negative observations contradict feasible region")
220         self.regions[channel] = updated
221         self.stats["paired_negative_clips"] = self.stats.get("paired_negative_clips", 0) + 1
222     self.stats["optical_fallbacks"] += 1
223     points = optical_cover(self.regions[channel], self.config.optical_spacing_m)
224     while points:
225         index = min(range(len(points)), key=lambda i: distance(client.position, points[i]))
226         if self._clear(client, points.pop(index), channel):
227             return True
228     raise RuntimeError("Optical cover exhausted without clearance")

```

程序 26: problem4/policy.py

```
1 """Q4 version 2: certified coverage, historical bearings, and shorter routes.
2
3 The previous delivery remains available as problem4.legacy_policy and through
4 --strategy legacy. Production algorithms do not import development experiments.
5 """
6 from __future__ import annotations
7
8 from problem4.coverage import static_coverage_points
9 from problem4.localization import LocalizationMixin
10 from problem4.routing import RoutingMixin
11 from problem4.legacy_policy import (
12     SearchPolicy as LegacyPolicy,
13     covering_triangles, directional_coverage_points, polar_covering_triangles,
14     polar_coverage_points, triangle_contains,
15 )
16
17
18 class SearchPolicy(RoutingMixin, LocalizationMixin, LegacyPolicy):
19     """Keep completion proofs independent of heuristic route estimates."""
20
21     def coverage_points(self):
22         if self.strategy == "adaptive":
23             return static_coverage_points()
24         return super().coverage_points()
25
26     def _result(self, reason, coverage_complete):
27         result = super()._result(reason, coverage_complete)
28         if self.strategy == "adaptive":
29             result["algorithm_version"] = "problem4_v2"
30             result["effective_settings"] = {
31                 "coverage_points": 23,
32                 "history_slices": self.localization_history_slices,
33                 "probe_quantile": self.localization_quantile,
34                 "probe_width_fraction": self.localization_width_fraction,
35                 "probe_min_width_m": self.localization_width_min,
36                 "probe_rounds": self.localization_rounds,
37                 "route_initial_constructions": self.escape_starts,
38                 "reuse_previous_route": self.escape_warm,
39                 "max_relocated_chain": self.escape_chain,
40                 "expected_scan_cost_weight": self.escape_info_weight,
41             }
42         return result
```

程序 27: problem4/routing.py

```
1 """Observation-only open-tour planning for mixed directional search.
2
3 Visits are ordered by geometric travel cost with 2-opt reversals, Or-opt
4 relocations of one to three consecutive visits, several initial constructions,
5 and the previous route as a warm start. An approximate discovery prior accounts
6 for future channel-scanning cost. These choices affect speed only: the calling
7 policy must retain its independent coverage and clearance certificates.
8
```

```

9 Once 16 distinct channels have been discovered, further discovery is omitted.
10 Only actual successful clearance of all 16 certifies early termination.
11 """
12 from __future__ import annotations
13 import math
14 from problem3.geometry import distance, polygon_centroid
15
16 def route_length(route,matrix):
17     return sum(matrix[len(matrix)-1 if i==0 else route[i-1]][j] for i,j in enumerate(route))
18
19
20 def two_opt(route,matrix,moves=80):
21     route=list(route);start=len(matrix)-1
22     for _ in range(moves):
23         best=-.01;change=None
24         for i in range(len(route)-1):
25             a=start if i==0 else route[i-1];b=route[i]
26             for j in range(i+1,len(route)):
27                 c=route[j];d=route[j+1] if j+1<len(route) else None
28                 delta=matrix[a][c]-matrix[a][b]
29                 if d is not None:delta+=matrix[b][d]-matrix[c][d]
30                 if delta<best:best=delta;change=(i,j)
31         if change is None:break
32         i,j=change;route[i:j+1]=reversed(route[i:j+1])
33     return route
34
35
36 def or_opt(route,matrix,max_moves=16,max_chain=3):
37     """Relocate contiguous chains, maintaining the fixed start and free end."""
38     route=list(route);start=len(matrix)-1;n=len(route)
39     for _ in range(max_moves):
40         best=-.01;change=None
41         for k in range(1,min(max_chain,n-1)+1):
42             for i in range(n-k+1):
43                 chain=route[i:i+k];rest=route[:i]+route[i+k:]
44                 a=start if i==0 else route[i-1];b=route[i+k] if i+k<n else None
45                 removal=-matrix[a][chain[0]]
46                 if b is not None:removal+=matrix[a][b]-matrix[chain[-1]][b]
47                 for j in range(len(rest)+1):
48                     if j==i:continue
49                     c=start if j==0 else rest[j-1];d=rest[j] if j<len(rest) else None
50                     for reverse in (False,True) if k>1 else (False,):
51                         first,last=(chain[-1],chain[0]) if reverse else (chain[0],chain[-1])
52                         delta=removal+matrix[c][first]
53                         if d is not None:delta+=matrix[last][d]-matrix[c][d]
54                         if delta<best:best=delta;change=(i,k,j,reverse)
55         if change is None:break
56         i,k,j,reverse=change;chain=route[i:i+k];rest=route[:i]+route[i+k:]
57         if reverse:chain.reverse()
58         route=rest[:j]+chain+rest[j:]
59         route=two_opt(route,matrix,20)
60     return route
61
62
63 def nearest_seed(matrix,first=None):

```

```

64     n=len(matrix)-1;remaining=set(range(n));route=[];last=n
65     if first is not None:route=[first];remaining.remove(first);last=first
66     while remaining:
67         nxt=min(remaining,key=lambda i:matrix[last][i]);route.append(nxt);remaining.remove(nxt);last=nxt
68     return route
69
70
71 def insertion_seed(matrix):
72     n=len(matrix)-1;start=n;remaining=set(range(n))
73     if not remaining:return []
74     first=max(remaining,key=lambda i:matrix[start][i]);route=[first];remaining.remove(first)
75     while remaining:
76         best=None
77         for point in remaining:
78             for index in range(len(route)+1):
79                 a=start if index==0 else route[index-1];b=route[index] if index<len(route) else None
80                 extra=matrix[a][point]+(matrix[point][b]-matrix[a][b] if b is not None else 0)
81                 candidate=(extra,point,index)
82                 if best is None or candidate<best:best=candidate
83             _,point,index=best;route.insert(index,point);remaining.remove(point)
84     return route
85
86
87 class RoutingMixin:
88     escape_oropt=True
89     escape_chain=3
90     escape_starts=4
91     escape_warm=True
92     escape_info_weight=1.
93     escape_oropt_final_only=True
94
95     # A discrete prior estimates future scanning cost only. It is never used
96     # to remove source candidates or to certify completion.
97     samples = None
98     mask_cache = {}
99
100     def __init__(self, *args, **kwargs):
101         super().__init__(*args, **kwargs)
102         self.scanned_mask = 0
103         if type(self).samples is None:
104             points = [(x,y) for x in range(-1800,1801,300)
105                       for y in range(-1800,1801,300) if math.hypot(x,y)<=1800]
106             type(self).samples = [(point,radius,angle) for point in points
107                                   for radius in (1000,1250,1500)
108                                   for angle in [None]*8+[i*math.tau/8 for i in range(8)]]
109
110     def _mask(self, point):
111         key = tuple(round(value,5) for value in point)
112         if key not in self.mask_cache:
113             mask = 0
114             for index,(source,radius,angle) in enumerate(self.samples):
115                 dx,dy = point[0]-source[0],point[1]-source[1]
116                 if math.hypot(dx,dy)<=radius and (angle is None or
117                                                     dx*math.cos(angle)+dy*math.sin(angle)>=-1e-8):
118                     mask |= 1<<index

```

```

119         self.mask_cache[key] = mask
120     return self.mask_cache[key]
121
122     def _scan(self, client, point):
123         super()._scan(client, point)
124         self.scanned_mask |= self._mask(point)
125
126     @staticmethod
127     def _goal_key(goal):
128         p, kind, ident = goal
129         return (kind, round(p[0], 5), round(p[1], 5)) if kind == 'scan' else (kind, ident)
130
131     def _route_goal(self, client, pending):
132         known = set(self.regions) | self.cleared
133         if len(known) == 16: pending = []
134         goals = [(p, 'scan', i) for i, p in enumerate(pending)] + [(polygon_centroid(p), 'clear', ch) for ch, p in
↪ self.regions.items() if ch not in self.cleared]
135         points = [g[0] for g in goals] + [client.position]
136         matrix = [[distance(a, b) for b in points] for a in points]
137         seeds = [nearest_seed(matrix)]
138         if self.escape_starts > 1:
139             seeds.append(insertion_seed(matrix))
140         if self.escape_starts > 2:
141             ordered = sorted(range(len(goals)), key=lambda i: math.atan2(goals[i][0][1], goals[i][0][0]))
142             if ordered:
143                 cut = min(range(len(ordered)), key=lambda i: matrix[-1][ordered[i]])
144                 angular = ordered[cut:] + ordered[:cut]
145                 seeds.append(angular)
146                 if self.escape_starts > 3: seeds.append([angular[0]] + list(reversed(angular[1:])))
147         if self.escape_starts > 4:
148             nearest = sorted(range(len(goals)), key=lambda i: matrix[-1][i])
149             seeds.extend(nearest_seed(matrix, i) for i in nearest[1:self.escape_starts-3])
150         if self.escape_warm and hasattr(self, '_route_keys'):
151             mapping = {self._goal_key(g): i for i, g in enumerate(goals)}
152             warm = [mapping[k] for k in self._route_keys if k in mapping]
153             for point in range(len(goals)):
154                 if point in warm: continue
155                 best = min(range(len(warm)+1), key=lambda i: matrix[-1 if i==0 else
↪ warm[i-1]][point] + (matrix[point][warm[i]] - matrix[-1 if i==0 else warm[i-1]][warm[i]]
↪ if i < len(warm) else 0))
156                 warm.insert(best, point)
157             seeds.append(warm)
158         candidates = []
159         for seed in seeds:
160             route = two_opt(seed, matrix)
161             if self.escape_oropt and not
↪ self.escape_oropt_final_only: route = or_opt(route, matrix, max_chain=self.escape_chain)
162             candidates.append(route)
163         route = min(candidates, key=lambda r: route_length(r, matrix))
164         if self.escape_oropt and
↪ self.escape_oropt_final_only: route = or_opt(route, matrix, max_chain=self.escape_chain)
165         if self.escape_info_weight and pending and len(pending) > 1:
166             total = len(self.samples); unseen = total - self.scanned_mask.bit_count()
167             if unseen:
168                 unknown = 20 - len(known); posterior = .65 * unseen / (.35 * total + .65 * unseen)

```

```

169         masks={i:self._mask(p) for i,p in enumerate(pending)}
170     def score(r):
171         cost=route_length(r,matrix)/5;covered=self.scanned_mask
172         for index in r:
173             _,kind,ident=goals[index]
174             if kind=='scan':
175                 cost+=self.escape_info_weight*6*unknown*((1-posterior)+posterior*(total-cover
176                 ↪ ed.bit_count())/unseen)
177                 covered|=masks[ident]
178         return cost
179     best=score(route)
180     for _ in range(20):
181         changed=False
182         for i in range(len(route)-1):
183             for j in range(i+1,len(route)):
184                 candidate=route[:i]+list(reversed(route[i:j+1]))+route[j+1:];cost=score(candi
185                 ↪ date)
186                 if cost<best-.01:route=candidate;best=cost;changed=True;break
187             if changed:break
188         if not changed:break
189     self._route_keys=[self._goal_key(goals[i]) for i in route]
190     return goals[route[0]]

```

程序 28: problem4/solve.py

```

1  """Problem 4 entry point, independent of the caller's working directory."""
2  from pathlib import Path
3  import sys
4
5  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
6  from problem3.run import main
7
8  if __name__ == '__main__':
9      raise SystemExit(main(4))

```

程序 29: problem4/tests/test_coverage.py

```

1  """Regression of the finite proof calculation, including geometric failures."""
2  from __future__ import annotations
3
4  import math
5  import unittest
6
7  from problem4.coverage import directional_cover_certificate, legacy_coverage_points, segment_cover_radius,
8  ↪ static_coverage_points
9
10 class VoronoiSegmentTests(unittest.TestCase):
11     def test_one_site_farthest_arc_is_enumerated(self):
12         result = segment_cover_radius([(-500.0, 0.0)], (1.0, 0.0), 0)
13         self.assertAlmostEqual(result["radius_bound_m"], 2300.0, places=5)
14
15     def test_clipped_arc_endpoints_are_enumerated(self):
16         result = segment_cover_radius([(500.0, 0.0)], (1.0, 0.0), 0)

```



```

17         self.assertAlmostEqual(result["radius_bound_m"], math.hypot(1800, 500), places=5)
18
19     def test_voronoi_bisector_and_circle_intersections(self):
20         result = segment_cover_radius([(−500.0, 0.0), (500.0, 0.0)], (1.0, 0.0), −1801)
21         self.assertAlmostEqual(result["radius_bound_m"], math.hypot(1800, 500), places=5)
22
23     def test_narrow_circular_cap(self):
24         result = segment_cover_radius([(1900.0, 0.0)], (1.0, 0.0), 1799)
25         self.assertAlmostEqual(result["radius_bound_m"], math.sqrt(101 ** 2 + 1800 ** 2 − 1799 ** 2),
26             ↪ places=5)
27
28     def test_nearly_coincident_sites_do_not_lose_a_circular_cap(self):
29         # Subtracting the four squared coordinates used to leave a 1.159 m
30         # numerical gap between these two Voronoi cells and return zero here.
31         a = (878.5434900187672, −767.9402882393684)
32         b = (878.543490018679, −767.9402882393205)
33         delta = (b[0] − a[0], b[1] − a[1])
34         norm = math.hypot(*delta)
35         normal = (−delta[0] / norm, −delta[1] / norm)
36         one = segment_cover_radius([a], normal, 1138.1, 1138.4)
37         two = segment_cover_radius([a, b], normal, 1138.1, 1138.4)
38         self.assertGreater(two["radius_bound_m"], 281)
39         # Adding a site only 1e−10 m away changes nearest-site distances by
40         # at most that separation (plus the outward computational margins).
41         self.assertAlmostEqual(one["radius_bound_m"], two["radius_bound_m"], places=5)
42
43     class DirectionalCertificateTests(unittest.TestCase):
44     def test_frozen_23_point_layout_has_large_reception_margin(self):
45         points = static_coverage_points()
46         result = directional_cover_certificate(points)
47         self.assertEqual(len(points), 23)
48         self.assertTrue(result["certified"])
49         self.assertEqual(result["pair_checks"], 494)
50         self.assertLess(result["max_directional_radius_bound_m"], 985)
51         self.assertGreater(result["hull_inradius_m"], 1800.9)
52
53     def test_old_25_point_proof_is_reproduced(self):
54         result = directional_cover_certificate(legacy_coverage_points())
55         self.assertTrue(result["certified"])
56         self.assertAlmostEqual(result["max_directional_radius_bound_m"], 983.598261, places=4)
57
58     def test_no_single_frozen_point_can_be_silently_deleted(self):
59         # Local irredundancy of this layout, not a lower bound on all layouts.
60         points = static_coverage_points()
61         for index in range(len(points)):
62             with self.subTest(index=index):
63                 self.assertFalse(directional_cover_certificate(points[:index] + points[index +
64                     ↪ 1:]))["certified"])
65
66     def test_previously_failed_symmetric_23_points_are_rejected(self):
67         n = 11
68         points = [(0.0, 0.0)]
69         points += [(1000 * math.cos((i + .5) * math.tau / n), 1000 * math.sin((i + .5) * math.tau / n))
70             ↪ for i in range(n)]

```

```

69         points += [(1876 * math.cos(i * math.tau / n), 1876 * math.sin(i * math.tau / n)) for i in
    ↪ range(n)]
70         self.assertFalse(directional_cover_certificate(points)["certified"])
71
72     def test_rotation_and_order_do_not_change_the_proof(self):
73         angle = .173
74         c, s = math.cos(angle), math.sin(angle)
75         points = [(x * c - y * s, x * s + y * c) for x, y in reversed(static_coverage_points())]
76         result = directional_cover_certificate(points)
77         self.assertTrue(result["certified"])
78         self.assertAlmostEqual(result["max_directional_radius_bound_m"], 984.418184, places=3)
79
80     def test_degenerate_hull_and_invalid_margin_are_rejected(self):
81         self.assertFalse(directional_cover_certificate([(-2000, 0), (0, 0), (2000, 0)])["certified"])
82         with self.assertRaises(ValueError):
83             directional_cover_certificate(static_coverage_points(), margin_m=1e-8)
84
85     def test_distinct_nearly_coincident_pairs_are_still_enumerated(self):
86         points = static_coverage_points() + [(878.5434900187672, -767.9402882393684), (878.543490018679,
    ↪ -767.9402882393205)]
87         result = directional_cover_certificate(points, full_report=True, early_exit=False)
88         pairs = {tuple(row["pair"]) for row in result["pair_details"]}
89         self.assertIn((23, 24), pairs)
90         self.assertIn((24, 23), pairs)
91         self.assertTrue(result["certified"])
92
93
94 if __name__ == "__main__":
95     unittest.main()

```

程序 30: problem4/tests/test_localization.py

```

1  """Physical half-plane counterexamples and source-retention boundary checks."""
2  from __future__ import annotations
3
4  import math
5  import random
6  import unittest
7
8  from problem3.geometry import contains
9  from problem3.policy import SearchPolicy as BasePolicy
10 from problem4.localization import LocalizationMixin
11
12
13 class _Policy(LocalizationMixin, BasePolicy):
14     pass
15
16
17 class _EmitterOracle:
18     """Independent test emitter; the policy receives only public responses."""
19
20     def __init__(self, source, radius=1000., heading=math.pi, error=0.):
21         self.__source = source
22         self.__radius = radius
23         self.__normal = (math.cos(heading), math.sin(heading))

```

```

24         self.__error = error
25         self.position = (0., 0.)
26         self.current_channel = 1
27         self.virtual_time_s = 0.
28
29     def check_budget(self):
30         return None
31
32     def measure(self, point, channel):
33         self.position = point
34         self.current_channel = channel
35         dx, dy = point[0] - self.__source[0], point[1] - self.__source[1]
36         visible = dx * self.__normal[0] + dy * self.__normal[1] >= -1e-9
37         if math.hypot(dx, dy) > self.__radius + 1e-9 or not visible:
38             return {"accepted": True, "measure_result": "no_signal"}
39         angle = math.degrees(math.atan2(-dy, -dx))
40         return {"accepted": True, "measure_result": "direction",
41                 "svd_deg": round((angle + self.__error) % 360, 2) % 360}
42
43
44 class HistoricalNegativeGeometryTests(unittest.TestCase):
45     def test_in_range_backside_pair_removes_shadow_but_retains_source(self):
46         source = (100., 0.)
47         client, policy = _EmitterOracle(source), _Policy()
48         for point in ((600., -40.), (600., 40.)):
49             self.assertEqual(policy._measure(client, point, 1), "no_signal")
50             self.assertEqual(policy._measure(client, (0., 0.), 1), "direction")
51             self.assertTrue(contains(policy.regions[1], source))
52             self.assertLess(max(p[0] for p in policy.regions[1]), 600.001)
53
54     def test_out_of_range_negatives_must_not_remove_source_behind_pair(self):
55         # Both negatives lie on the transmitting side. They are absent only
56         # because of distance; deleting their shadow without a range proof
57         # would incorrectly remove the true source at x=1400.
58         source = (1400., 0.)
59         client, policy = _EmitterOracle(source, radius=1500.), _Policy()
60         for point in ((300., -1400.), (300., 1400.)):
61             self.assertGreater(math.dist(point, source), 1500.)
62             self.assertEqual(policy._measure(client, point, 1), "no_signal")
63             self.assertEqual(policy._measure(client, (0., 0.), 1), "direction")
64             self.assertTrue(contains(policy.regions[1], source))
65
66     def test_positive_radius_lower_bound_cuts_beyond_1000m(self):
67         source = (0., 0.)
68         client, policy = _EmitterOracle(source, radius=1500.), _Policy()
69         negatives = ((100., -1100.), (100., 1100.))
70         for point in negatives:
71             self.assertGreater(math.dist(point, source), 1000.)
72             self.assertEqual(policy._measure(client, point, 1), "no_signal")
73             self.assertEqual(policy._measure(client, (-1100., 0.), 1), "direction")
74             self.assertTrue(contains(policy.regions[1], source))
75             self.assertLess(max(p[0] for p in policy.regions[1]), 100.001)
76
77     def test_two_negatives_may_require_different_positive_radius_witnesses(self):
78         source = (0., 0.)

```

```

79     client = _EmitterOracle(source, radius=1500.)
80     lower, upper = (-1000., -1000.), (-1000., 1000.)
81     q_upper, q_lower = (100., 1200.), (100., -1200.)
82     for point in (lower, upper):
83         self.assertEqual(client.measure(point, 1)["measure_result"], "direction")
84     for point in (q_upper, q_lower):
85         self.assertEqual(client.measure(point, 1)["measure_result"], "no_signal")
86     cell = [(-10., -300.), (10., -300.), (10., 300.), (-10., 300.)]
87     policy = _Policy()
88     policy.observations[1] = [(lower, 45.)]
89     self.assertFalse(policy._history_point_in_range(q_upper, cell, 1))
90     self.assertTrue(policy._history_point_in_range(q_lower, cell, 1))
91     policy.observations[1] = [(upper, 315.)]
92     self.assertTrue(policy._history_point_in_range(q_upper, cell, 1))
93     self.assertFalse(policy._history_point_in_range(q_lower, cell, 1))
94     policy.observations[1] += [(lower, 45.)]
95     self.assertTrue(all(policy._history_point_in_range(q, cell, 1)
96                        for q in (q_upper, q_lower)))
97
98     def test_rounded_fixed_error_boundary_is_retained_without_averaging(self):
99         for sign in (-1., 1.):
100             angle = sign * .00500001
101             source = (500. * math.cos(math.radians(angle)),
102                     500. * math.sin(math.radians(angle)))
103             client, policy = _EmitterOracle(source, error=sign), _Policy()
104             for point in ((700., -30.), (700., 30.)):
105                 self.assertEqual(policy._measure(client, point, 1), "no_signal")
106             policy._measure(client, (0., 0.), 1)
107             reading = policy.observations[1][-1][1]
108             angular_error = abs((reading - angle + 180.) % 360. - 180.)
109             self.assertGreater(angular_error, 1.00499)
110             self.assertTrue(contains(policy.regions[1], source))
111             policy._measure(client, (0., 0.), 1)
112             self.assertEqual(policy.observations[1][-1][1], reading)
113             self.assertTrue(contains(policy.regions[1], source))
114
115     def test_collinear_negative_pair_cannot_claim_two_dimensional_cut(self):
116         policy = _Policy()
117         policy.regions[1] = [(50., 0.), (150., 0.)]
118         policy.observations[1] = [(0., 0.), (0., 0.)]
119         policy._negative_history = {1: [(200., 0.), (400., 0.)]}
120         policy._clip_history(1)
121         self.assertEqual(policy.regions[1], [(50., 0.), (150., 0.)])
122
123     def test_singleton_feasible_region_survives_history_refinement(self):
124         policy = _Policy()
125         policy.regions[1] = [(100., 0.)]
126         policy.observations[1] = [(0., 0.), (0., 0.)]
127         policy._negative_history = {1: [(200., -20.), (200., 20.)]}
128         policy._clip_history(1)
129         self.assertEqual(policy.regions[1], [(100., 0.)])
130
131     def test_random_physical_halfplanes_and_radius_causes_retain_true_source(self):
132         # Independent geometric cases, not new holdout/stress benchmark data.
133         rng = random.Random(972213)

```

```

134     for index in range(128):
135         source = (rng.uniform(-900., 900.), rng.uniform(-900., 900.))
136         heading = rng.uniform(0., math.tau)
137         radius = rng.uniform(1000., 1500.)
138         client = _EmitterOracle(source, radius, heading, 1. if index % 2 else -1.)
139         policy = _Policy()
140         for j in range(5):
141             angle = heading + math.pi + rng.uniform(-1.4, 1.4)
142             r = rng.uniform(40., radius) if j % 2 else radius + rng.uniform(1., 700.)
143             point = (source[0] + r * math.cos(angle), source[1] + r * math.sin(angle))
144             self.assertEqual(policy._measure(client, point, 1), "no_signal")
145         for j in range(3):
146             angle = heading + rng.uniform(-1.55, 1.55)
147             r = rng.uniform(30., radius)
148             point = (source[0] + r * math.cos(angle), source[1] + r * math.sin(angle))
149             self.assertEqual(policy._measure(client, point, 1), "direction")
150             self.assertTrue(contains(policy.regions[1], source), (index, j))
151
152
153 if __name__ == "__main__":
154     unittest.main()

```

程序 31: problem4/tests/test_policy.py

```

1  """Coverage geometry and adversarial public-interface regression tests."""
2  from __future__ import annotations
3
4  import math
5  import random
6  import unittest
7
8  from problem4.policy import SearchPolicy, covering_triangles, directional_coverage_points,
   ↪  polar_covering_triangles, polar_coverage_points, triangle_contains
9
10
11 class _PublicOnlyClient:
12     """Small independent oracle; production policy sees responses only."""
13
14     def __init__(self, sources):
15         self.__sources = {source[0]: source[1:] for source in sources}
16         self.__cleared = set()
17         self.position = (0.0, 0.0)
18         self.current_channel = 1
19         self.virtual_time_s = 0.0
20         self.actions = 0
21         self.negative_after_detection = 0
22         self.__detected = set()
23
24     def check_budget(self):
25         if self.actions > 12000:
26             raise RuntimeError("policy failed to terminate within the regression budget")
27         return True
28
29     def _move(self, point):
30         self.check_budget()

```

```

31         self.virtual_time_s += math.dist(point, self.position) / 5.0
32         self.position = tuple(point)
33         self.actions += 1
34
35     def measure(self, point, channel):
36         self._move(point)
37         self.virtual_time_s += 5 + (self.current_channel != channel)
38         self.current_channel = channel
39         result = {"accepted": True, "measure_result": "no_signal"}
40         if channel not in self.__cleared and channel in self.__sources:
41             x, y, radius, direction, error_mode = self.__sources[channel]
42             dx, dy = point[0] - x, point[1] - y
43             distance = math.hypot(dx, dy)
44             visible = direction is None or dx * math.cos(direction) + dy * math.sin(direction) >= -1e-9
45             if distance <= radius + 1e-9 and visible:
46                 result["measure_result"] = "near" if distance <= 5 else "direction"
47                 if distance > 5:
48                     # Fixed at each point, so repetition cannot reduce bias.
49                     sign = 1.0 if error_mode == "positive" else -1.0
50                     if error_mode == "spatial":
51                         sign = math.sin(point[0] * .017 + point[1] * .011 + channel * .97)
52                         result["svd_deg"] = round((math.degrees(math.atan2(-dy, -dx)) + sign) % 360, 2) % 360
53                 self.__detected.add(channel)
54             if result["measure_result"] == "no_signal" and channel in self.__detected:
55                 self.negative_after_detection += 1
56             result["virtual_time_s"] = self.virtual_time_s
57         return result
58
59     def clear(self, point, channel):
60         self._move(point)
61         success = channel not in self.__cleared and channel in self.__sources and math.dist(point,
62         ↪ self.__sources[channel][:2]) <= 20 + 1e-9
63         self.virtual_time_s += 5 if success else 3
64         if success:
65             self.__cleared.add(channel)
66         return {"accepted": True, "clear_result": "success" if success else "no_target_in_range",
67         ↪ "virtual_time_s": self.virtual_time_s}
68
69
70
71     class DirectionalCoverageTests(unittest.TestCase):
72         def test_triangles_have_strictly_subminimum_receive_diameter(self):
73             triangles = covering_triangles()
74             self.assertGreater(len(triangles), 0)
75             for triangle in triangles:
76                 for i in range(3):
77                     self.assertAlmostEqual(math.dist(triangle[i], triangle[(i + 1) % 3]), 990, places=7)
78
79         def test_boundary_and_interior_have_convex_hull_witnesses(self):
80             # Numerical regression of the implementation, not the mathematical proof.
81             triangles = polar_covering_triangles()
82             rng = random.Random(80413)
83             points = [(0.0, 0.0)]

```

```

84     points += [(1800 * math.cos(i * math.tau / 360), 1800 * math.sin(i * math.tau / 360)) for i in
↪     range(360)]
85     for _ in range(400):
86         angle = rng.random() * math.tau
87         radius = 1800 * math.sqrt(rng.random())
88         points.append((radius * math.cos(angle), radius * math.sin(angle)))
89     for point in points:
90         witnesses = [triangle for triangle in triangles if triangle_contains(triangle, point)]
91         self.assertTrue(witnesses, point)
92         self.assertTrue(all(math.dist(point, vertex) < 1000 for vertex in witnesses[0]), point)
93
94     def test_outward_boundary_sources_detectable(self):
95         points = polar_coverage_points()
96         self.assertTrue(any(math.hypot(*point) > 1800 for point in points))
97         self.assertEqual(points[0], (0.0, 0.0))
98         self.assertEqual(len(set(points)), len(points))
99         for index in range(720):
100             angle = index * math.tau / 720
101             source = (1800 * math.cos(angle), 1800 * math.sin(angle))
102             self.assertTrue(any(math.dist(source, point) < 1000 and (point[0] - source[0]) *
↪             math.cos(angle) + (point[1] - source[1]) * math.sin(angle) >= -1e-8 for point in points),
↪             index)
103
104     def test_polar_cover_has_25_vertices_with_robust_margins(self):
105         self.assertEqual(len(polar_coverage_points()), 25)
106         self.assertGreater(1870 * math.cos(math.pi / 12), 1800)
107         for triangle in polar_covering_triangles():
108             self.assertLess(max(math.dist(a, b) for a in triangle for b in triangle), 984)
109
110     def test_invalid_spacing_cannot_claim_coverage(self):
111         for spacing in (0, -1, 1000, float("nan")):
112             with self.assertRaises(ValueError):
113                 covering_triangles(spacing=spacing)
114
115
116 class MixedPolicyTests(unittest.TestCase):
117     def test_paired_negative_probe_reduces_range_before_optical_clear(self):
118         # The first bearing is received from the west. Both forward probes
119         # overshoot a west-facing source: they are in range but on its back.
120         client = _PublicOnlyClient([(1, 100.0, 0.0, 1000, math.pi, "positive")])
121         policy = SearchPolicy()
122         self.assertEqual(policy._measure(client, (0.0, 0.0), 1), "direction")
123         self.assertTrue(policy._localize(1, client))
124         self.assertGreater(policy.stats.get("paired_negative_clips", 0), 0)
125         self.assertEqual(client.cleared_count_for_test(), 1)
126         self.assertLess(client.actions, 20)
127
128     def test_outward_boundary_minimum_radius_extreme_fixed_errors(self):
129         sources = []
130         for i in range(10):
131             angle = (i + .37) * math.tau / 10
132             sources.append((i * 2 + 1, 1800 * math.cos(angle), 1800 * math.sin(angle), 1000, angle,
↪             "positive" if i % 2 else "negative"))
133         client = _PublicOnlyClient(sources)
134         result = SearchPolicy().run(client)

```

```

135         self.assertEqual(client.cleared_count_for_test(), 10)
136         self.assertEqual(set(result["cleared_channels"]), {source[0] for source in sources})
137         self.assertTrue(result["coverage_complete"])
138
139     def test_mixed_clustered_and_tangent_directions(self):
140         sources = []
141         for i in range(16):
142             angle = (i + .11) * math.tau / 16
143             radius = 1740 if i % 3 else 25 + i * 5
144             heading = None if i % 4 == 0 else angle + math.pi / 2 + (1e-7 if i % 2 else -1e-7)
145             sources.append((i + 1, radius * math.cos(angle), radius * math.sin(angle), 1000 + (i % 4) *
146                             ↪ 160, heading, "spatial"))
147         client = _PublicOnlyClient(sources)
148         result = SearchPolicy().run(client)
149         self.assertEqual(client.cleared_count_for_test(), 16)
150         self.assertEqual(len(result["cleared_channels"]), 16)
151
152     def test_wrong_problem_rejected(self):
153         with self.assertRaises(ValueError):
154             SearchPolicy(problem=3)
155
156 if __name__ == "__main__":
157     unittest.main()

```

程序 32: problem4/tests/test_routing.py

```

1  """Route transformations preserve all visits and improve real geometric cost."""
2  import math
3  import random
4  import unittest
5  from types import SimpleNamespace
6  from problem3.geometry import distance
7  from problem4.legacy_policy import SearchPolicy as LegacyPolicy
8  from problem4.routing import RoutingMixin,nearest_seed,two_opt,or_opt,route_length
9
10 class RoutingTests(unittest.TestCase):
11     def test_oropt_escapes_a_two_opt_local_minimum(self):
12         points=[(-814,603),(1186,-783),(408,-83),(-751,755),(1020,-1188),
13                (-941,-456),(638,597),(15,1),(-322,-1328),(271,-718),
14                (1489,668),(975,1237),(-1134,652)]
15         matrix=[[distance(a,b) for b in points] for a in points]
16         local=two_opt(nearest_seed(matrix),matrix)
17         escaped=or_opt(local,matrix)
18         self.assertEqual(sorted(local),sorted(escaped))
19         self.assertGreater(route_length(local,matrix)-route_length(escaped,matrix),60)
20         self.assertEqual(local,two_opt(local,matrix))
21
22     def test_random_routes_preserve_visits_and_never_increase_length(self):
23         rng=random.Random(261109)
24         for count in (0,1,3,12,30):
25             for _ in range(4):
26                 points=[(rng.uniform(-2000,2000),rng.uniform(-2000,2000)) for _ in range(count+1)]
27                 matrix=[[distance(a,b) for b in points] for a in points]
28                 seed=nearest_seed(matrix);first=two_opt(seed,matrix);last=or_opt(first,matrix)

```



```

29         self.assertEqual(sorted(last),list(range(count)))
30         self.assertEqual(route_length(last,matrix),route_length(first,matrix)+1e-7)
31         self.assertEqual(route_length(first,matrix),route_length(seed,matrix)+1e-7)
32
33     def test_sixteen_detected_stops_scanning_without_claiming_completion(self):
34         class Policy(RoutingMixin,LegacyPolicy):pass
35         policy=Policy()
36         for channel in range(1,17):
37             angle=channel*math.tau/16;x,y=800*math.cos(angle),800*math.sin(angle)
38             policy.regions[channel]=[(x-1,y-1),(x+1,y-1),(x,y+1)]
39             pending=[(1800.,0.), (0.,1800.), (-1800.,0.)]
40             goal=policy._route_goal(SimpleNamespace(position=(0.,0.)),pending)
41             self.assertEqual(goal[1],'clear')
42             self.assertEqual(len(pending),3)
43             self.assertFalse(policy._result('test',False)['completion_certified'])
44
45 if __name__=='__main__':unittest.main()

```

程序 33: problem4/tests/test_v2_integration.py

```

1  """Version-2 public-interface integration and matched-comparison semantics."""
2  from __future__ import annotations
3
4  import math
5  import unittest
6  from types import SimpleNamespace
7  from unittest.mock import patch
8
9  from problem3.scenarios import generate_case
10 from problem3.simulator import LocalSimulator, ObservationClient
11 from problem4.coverage import static_coverage_points
12 from problem4.legacy_policy import SearchPolicy as LegacyPolicy
13 from problem4.policy import SearchPolicy
14 from scripts.benchmark_search import get_policy, run_case
15
16
17 def _near_origin_case(count):
18     # Public physical edge case, independent of held-out benchmark suites.
19     sources = [dict(channel=1, x=-1.0, y=0.0, radius=1000.0, orientation_deg=0.0)]
20     sources += [dict(channel=i, x=math.cos(i), y=math.sin(i), radius=1000.0,
21                     orientation_deg=None) for i in range(2, count + 1)]
22     return dict(case_id=f"integration_near_origin_{count}", problem=4,
23               seed=991237 + count, split="unit", family="close_pairs",
24               noise="extreme", sources=sources)
25
26
27 class Version2IntegrationTests(unittest.TestCase):
28     def test_default_and_legacy_entry_points_keep_distinct_layouts(self):
29         self.assertEqual(get_policy(4, "adaptive").coverage_points(), static_coverage_points())
30         self.assertIsInstance(get_policy(4, "legacy"), LegacyPolicy)
31         self.assertNotIsInstance(get_policy(4, "legacy"), SearchPolicy)
32         self.assertEqual(len(get_policy(4, "legacy").coverage_points()), 25)
33         self.assertEqual(len(get_policy(4, "baseline").coverage_points()), 31)
34         self.assertEqual(len(get_policy(4, "optical").coverage_points()), 25)
35         with self.assertRaises(ValueError):

```

```

36         get_policy(3, "legacy")
37
38     def test_legacy_alias_replays_the_original_policy_actions(self):
39         case = generate_case(4, 2, "development_v2")
40         alias, alias_events = run_case(case, "legacy", record=True)
41         sim = LocalSimulator(case, record=True)
42         sim.enter()
43         LegacyPolicy().run(ObservationClient(sim))
44         sim.exit()
45
46     def physical_trace(events):
47         return [(event["action"], event["position"], event["channel"],
48                 {k: v for k, v in event["response"].items() if k != "real_timestamp_ms"})
49                 for event in events]
50
51     self.assertEqual(alias["strategy"], "legacy")
52     self.assertEqual(physical_trace(alias_events), physical_trace(sim.events))
53     self.assertEqual(alias["virtual_time_s"], sim.virtual_time_s)
54
55     def test_sixteen_successful_clears_certify_early_exit_without_fake_coverage(self):
56         row, _ = run_case(_near_origin_case(16))
57         self.assertIsNone(row["error"])
58         self.assertEqual(row["cleared_count"], 16)
59         self.assertTrue(row["completion_certified"])
60         self.assertFalse(row["coverage_complete"])
61         self.assertEqual(row["policy"]["termination_reason"], "maximum_source_count_cleared")
62         self.assertLess(row["policy"]["coverage_points_visited"], 23)
63
64     def test_sixteen_discoveries_omit_scans_but_do_not_certify_clearance(self):
65         policy = SearchPolicy()
66         square = [(99.0, -1.0), (101.0, -1.0), (101.0, 1.0), (99.0, 1.0)]
67         policy.regions = {channel: list(square) for channel in range(1, 17)}
68         policy.cleared = set(range(1, 16))
69         pending = policy.coverage_points()
70         retained = list(pending)
71         goal = policy._route_goal(SimpleNamespace(position=(0.0, 0.0)), pending)
72         self.assertEqual(goal[1:], ("clear", 16))
73         self.assertEqual(pending, retained)
74         self.assertFalse(policy._result("not_finished", False)["completion_certified"])
75
76     def test_ten_early_clears_still_pay_for_complete_unknown_channel_search(self):
77         row, _ = run_case(_near_origin_case(10))
78         self.assertIsNone(row["error"])
79         self.assertEqual(row["cleared_count"], 10)
80         self.assertTrue(row["coverage_complete"])
81         self.assertEqual(row["policy"]["coverage_points_visited"], 23)
82         self.assertGreater(row["virtual_time_s"], row["last_clear_time_s"] + 1000)
83         self.assertAlmostEqual(row["average_clear_time_s"], row["virtual_time_s"] / 10)
84
85     def test_fast_uncertified_exit_does_not_earn_a_threshold_reward(self):
86         class UncertifiedPolicy:
87             def run(self, client):
88                 for channel in range(1, 11):
89                     client.clear((0.0, 0.0), channel)
90                 return {"cleared_channels": list(range(1, 11)), "coverage_complete": False,

```

```

91         "completion_certified": False, "termination_reason": "insufficient_evidence"}
92
93     with patch("scripts.benchmark_search.get_policy", return_value=UncertifiedPolicy()):
94         row, _ = run_case(_near_origin_case(10))
95         self.assertEqual(row["cleared_count"], 10)
96         self.assertLess(row["virtual_time_s"], 100)
97         self.assertFalse(row["certified_full_clear"])
98         self.assertFalse(row["threshold_passed"])
99         self.assertLessEqual(row["reward"], -1000000)
100
101
102 if __name__ == "__main__":
103     unittest.main()

```

程序 34: scripts/benchmark_search.py

```

1  """Compare observation-only policies on matched synthetic cases.
2
3  Development cases are for selection. Holdout/stress results must never be used
4  to silently retune a selected policy. Reports retain every failure and seed.
5  """
6  from __future__ import annotations
7
8  import argparse
9  import hashlib
10 import json
11 import math
12 import statistics
13 import sys
14 import time
15 from pathlib import Path
16
17 ROOT = Path(__file__).resolve().parents[1]
18 sys.path.insert(0, str(ROOT))
19 from problem3.scenarios import SPLITS, FAMILIES, generate_suite
20 from problem3.simulator import LocalSimulator, ObservationClient
21
22 THRESHOLDS = {3: 300.0, 4: 500.0}
23
24
25 def get_policy(problem, strategy, config=None):
26     if strategy == 'legacy':
27         if problem != 4:
28             raise ValueError('legacy is the previous problem4 adaptive policy')
29         from problem4.legacy_policy import SearchPolicy
30         return SearchPolicy(problem=4, strategy='adaptive', config=config)
31     if problem == 3:
32         from problem3.policy import SearchPolicy
33     else:
34         from problem4.policy import SearchPolicy
35     return SearchPolicy(problem=problem, strategy=strategy, config=config)
36
37
38 def run_case(case, strategy='adaptive', config=None, record=False):
39     sim = LocalSimulator(case, record=record)

```

```

40     sim.enter()
41     start = time.perf_counter()
42     result, error = {}, None
43     try:
44         result = get_policy(case['problem'], strategy, config).run(ObservationClient(sim))
45     except Exception as exc:
46         error = f'{type(exc).__name__}: {exc}'
47     elapsed = time.perf_counter()-start
48     if sim.active:
49         sim.exit()
50     stats = sim.statistics()
51     complete = bool(result.get('completion_certified', result.get('coverage_complete')))
52     full = stats['cleared_count'] == stats['source_count']
53     certified = full and complete and not error
54     threshold = THRESHOLDS[case['problem']]*stats['source_count']
55     passed = certified and stats['virtual_time_s'] <= threshold
56     # A miss/uncertified exit can never be rewarded for being fast.
57     reward = (-1000000*(stats['source_count']-stats['cleared_count'])-1000000
58              if not certified else
59              1000*int(passed) + max(-1000, 100*(1-stats['virtual_time_s']/threshold)))
60     row = dict(case_id=case['case_id'], seed=case['seed'], split=case['split'],
61              family=case['family'], noise=case['noise'], strategy=strategy,
62              **stats, coverage_complete=bool(result.get('coverage_complete')),
63              completion_certified=complete, certified_full_clear=certified,
64              threshold_total_s=threshold, threshold_passed=passed, reward=reward,
65              program_runtime_s=elapsed, error=error, policy=result)
66     return row, sim.events
67
68
69 def quantile(values, q):
70     ordered = sorted(values)
71     return ordered[max(0, math.ceil(q*len(ordered))-1)] if ordered else None
72
73
74 def aggregate(rows):
75     times = [r['virtual_time_s'] for r in rows]
76     averages = [r['average_clear_time_s'] for r in rows if r['average_clear_time_s'] is not None]
77     return dict(cases=len(rows), all_clear_cases=sum(r['certified_full_clear'] for r in rows),
78               missed_sources=sum(r['source_count']-r['cleared_count'] for r in rows),
79               threshold_pass_rate=statistics.mean(r['threshold_passed'] for r in rows),
80               mean_virtual_time_s=statistics.mean(times), p95_virtual_time_s=quantile(times,.95),
81               worst_virtual_time_s=max(times),
82               mean_average_clear_time_s=statistics.mean(averages) if averages else None,
83               mean_actions=statistics.mean(r['actions'] for r in rows),
84               mean_failed_clear=statistics.mean(r['failed_clear'] for r in rows),
85               mean_movement_m=statistics.mean(r['movement_m'] for r in rows),
86               mean_reward=statistics.mean(r['reward'] for r in rows),
87               total_program_runtime_s=sum(r['program_runtime_s'] for r in rows),
88               max_program_runtime_s=max(r['program_runtime_s'] for r in rows),
89               failures=[r['case_id'] for r in rows if not r['certified_full_clear']],
90               threshold_curve={str(t): statistics.mean(r['certified_full_clear'] and
91               r['average_clear_time_s'] <= t for r in rows)
92               for t in (100,150,200,250,300,400,500,700,1000)})
93
94

```

```

95 def source_fingerprint():
96     paths = [ROOT/'problem3'/f for f in ('policy.py','geometry.py','simulator.py','scenarios.py')]
97     paths += [ROOT/'problem4'/f for f in
98         ('policy.py','legacy_policy.py','localization.py','coverage.py','routing.py','adaptive_cove
99         ↪ rage.py')]
100     paths += [Path(__file__).resolve()]
101     return {p.relative_to(ROOT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest()
102         for p in paths if p.exists()}
103
104 def main():
105     parser = argparse.ArgumentParser(description=__doc__)
106     parser.add_argument('--problem', type=int, choices=(3,4), required=True)
107     parser.add_argument('--split', choices=tuple(SPLITS), default='development')
108     parser.add_argument('--count', type=int, help='Default: full predeclared split')
109     parser.add_argument('--strategies', nargs='+', default=['baseline','adaptive'])
110     parser.add_argument('--config', type=Path, help='Optional fixed policy configuration JSON')
111     parser.add_argument('--tag', default='', help='Distinct result suffix for development experiments')
112     parser.add_argument('--save-cases', action='store_true')
113     args = parser.parse_args()
114     if args.count is not None and args.count <= 0:
115         parser.error('--count must be positive')
116     if args.tag and not all(c.isalnum() or c=='_' for c in args.tag):
117         parser.error('--tag may only contain letters, digits and underscores')
118     cases = generate_suite(args.problem, args.split, args.count)
119     config = json.loads(args.config.read_text(encoding='utf-8')) if args.config else None
120     rows = []
121     fingerprints = source_fingerprint()
122     for strategy in args.strategies:
123         for i, case in enumerate(cases):
124             row, _ = run_case(case, strategy, config)
125             rows.append(row)
126             if (i+1)%24 == 0 or row['error'] or i+1 == len(cases):
127                 print(f"p{args.problem} {strategy} {args.split} {i+1}/{len(cases)} "
128                     f"cleared={row['cleared_count']}/{row['source_count']} "
129                     f"T={row['virtual_time_s']:.1f}s error={row['error']}", flush=True)
130     if fingerprints != source_fingerprint():
131         raise RuntimeError('Algorithm changed during benchmark; rerun after freezing code')
132     summary = {s: aggregate([r for r in rows if r['strategy']==s]) for s in args.strategies}
133     by_family = {s: {f: aggregate([r for r in rows if r['strategy']==s and r['family']==f])
134         for f in FAMILIES if any(r['family']==f for r in rows)} for s in args.strategies}
135     body = dict(provenance='Local synthetic results, NOT official practice/formal results',
136         problem=args.problem, split=args.split, case_count=len(cases),
137         cases_sha256=hashlib.sha256(json.dumps(cases, sort_keys=True, separators=(',', ':'),
138             ensure_ascii=False).encode('utf-8')).hexdigest(),
139         threshold_seconds_per_source=THRESHOLDS[args.problem],
140         threshold_definition='Full clearance AND completion certificate; total virtual time <=
141         ↪ threshold * N',
142         config=config, source_sha256=fingerprints, summary=summary, by_family=by_family,
143         ↪ episodes=rows)
144     folder = ROOT/f'problem{args.problem}/results'
145     folder.mkdir(parents=True, exist_ok=True)
146     stem = f"benchmark_{args.split}" + ('_'+args.tag if args.tag else '')
147     (folder/f'{stem}.json').write_text(json.dumps(body, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
148     lines = [f'# 问题 {args.problem} 自建样本比较: {args.split}', '',

```

```

147         f'每分支 {len(cases)} 例；所有分支逐例使用相同样本和固定地点误差。不是官方演练或正式成绩。', '',
148         f'自定阈值：确保全清后，总虚拟时间 ≤ {THRESHOLDS[args.problem]:g} × 实际源数（秒）。
        ↳ 源数只供事后评分，算法不可读取。', '',
149         '| 分支 | 全清且完成确认 | 阈值通过率 | 平均总耗时/秒 | P95/秒 | 最差/秒 | 平均每源/秒 |
        ↳ 平均动作数 |',
150         '| --- | --- | --- | --- | --- | --- | --- | --- |'
151     for s,a in summary.items():
152         average = '无定义' if a['mean_average_clear_time_s'] is None else
        ↳ f"{a['mean_average_clear_time_s']:.1f}"
153         lines.append(f"| {s} | {a['all_clear_cases']}/{a['cases']} | {a['threshold_pass_rate']:.1%} | "
154                      f"{a['mean_virtual_time_s']:.1f} | {a['p95_virtual_time_s']:.1f} |
        ↳ {a['worst_virtual_time_s']:.1f} | "
155                      f"{average} | {a['mean_actions']:.1f} |")
156     lines += [' ', '奖励：未全清或未完成确认给至少一百万负分；全清且过阈值加1000分，再按节省的时间加分。
        ↳ 此奖励只用于本地比较，题目没有给官方奖励函数。', '',
157             '总耗时包括发现最后一个源后的排除/覆盖确认，不只统计最后一次清除时刻。P95为最近秩分位数；
        ↳ 本地程序运行耗时不含HTTP和平台延迟。', '',
158             '下表使用相同结果扫描更严格/宽松阈值，没有重新训练或调参。', '',
159             '| 分支 | 150 秒/源 | 200 秒/源 | 250 秒/源 | 300 秒/源 | 400 秒/源 | 500 秒/源 |',
160             '| --- | --- | --- | --- | --- | --- | --- |']
161     for s,a in summary.items():
162         lines.append('| '+s+' | '+' | '.join(f"{a['threshold_curve'][str(t)]:.1%}" for t in
        ↳ (150,200,250,300,400,500))+ ' |')
163     lines += [' ', f'逐例数据、分场景汇总、失败记录及代码指纹见 [{stem}.json]({stem}.json)。', ''
164             (folder/f'{stem}.md').write_text('\n'.join(lines),encoding='utf-8')
165     if args.save_cases:
166         ex = ROOT/f'problem{args.problem}/examples'; ex.mkdir(parents=True,exist_ok=True)
167         (ex/f'{args.split}_cases.json').write_text(json.dumps(cases,ensure_ascii=False,indent=2)+'\n',enc
        ↳ oding='utf-8')
168     print(json.dumps(summary,ensure_ascii=False,indent=2))
169     return 0 if all(r['certified_full_clear'] for r in rows) else 1
170
171
172 if __name__ == '__main__':
173     raise SystemExit(main())

```

程序 35: scripts/plot_search.py

```

1  """Optional local benchmark figure (requires matplotlib; not needed to solve)."""
2  from pathlib import Path
3  import json
4  import matplotlib
5  matplotlib.use('Agg')
6  import matplotlib.pyplot as plt
7
8  ROOT = Path(__file__).resolve().parents[1]
9
10
11 def main():
12     colors = {'development': '#2b6cb0', 'holdout': '#2f855a', 'stress': '#c05621'}
13     fig, axes = plt.subplots(2, 2, figsize=(12, 8), constrained_layout=True)
14     for row, problem in enumerate((3,4)):
15         folder = ROOT/f'problem{problem}/results'
16         data = json.loads((folder/'benchmark_development.json').read_text(encoding='utf-8'))
17         variants = ('baseline', 'adaptive', 'optical')

```

```

18     values = [data['summary'][s]['mean_average_clear_time_s'] for s in variants]
19     ax = axes[row,0]
20     bars = ax.bar(variants, values, color=['#a0aec0','#2b6cb0','#718096'], width=.62)
21     for b,v in zip(bars,values):
22         ax.text(b.get_x()+b.get_width()/2,v+12,f'{v:.1f}',ha='center',fontsize=10)
23     ax.set_ylim(0,max(values)*1.2)
24     ax.set_ylabel('Mean virtual seconds / cleared source')
25     ax.set_title(f'Problem {problem}: 96 matched development cases')
26     ax.yaxis.grid(alpha=.2); ax.set_axisbelow(True)
27     ax = axes[row,1]
28     for split in ('development','holdout','stress'):
29         data = json.loads((folder/f'benchmark_{split}.json').read_text(encoding='utf-8'))
30         episodes = [r for r in data['episodes'] if r['strategy']=='adaptive']
31         times = sorted(r['average_clear_time_s'] for r in episodes if r['certified_full_clear'])
32         ax.step(times,[(i+1)*100/len(episodes) for i in range(len(times))],where='post',
33                 label=f'{split} (n={len(episodes)})',color=colors[split],linewidth=1.8)
34         threshold = 300 if problem==3 else 500
35         ax.axvline(threshold,color='#b83280',linestyle='--',label=f'Preset: {threshold} s/source')
36         ax.set_xlabel('Allowed virtual seconds / source')
37         ax.set_ylabel('Certified full-clear cases (%)')
38         ax.set_ylim(0,102); ax.grid(alpha=.2); ax.legend(fontsize=8,loc='lower right')
39         ax.set_title(f'Problem {problem}: fixed adaptive policy')
40     fig.suptitle('Synthetic local evaluation - NOT official simulator results',fontsize=15)
41     dest=ROOT/'validation/search_comparison.png'
42     fig.savefig(dest,dpi=170,facecolor='white'); plt.close(fig)
43     print(dest)
44
45
46 if __name__ == '__main__':
47     main()

```

程序 36: scripts/prepare_materials.py

```

1  """Copy B originals and create searchable companions; paths are repo-relative."""
2  from __future__ import annotations
3
4  import hashlib
5  import json
6  import shutil
7  from pathlib import Path
8  from xml.etree import ElementTree as ET
9  from zipfile import ZipFile
10
11  ROOT = Path(__file__).resolve().parents[1]
12  MATERIALS = ROOT / "materials" / "problem_b"
13  SOURCES = (
14      ("CUMCM2026Problems/B 题/B 题.pdf", "statement.pdf"),
15      ("CUMCM2026Problems/B 题/附件/附件 1.docx", "attachments/attachment_1_simulator_guide.docx"),
16      ("CUMCM2026Problems/B 题/附件/附件 2.docx", "attachments/attachment_2_api_protocol.docx"),
17  )
18  NS = {"w": "http://schemas.openxmlformats.org/wordprocessingml/2006/main",
19        "m": "http://schemas.openxmlformats.org/officeDocument/2006/math"}
20
21
22  def sha256(path: Path) -> str:

```

```

23     return hashlib.sha256(path.read_bytes()).hexdigest()
24
25
26 def inline_text(node: ET.Element) -> str:
27     """Keep both Word text and OMML math tokens (math layout is flattened)."""
28     parts = []
29     for item in node.iter():
30         if item.tag in (f"{{{NS['w']}}t}", f"{{{NS['m']}}t}"):
31             parts.append(item.text or "")
32         elif item.tag == f"{{{NS['w']}}tab":
33             parts.append(" ")
34         elif item.tag in (f"{{{NS['w']}}br", f"{{{NS['w']}}cr"):
35             parts.append("\n")
36     return "".join(parts)
37
38
39 def docx_markdown(path: Path) -> tuple[str, dict]:
40     with ZipFile(path) as archive:
41         bad = archive.testzip()
42         if bad:
43             raise ValueError(f"Damaged DOCX member: {path}: {bad}")
44         doc = ET.fromstring(archive.read("word/document.xml"))
45     body = doc.find("w:body", NS)
46     if body is None:
47         raise ValueError(f"Missing document body: {path}")
48     lines = [f"# {path.stem}", "",
49             "> 本文件由原 Word 正文按段落和表格顺序自动提取，供检索和程序读取。",
50             "> 已保留 Word 公式中的文字，但上下标、分式等结构被线性化；图像未转录，公式与图示请以原 DOCX 为准。",
51             "↪ ", ""]
52     tables = 0
53     for block in body:
54         if block.tag == f"{{{NS['w']}}p":
55             value = inline_text(block)
56             if value.strip():
57                 lines.extend([value, ""])
58         elif block.tag == f"{{{NS['w']}}tbl":
59             tables += 1
60             rows = []
61             for row in block.findall("w:tr", NS):
62                 cells = []
63                 for cell in row.findall("w:tc", NS):
64                     value = "<br>".join(inline_text(p) for p in cell.findall("w:p", NS))
65                     cells.append(value.replace("|", "\\|").replace("\n", "<br>"))
66                 rows.append(cells)
67             if rows:
68                 width = max(map(len, rows))
69                 rows = [r + [""] * (width - len(r)) for r in rows]
70                 lines.append("| " + " | ".join(rows[0]) + " |")
71                 lines.append("| " + " | ".join(["---"] * width) + " |")
72                 lines.extend("| " + " | ".join(row) + " |" for row in rows[1:])
73                 lines.append("")
74     return "\n".join(lines), {
75         "table_count": tables,
76         "math_expression_count": len(body.findall("./m:oMath", NS)),
77         "text_token_count": sum(1 for n in body.iter() if n.tag in (f"{{{NS['w']}}t}", f"{{{NS['m']}}t}")),

```



```

77     }
78
79
80 def main() -> None:
81     from pypdf import PdfReader
82
83     MATERIALS.mkdir(parents=True, exist_ok=True)
84     (MATERIALS / "extracted").mkdir(exist_ok=True)
85     entries = []
86     for original, relative in SOURCES:
87         source = ROOT / original
88         target = MATERIALS / relative
89         target.parent.mkdir(parents=True, exist_ok=True)
90         digest = sha256(source)
91         if target.exists() and sha256(target) != digest:
92             raise ValueError(f"Refusing to overwrite changed material: {target}")
93         if not target.exists():
94             shutil.copy2(source, target)
95         entry = {"source": original, "working_copy": target.relative_to(ROOT).as_posix(),
96                 "bytes": source.stat().st_size, "sha256": digest}
97         if target.suffix == ".pdf":
98             reader = PdfReader(target)
99             text = "# B 题题面检索文本\n\n> 自动提取; 公式排版、图示和原始表格以 statement.pdf 为准。 \n\n"
100            text += "\n\n".join(f"## 原题第 {i} 页\n\n{page.extract_text()}" for i, page in
101                                ↪ enumerate(reader.pages, 1))
102            entry["page_count"] = len(reader.pages)
103        else:
104            text, statistics = docx_markdown(target)
105            entry.update(statistics)
106            extracted = MATERIALS / "extracted" / f"{target.stem}.md"
107            text = "\n".join(line.rstrip() for line in text.splitlines()).rstrip() + "\n"
108            extracted.write_text(text, encoding="utf-8", newline="\n")
109            entry["extracted_text"] = extracted.relative_to(ROOT).as_posix()
110            entry["extracted_sha256"] = sha256(extracted)
111            entries.append(entry)
112        manifest = {"schema_version": 1, "policy": "Original bundle retained; working copies are
113                    ↪ byte-identical.", "files": entries}
114        (MATERIALS / "manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2) + "\n",
115                    ↪ encoding="utf-8", newline="\n")
116        print(f"Prepared {len(entries)} B materials and searchable companions.")
117
118
119 if __name__ == "__main__":
120     main()

```

程序 37: scripts/report_problem4_speed.py

```

1  """Summarize frozen, paired Q4 speed experiments; standard library only."""
2  from __future__ import annotations
3
4  import json
5  import random
6  import statistics
7  from pathlib import Path
8

```

```

9 ROOT = Path(__file__).resolve().parents[1]
10 RESULTS = ROOT / 'problem4/results'
11 SPLITS = ('development_v2', 'holdout_v2', 'stress_v2')
12
13
14 def paired(rows):
15     old = {r['case_id']: r for r in rows if r['strategy'] == 'legacy'}
16     new = {r['case_id']: r for r in rows if r['strategy'] == 'adaptive'}
17     if old.keys() != new.keys() or not old:
18         raise ValueError('The two branches must use exactly the same nonempty cases')
19     pairs = [(old[k], new[k]) for k in sorted(old)]
20     if any(a['source_count'] != b['source_count'] for a, b in pairs):
21         raise ValueError('Mismatched source counts')
22     delta = [a['average_clear_time_s'] - b['average_clear_time_s'] for a, b in pairs]
23     before = statistics.mean(a['average_clear_time_s'] for a, _ in pairs)
24     after = statistics.mean(b['average_clear_time_s'] for _, b in pairs)
25     rng = random.Random(260911)
26     bootstrap = sorted(statistics.mean(rng.choices(delta, k=len(delta))) for _ in range(4000))
27     return dict(cases=len(pairs), old_seconds_per_source=before, new_seconds_per_source=after,
28               reduction_fraction=1-after/before,
29               mean_saved_seconds_per_source=statistics.mean(delta),
30               bootstrap_95_mean_saved_seconds_per_source=[bootstrap[99], bootstrap[3899]],
31               faster_cases=sum(d > .000001 for d in delta),
32               slower_cases=sum(d < -.000001 for d in delta),
33               tied_cases=sum(abs(d) <= .000001 for d in delta),
34               gained_threshold_cases=sum(not a['threshold_passed'] and b['threshold_passed'] for a,b in
35               ↪ pairs),
36               lost_threshold_cases=sum(a['threshold_passed'] and not b['threshold_passed'] for a,b in
37               ↪ pairs),
38               old_threshold_pass_rate=statistics.mean(a['threshold_passed'] for a,_ in pairs),
39               new_threshold_pass_rate=statistics.mean(b['threshold_passed'] for _,b in pairs))
40
41 def main():
42     reports = {s: json.loads((RESULTS/f'benchmark_{s}.json').read_text(encoding='utf-8')) for s in SPLITS}
43     fingerprints = [r['source_sha256'] for r in reports.values()]
44     if any(f != fingerprints[0] for f in fingerprints[1:]):
45         raise ValueError('Benchmarks were not generated with the same frozen sources')
46     frozen = json.loads((ROOT/'validation/problem4_freeze_v2.json').read_text(encoding='utf-8'))
47     if frozen['source_sha256'] != fingerprints[0]:
48         raise ValueError('Evaluation differs from the recorded pre-holdout freeze')
49     seeds = [set(r['seed'] for r in report['episodes']) for report in reports.values()]
50     if any(seeds[i] & seeds[j] for i in range(len(seeds)) for j in range(i)):
51         raise ValueError('Split seed overlap')
52     comparison = {split: paired(report['episodes']) for split,report in reports.items()}
53     stratified = {split: {str(n): paired([r for r in report['episodes'] if r['source_count']==n])
54                          for n in range(10,17) if any(r['source_count']==n for r in report['episodes'])}}
55     for split,report in reports.items()
56     body = dict(provenance='Synthetic local evaluation, not official platform results',
57               source_sha256=fingerprints[0], paired=comparison, by_source_count=stratified,
58               uncertainty_note='Paired bootstrap conditional on these synthetic distributions; not
59               ↪ official performance bounds')
60     (RESULTS/'speed_comparison_v2.json').write_text(json.dumps(body,ensure_ascii=False,indent=2)+'\n',enc
61     ↪ oding='utf-8')
62     lines = ['# 第四问第二轮速度改进', '',

```

```

60         '对手是上一版已交付的 `adaptive`（本轮名为 `legacy`），不是更弱的基础分支。两版逐例使用相同输入、
        ↳ 固定地点误差和计时。以下均为自建模拟，未连接官方平台。', '',
61         '阈值保持 **500秒/源**：全清且取得完成确认后，总虚拟时间不超过500×实际源数。
        ↳ 总时间包含最后一次清除后的覆盖确认：算法不能读取实际源数。', '',
62         '| 集合 | 案例数 | 旧版→新版 秒/源 | 耗时减少 | 旧版→新版 阈值通过率 | 新版全清且确认 |',
63         '| --- | ---: | ---: | ---: | ---: | ---: |']
64 labels = {'development_v2': '开发', 'holdout_v2': '新留出', 'stress_v2': '新压力'}
65 for split, p in comparison.items():
66     a = reports[split]['summary']['adaptive']
67     lines.append(f"| {labels[split]} | {p['cases']} | {p['old_seconds_per_source']:.2f} →
        ↳ {p['new_seconds_per_source']:.2f} | {p['reduction_fraction']:.1%} |
        ↳ {p['old_threshold_pass_rate']:.1%} → {p['new_threshold_pass_rate']:.1%} |
        ↳ {a['all_clear_cases']}/{a['cases']} |")
68 lines += ['', '## 成本与逐例变化', '',
69           '| 集合 | 移动距离 米/局（旧→新） | 动作数/局（旧→新） | 总时间P95 秒（旧→新） | 新版更快/更慢
        ↳ | 新增/失去阈值通过 |',
70           '| --- | ---: | ---: | ---: | ---: | ---: |']
71 for split, p in comparison.items():
72     a, b = (reports[split]['summary'][s] for s in ('legacy', 'adaptive'))
73     lines.append(f"| {labels[split]} | {a['mean_movement_m']:.0f} → {b['mean_movement_m']:.0f} |
        ↳ {a['mean_actions']:.1f} → {b['mean_actions']:.1f} | {a['p95_virtual_time_s']:.0f} →
        ↳ {b['p95_virtual_time_s']:.0f} | {p['faster_cases']}/{p['slower_cases']} |
        ↳ {p['gained_threshold_cases']}/{p['lost_threshold_cases']} |")
74 lines += ['', '提速不是逐例保证：更慢及失去阈值的案例没有删除。JSON保留按源数分层与配对自助法区间；
        ↳ 该区间只描述本地生成分布，不表示官方成绩区间。', '',
75           '## 少源场景是否仍慢', '',
76           '| 集合 | 实际源数 | 案例数 | 旧版→新版 秒/源 | 旧版→新版 阈值通过率 |',
77           '| --- | ---: | ---: | ---: | ---: |']
78 for split in ('holdout_v2', 'stress_v2'):
79     for n, p in stratified[split].items():
80         lines.append(f"| {labels[split]} | {n} | {p['cases']} | {p['old_seconds_per_source']:.1f} →
        ↳ {p['new_seconds_per_source']:.1f} | {p['old_threshold_pass_rate']:.1%} →
        ↳ {p['new_threshold_pass_rate']:.1%} |")
81 lines += ['', '源数分组只用于事后分析。发现16个频道可停止寻找新频道，但必须实际清除16个才结束；
        ↳ 少于16个时仍要完成覆盖。', '',
82           '## 复现', '', '```powershell',
83           *[f'python scripts/benchmark_search.py --problem 4 --split {s} --strategies legacy adaptive
        ↳ --save-cases' for s in SPLITS],
84           'python scripts/report_problem4_speed.py', '```', '',
85           '新版集合与第一轮集合种子互不重叠。开发384例用于选型；代码冻结后再运行新留出512例、新压力256例。
        ↳ 全部1280个输入保存在 `problem4/examples/`，不按结果删样本。', '',
86           '每组完整明细：'+ '、'.join(f"[{labels[s]}](benchmark_{s}.md)" for s in SPLITS)+'；
        ↳ [配对统计JSON](speed_comparison_v2.json)。', ''
87 (RESULTS/'speed_comparison_v2.md').write_text('\n'.join(lines), encoding='utf-8')
88 print(json.dumps(comparison, ensure_ascii=False, indent=2))
89
90
91 if __name__ == '__main__':
92     main()

```

程序 38: scripts/verify_project.py

```

1 """Verify preserved inputs, four solutions, and their regression suites."""
2 from __future__ import annotations
3

```

```

4 import hashlib
5 import argparse
6 import csv
7 import json
8 import os
9 import re
10 import subprocess
11 import sys
12 import tempfile
13 import time
14 from datetime import datetime, timezone
15 from pathlib import Path
16 from xml.etree import ElementTree as ET
17 from zipfile import ZipFile
18
19 ROOT = Path(__file__).resolve().parents[1]
20 REPORT_DIR = ROOT / "validation"
21
22
23 def sha256(path: Path) -> str:
24     return hashlib.sha256(path.read_bytes()).hexdigest()
25
26
27 def main() -> int:
28     parser = argparse.ArgumentParser(description=__doc__)
29     parser.add_argument("--reference", action="store_true",
30                         help="Also run the optional independent SciPy/HiGHS geometry check")
31     args = parser.parse_args()
32     checks = []
33
34     def record(name, action):
35         started = time.perf_counter()
36         try:
37             detail = action()
38             check = {"name": name, "passed": True, "detail": detail}
39         except Exception as exc:
40             check = {"name": name, "passed": False, "detail": str(exc)}
41         check["elapsed_seconds"] = round(time.perf_counter() - started, 3)
42         checks.append(check)
43         print(f"{'PASS' if check['passed'] else 'FAIL'} {name}", flush=True)
44
45     def materials():
46         manifest = json.loads((ROOT / "materials/problem_b/manifest.json").read_text(encoding="utf-8"))
47         if len(manifest["files"]) != 3:
48             raise AssertionError("Expected exactly three original B material files")
49         for entry in manifest["files"]:
50             for key in ("source", "working_copy"):
51                 path = ROOT / entry[key]
52                 if path.stat().st_size != entry["bytes"] or sha256(path) != entry["sha256"]:
53                     raise AssertionError(f"Material content mismatch: {entry[key]}")
54             extracted = ROOT / entry["extracted_text"]
55             if sha256(extracted) != entry["extracted_sha256"]:
56                 raise AssertionError(f"Extracted text mismatch: {extracted.name}")
57             if not extracted.read_text(encoding="utf-8").strip():
58                 raise AssertionError(f"Empty extracted text: {extracted.name}")

```

```

59         copy = ROOT / entry["working_copy"]
60         if copy.suffix == ".docx":
61             with ZipFile(copy) as archive:
62                 if archive.testzip() is not None:
63                     raise AssertionError(f"Corrupt DOCX: {copy.name}")
64                     ET.fromstring(archive.read("word/document.xml"))
65             elif not copy.read_bytes().startswith(b"%PDF-"):
66                 raise AssertionError("Missing PDF header")
67         return {"original_files_checked": 3, "searchable_text_files_checked": 3,
68               "method": "SHA-256 and byte counts; DOCX ZIP CRC and XML parsing; PDF signature"}
69
70     def command(args: list[str], cwd: Path):
71         env = dict(os.environ, PYTHONIOENCODING="utf-8", PYTHONUTF8="1")
72         completed = subprocess.run([sys.executable, *args], cwd=cwd, env=env,
73                                   capture_output=True, text=True, encoding="utf-8", timeout=240)
74         output = (completed.stdout + completed.stderr).strip()
75         if completed.returncode:
76             raise AssertionError(f"Exit {completed.returncode}\n{output}")
77         return {"exit_code": 0, "output": output}
78
79     record("original_materials_and_searchable_copies", materials)
80     for problem in ("problem1", "problem2", "problem3", "problem4"):
81         def suite(problem=problem):
82             tests = ROOT / problem / "tests"
83             if not list(tests.glob("test*.py")):
84                 raise AssertionError(f"No tests found: {problem}")
85             result = command(["-m", "unittest", "discover", "-s", f"{problem}/tests", "-v"], ROOT)
86             if "Ran 0 tests" in result["output"]:
87                 raise AssertionError(f"No tests executed: {problem}")
88             return result
89
90         record(f"{problem}_tests", suite)
91
92         def outside_cwd(problem=problem):
93             with tempfile.TemporaryDirectory(prefix="cumcm_b_verify_") as temporary:
94                 return command([str(ROOT / problem / "solve.py")], Path(temporary))
95
96         record(f"{problem}_solve_from_outside_repository", outside_cwd)
97
98     def strategy_comparison():
99         with tempfile.TemporaryDirectory(prefix="cumcm_b_comparison_") as temporary:
100             return command([str(ROOT / "problem2/compare_strategies.py")], Path(temporary))
101
102     record("problem2_strategy_comparison_from_outside_repository", strategy_comparison)
103
104     def scoring_benchmark():
105         with tempfile.TemporaryDirectory(prefix="cumcm_b_scoring_") as temporary:
106             return command([str(ROOT / "problem2/benchmark_scoring.py")], Path(temporary))
107
108     record("problem2_early_stop_equivalence_from_outside_repository", scoring_benchmark)
109
110     for problem_number in (3, 4):
111         def search_smoke(problem_number=problem_number):
112             with tempfile.TemporaryDirectory(prefix="cumcm_b_search_") as temporary:

```

```

113         return command([str(ROOT / "scripts/benchmark_search.py"), "--problem",
114             ↪ str(problem_number),
115                 "--split", "stress", "--count", "8", "--strategies", "adaptive",
116                 "--tag", "verification"], Path(temporary))
117     record(f"problem{problem_number}_search_smoke_from_outside_repository", search_smoke)
118 if args.reference:
119     with tempfile.TemporaryDirectory(prefix="cumcm_b_reference_") as temporary:
120         record("independent_highs_geometry_reference",
121             lambda: command([str(ROOT / "scripts/verify_reference.py"), Path(temporary)]), Path(temporary)))
122
123 def outputs():
124     files = []
125     for problem in ("problem1", "problem2", "problem3", "problem4"):
126         results = ROOT / problem / "results"
127         if not results.is_dir() or not any(results.iterdir()):
128             raise AssertionError(f"Missing results: {problem}")
129         for path in sorted(results.rglob("*")):
130             if not path.is_file():
131                 continue
132             if path.stat().st_size == 0:
133                 raise AssertionError(f"Empty result: {path}")
134             if path.suffix == ".json":
135                 json.loads(path.read_text(encoding="utf-8"))
136             elif path.suffix == ".jsonl":
137                 for line in path.read_text(encoding="utf-8").splitlines():
138                     json.loads(line)
139             elif path.suffix == ".csv":
140                 with path.open(encoding="utf-8-sig", newline="") as stream:
141                     rows = list(csv.reader(stream))
142                     if len(rows) < 2 or not rows[0] or any(len(r) != len(rows[0]) for r in rows[1:]):
143                         raise AssertionError(f"Invalid or empty CSV table: {path}")
144             elif path.suffix == ".svg":
145                 ET.parse(path)
146             files.append({"path": path.relative_to(ROOT).as_posix(), "bytes": path.stat().st_size,
147                 "sha256": sha256(path)})
148
149     return files
150
151 record("result_files_readable", outputs)
152
153 def documentation_links():
154     count = 0
155     paths = [ROOT / "README.md", ROOT / "AGENTS.md", ROOT / "materials/problem_b/README.md",
156         ROOT / "simulation_guide.md", ROOT / "validation/search_experiments.md"]
157     for problem in ("problem1", "problem2", "problem3", "problem4"):
158         paths.extend(sorted((ROOT / problem).rglob "*.md"))
159     paths.append(ROOT / "validation/merge_notes.md")
160     for path in paths:
161         relative = path.relative_to(ROOT).as_posix()
162         content = path.read_text(encoding="utf-8")
163         for target in re.findall(r"\\[\\^\\]*\\[\\(\\[\\^\\]+\\)\\]", content):
164             if "://" in target or target.startswith("#"):
165                 continue
166             target = target.split("#", 1)[0].strip("<>")
167             if not (path.parent / target).exists():
168                 raise AssertionError(f"Broken link in {relative}: {target}")

```

```

167         count += 1
168     return {"local_links_checked": count}
169
170 def write_reports() -> bool:
171     passed = all(check["passed"] for check in checks)
172     test_counts = {}
173     for check in checks:
174         if check["name"] in tuple(f"problem{n}_tests" for n in range(1,5)) and
175             ↪ isinstance(check["detail"], dict):
176             match = re.search(r"Ran (\d+) tests?", check["detail"].get("output", ""))
177             if match:
178                 test_counts[check["name"].split("_")[0]] = int(match.group(1))
179     report = {"verified_at_utc": datetime.now(timezone.utc).isoformat(),
180             "python_version": sys.version.split()[0], "passed": passed,
181             "unit_test_counts": test_counts,
182             "scope": "Local problems 1 to 4; synthetic examples and local mock HTTP; no official
183             ↪ simulator run.",
184             "checks": checks}
185     REPORT_DIR.mkdir(exist_ok=True)
186     (REPORT_DIR / "report.json").write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n",
187             ↪ encoding="utf-8")
188     lines = ["# B题四问验证报告", "", f"总结果: {'全部通过' if passed else '存在失败, 请查看具体记录'}.",
189             ↪ "",
190             f" 验证时间 (UTC): {report['verified_at_utc']}; Python: {report['python_version']}.", "",
191             "自动测试: " + ", ".join(f"问题{n} {test_counts.get(f'problem{n}',0)} 项" for n in
192             ↪ range(1,5)) + ". ", "",
193             "| 检查项 | 结果 |", "| --- | --- |"]
194     labels = {"original_materials_and_searchable_copies": " 原材料与检索副本完整性",
195             "problem1_tests": " 第一问数学与程序测试", "problem2_tests": " 第二问数学与程序测试",
196             "problem1_solve_from_outside_repository": " 第一问从仓库外目录运行",
197             "problem2_solve_from_outside_repository": " 第二问从仓库外目录运行",
198             "problem2_strategy_comparison_from_outside_repository": " 同口径策略比较及跨目录复现",
199             "problem2_early_stop_equivalence_from_outside_repository":
200             ↪ " 提前停止的评分与选点等价性及计算量",
201             "independent_highs_geometry_reference": " 独立 HiGHS 几何核验",
202             "result_files_readable": "结果文件完整且可读取", "documentation_links": "说明文档本地链接"}
203     lines.extend(f"| {labels.get(check['name'], check['name'])} | {'通过' if check['passed'] else
204             ↪ '失败'} |" for check in checks)
205     lines.extend(["", "验证包括三份资料及三个文本副本的一致性、四问测试、从仓库以外目录启动求解与策略对比、
206             ↪ 问题三四各8例困难样本回归、结果文件可读取性和说明文档链接。",
207             "", " 算例均为自行构造, 只验证本地数学算法; 没有运行问题 3、4 的官方演练或正式测试。",
208             "", " 详细测试名称、输出、耗时、结果文件摘要见 [report.json](report.json)。",
209             "", "复现: 在仓库根运行 `python scripts/verify_project.py`。加 `--reference`
210             ↪ 可额外运行独立 SciPy/HiGHS 几何核验 (仅这一可选项需要 SciPy)。",
211             "", f"本次独立参考核验: {'已执行, 结果见上表及 reference_geometry.json' if
212             ↪ args.reference else '未请求; 已有 reference_geometry.json 不代表本次已重跑'}。"])
213     (REPORT_DIR / "report.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
214     return passed
215
216 # Generate this report before checking links pointing to it on a clean checkout.
217 write_reports()
218 record("documentation_links", documentation_links)
219 return 0 if write_reports() else 1
220
221

```

```

212 if __name__ == "__main__":
213     raise SystemExit(main())

```

程序 39: scripts/verify_reference.py

```

1  """Optional independent HiGHS check of problem 1; requires SciPy only here.
2
3  Adapted from the teammate bundle's scripts/verify_reference.py. The current
4  Fraction-based solver, rather than the archived implementation, is checked.
5  All data are deterministic synthetic half-plane systems, not simulator runs.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import json
11 import random
12 import sys
13 from collections import Counter
14 from pathlib import Path
15
16 ROOT = Path(__file__).resolve().parents[1]
17 sys.path.insert(0, str(ROOT))
18
19 from problem1.solve import HalfPlane, analyze_halfplanes
20
21
22 def verify(random_cases: int, seed: int) -> dict:
23     import scipy
24     from scipy.optimize import linprog
25
26     rng = random.Random(seed)
27     # Explicit low-dimensional cases complement generic random systems.
28     cases = [
29         ("empty", [(1, 0, 0), (-1, 0, -1)]),
30         ("halfplane", [(1, 0, 2)]),
31         ("strip", [(1, 0, 2), (-1, 0, 1)]),
32         ("line", [(1, 0, 2), (-1, 0, -2)]),
33         ("point", [(1, 0, 2), (-1, 0, -2), (0, 1, 3), (0, -1, -3)]),
34         ("segment", [(1, 0, 2), (-1, 0, -2), (0, 1, 3), (0, -1, 0)]),
35         ("triangle", [(-1, 0, 0), (0, -1, 0), (1, 1, 10)]),
36         ("rectangle", [(1, 0, 8), (-1, 0, 3), (0, 1, 7), (0, -1, 2)]),
37     ]
38     for index in range(random_cases):
39         planes = []
40         for _ in range(rng.randint(2, 12)):
41             a, b = 0, 0
42             while (a, b) == (0, 0):
43                 a, b = rng.randint(-15, 15), rng.randint(-15, 15)
44             planes.append((a, b, rng.randint(-100, 100)))
45         cases.append((f"random_{index}", planes))
46
47     directions = [(1, 0), (-1, 0), (0, 1), (0, -1),
48                  (1, 1), (-1, 1), (1, -1), (-1, -1)]
49     counts: Counter = Counter()
50     failures = []

```



```

51     supports_checked = 0
52     for name, coefficients in cases:
53         matrix = [[a, b] for a, b, _ in coefficients]
54         limits = [c for _, _, c in coefficients]
55
56         def reference(objective):
57             return linprog(objective, A_ub=matrix, b_ub=limits,
58                             bounds=[(None, None)] * 2, method="highs")
59
60         actual = analyze_halfplanes(HalfPlane(*row) for row in coefficients)
61         feasible = reference((0, 0))
62         references = []
63         if feasible.status == 2:
64             expected = "empty"
65         elif feasible.status == 0:
66             references = [reference(direction) for direction in directions]
67             if any(result.status not in (0, 3) for result in references):
68                 failures.append({"case": name, "error": "Reference support solve failed"})
69                 continue
70             expected = "unbounded" if any(result.status == 3 for result in references) else "bounded"
71         else:
72             failures.append({"case": name, "error": feasible.message})
73             continue
74         counts[expected] += 1
75         category = actual["status"] if actual["status"] in ("empty", "unbounded") else "bounded"
76         if category != expected:
77             failures.append({"case": name, "expected": expected, "actual": actual["status"]})
78             continue
79         if expected == "bounded":
80             for objective, result in zip(directions, references):
81                 value = min(objective[0] * x + objective[1] * y for x, y in actual["vertices"])
82                 supports_checked += 1
83                 if abs(value - result.fun) > 1e-7 * max(1.0, abs(result.fun)):
84                     failures.append({"case": name, "objective": objective,
85                                     "actual_support": value, "reference_support": result.fun})
86
87     return {"passed": not failures, "reference": "scipy.optimize.linprog / HiGHS",
88            "scipy_version": scipy.__version__, "seed": seed,
89            "random_halfplane_systems": random_cases, "explicit_edge_cases": 8,
90            "classification_counts": dict(counts), "bounded_support_values_checked": supports_checked,
91            "failures": failures,
92            "scope": "Deterministic synthetic geometry; no official simulator. HiGHS has floating
93                    ↳ tolerances; extreme near-parallel cases remain in the exact solver's own regression
94                    ↳ suite."}
95
96 def main() -> int:
97     parser = argparse.ArgumentParser(description=__doc__)
98     parser.add_argument("--cases", type=int, default=500)
99     parser.add_argument("--seed", type=int, default=611209)
100    parser.add_argument("--output", type=Path, default=Path("validation/reference_geometry.json"))
101    args = parser.parse_args()
102    if args.cases < 1:
103        parser.error("--cases must be positive")
104    try:

```

```

104         result = verify(args.cases, args.seed)
105     except ImportError:
106         parser.error("This optional check requires SciPy; the solvers and standard validation do not.")
107     output = args.output if args.output.is_absolute() else ROOT / args.output
108     output.parent.mkdir(parents=True, exist_ok=True)
109     output.write_text(json.dumps(result, ensure_ascii=False, indent=2, allow_nan=False) + "\n",
110                       ↪ encoding="utf-8")
111     print(json.dumps(result, ensure_ascii=False, indent=2))
112     return 0 if result["passed"] else 1
113
114 if __name__ == "__main__":
115     raise SystemExit(main())

```

程序 40: paper/plot_geometry.py

```

1  """Rebuild two compact paper figures from the saved, self-created P1/P2 results.
2
3  Run with Python and matplotlib; input paths are relative to this file.
4  Optional --preview-dir writes PNGs for local visual review. No solver results
5  are overwritten and no new measurements or online requests are generated.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import csv
11 import json
12 import math
13 from pathlib import Path
14 import sys
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 from matplotlib.lines import Line2D
21 from matplotlib.patches import Circle, Patch, Polygon
22 import numpy as np
23
24 PAPER = Path(__file__).resolve().parent
25 ROOT = PAPER.parent
26 sys.path.insert(0, str(ROOT))
27 from problem2.solve import first_region_outer_local, safe_radial_limit
28
29 BLUE = "#27647b"
30 GREY = "#737373"
31 INK = "#202020"
32
33
34 def read_json(path: Path) -> dict:
35     return json.loads(path.read_text(encoding="utf-8"))
36
37
38 def style() -> None:
39     plt.rcParams.update({

```

```

40     "font.family": "DejaVu Serif",
41     "font.size": 9,
42     "axes.labelsize": 9,
43     "axes.titlesize": 9.5,
44     "xtick.labelsize": 8,
45     "ytick.labelsize": 8,
46     "legend.fontsize": 8.3,
47     "mathtext.fontset": "dejavuserif",
48     "pdf.fonttype": 42,
49     "ps.fonttype": 42,
50     "axes.linewidth": 0.6,
51     "lines.linewidth": 1.1,
52     "savefig.facecolor": "white",
53 })
54
55
56 def finish(fig: plt.Figure, filename: str, preview_dir: Path | None) -> None:
57     output = PAPER / "figures"
58     output.mkdir(parents=True, exist_ok=True)
59     fig.savefig(output / f"{filename}.pdf", bbox_inches="tight", pad_inches=0.04,
60                 metadata={"Title": filename, "Author": "", "Subject": "Self-created mathematical
61                 ↪ examples"})
62     if preview_dir is not None:
63         preview_dir.mkdir(parents=True, exist_ok=True)
64         fig.savefig(preview_dir / f"{filename}.png", dpi=180, bbox_inches="tight", pad_inches=0.04)
65     plt.close(fig)
66
67 def triangle_figure(preview_dir: Path | None) -> None:
68     data = read_json(ROOT / "problem1/results/triangle_optical_counterexample.result.json")
69     vertices = np.array(data["vertices"])
70     endpoints = np.array(data["diameter_endpoints"])
71     disk_d = data["diameter_circle"]
72     disk_e = data["minimum_enclosing_circle"]
73     fig = plt.figure(figsize=(6.4, 3.15))
74     ax = fig.add_axes((0.075, 0.18, 0.53, 0.76))
75     ax.add_patch(Polygon(vertices, closed=True, facecolor="#e8e8e8", edgecolor=INK, linewidth=1.2))
76     ax.add_patch(Circle(disk_d["center"], disk_d["radius_m"], fill=False,
77                         edgecolor=BLUE, linewidth=1.4, linestyle=(0, (4, 2))))
78     ax.add_patch(Circle(disk_e["center"], disk_e["radius_m"], fill=False,
79                         edgecolor=INK, linewidth=1.35))
80     ax.plot(*endpoints.T, color=BLUE, linewidth=1.9)
81     ax.scatter(*vertices.T, s=20, color=INK, zorder=5)
82     ax.scatter(*disk_d["center"], marker="+", s=62, linewidth=1.3, color=BLUE, zorder=6)
83     ax.scatter(*disk_e["center"], marker="x", s=32, linewidth=1.1, color=INK, zorder=6)
84     for text, xy, offset in (("A", vertices[0], (-12, -8)),
85                             ("B", vertices[1], (5, -4)),
86                             ("C", vertices[2], (4, 6))):
87         ax.annotate(text, xy=xy, xytext=offset, textcoords="offset points", fontsize=9)
88     ax.set(xlim=(-13, 49), ylim=(-14, 42), xlabel="$x$ (m)", ylabel="$y$ (m)")
89     ax.set_aspect("equal", adjustable="box")
90     ax.set_xticks([0, 20, 40])
91     ax.set_yticks([0, 20, 40])
92     ax.grid(color="#dedede", linewidth=0.45)
93     ax.set_axisbelow(True)

```

```

94
95     legend_handles = [
96         Patch(facecolor="#e8e8e8", edgecolor=INK, label="Bearing-feasible region"),
97         Line2D([0], [0], color=BLUE, lw=1.4, ls=(0, (4, 2)), label="Diameter circle"),
98         Line2D([0], [0], color=INK, lw=1.3, label="Minimum enclosing circle"),
99     ]
100     fig.legend(handles=legend_handles, loc="upper left", bbox_to_anchor=(0.62, 0.93),
101               frameon=False, handlelength=2.4, labelspacing=1.2)
102     fig.text(0.65, 0.52, "$D=40$ m\n$D/2=20$ m\n$R_*=23.0940$ m", va="top", linespacing=1.65, fontsize=10)
103     fig.text(0.65, 0.24, "$B$ lies outside the\ndiameter circle.", fontsize=9, linespacing=1.4)
104     fig.text(0.08, 0.02, r"Constructed from three legal $\pm\circ$ bearing observations.", fontsize=8.5)
105     finish(fig, "p12_geometry", preview_dir)
106
107
108 def candidate_rows(folder: Path, threshold: float) -> np.ndarray:
109     with (folder / "second_point_candidates.csv").open(encoding="utf-8-sig", newline="") as stream:
110         rows = list(csv.DictReader(stream))
111         # Explicitly filter even if the saved shortlist already obeys this limit.
112         return np.array([(float(row["local_a_m"]), float(row["local_b_m"]))
113                          for row in rows if float(row["diameter_upper_bound_m"]) <= threshold + 1e-7])
114
115
116 def candidates_figure(preview_dir: Path | None) -> None:
117     fig, axes = plt.subplots(1, 2, figsize=(7.1, 4.5))
118     fig.subplots_adjust(left=0.075, right=0.985, top=0.9, bottom=0.3, wspace=0.24)
119     beta = np.linspace(-89, 89, 1601)
120     radii = np.array([safe_radial_limit(float(b)) for b in beta])
121     safe_boundary = np.column_stack((radii * np.cos(np.radians(beta)), radii * np.sin(np.radians(beta))))
122     folders = [ROOT / "problem2/results", ROOT / "problem2/results/boundary"]
123     for index, (ax, folder) in enumerate(zip(axes, folders)):
124         plan = read_json(folder / "selection.json")
125         station = tuple(plan["first_station_xy_m"])
126         theta = plan["first_bearing_deg"]
127         prior = np.array(first_region_outer_local(station, theta, plan["error_deg"], plan["circle_sides"]))
128         near_best = candidate_rows(folder, plan["candidate_threshold_m"])
129         selected = plan["selected"]["local_xy_m"]
130         score = plan["selected"]["diameter_upper_bound_m"]
131         ax.add_patch(Polygon(safe_boundary, closed=True, facecolor="#e8e8e8", edgecolor=INK,
132                             ↪ linewidth=0.9, zorder=1))
133         ax.add_patch(Polygon(prior, closed=True, facecolor=BLUE, edgecolor=BLUE, linewidth=1.15,
134                             ↪ alpha=0.8, zorder=3))
135         ax.scatter(*near_best.T, marker=".", s=6, color="#888888", alpha=0.6, linewidth=0, zorder=4)
136         ax.plot([0, selected[0]], [0, selected[1]], color=INK, lw=0.75, zorder=5)
137         ax.scatter(*selected, marker="*", s=100, color=INK, edgecolor="white", linewidth=0.5, zorder=6)
138         ax.scatter(0, 0, marker="o", s=15, color=INK, zorder=6)
139         ax.annotate("$S_1$", (0, 0), (-14, -15), textcoords="offset points")
140         ax.annotate("$q_*$", selected, (8, 8), textcoords="offset points", fontsize=10)
141         ax.text(780, -690, r"$\mathcal{C}_{\rm safe}$", ha="center", fontsize=11)
142         prior_tip = (1150, 0) if index == 0 else (160, 0)
143         prior_label = (1250, -300) if index == 0 else (720, -325)
144         ax.annotate(r"$\overline{U}_1$", xy=prior_tip, xytext=prior_label,
145                   arrowprops={"arrowstyle": "->", "lw": 0.75}, fontsize=11, ha="center")
146         if index == 1:
147             # This is a source-position constraint; the robot-safe set is
148             # intentionally drawn on both sides of the arc.

```

```

147         ax.add_patch(Circle((-1500, 0), 1800, fill=False, edgecolor=GREY,
148                             linewidth=0.9, linestyle=(0, (4, 3)), zorder=2))
149         ax.annotate("Source target\nboundary", xy=(110, 800), xytext=(830, 785),
150                     arrowprops={"arrowstyle": "->", "lw": 0.65, "color": GREY},
151                     fontsize=8, va="center", color="#505050")
152         ax.set(xlim=(-100, 1570), ylim=(-950, 950), xlabel="$a$ (m)", ylabel="$b$ (m)")
153         ax.set_aspect("equal", adjustable="box")
154         ax.set_xticks([0, 500, 1000, 1500])
155         ax.set_yticks([-750, 0, 750])
156         ax.grid(color="#ddddd", linewidth=0.4)
157         ax.set_axisbelow(True)
158         ax.set_title(f"({chr(97 + index)}) $S_1=({int(station[0])},0)$ m, $\theta_1=0^\circ$, pad=9)
159         ax.text(0.5, -0.19, fr"$q_*=({selected[0]:.1f},\{selected[1]:.1f})$ m" + f"\nC(q_*)={score:.4f}$
        ↪ m",
160                 transform=ax.transAxes, ha="center", va="top", fontsize=8.8, linespacing=1.5)
161         fig.legend(handles=[Patch(facecolor="#eeeeee", edgecolor=INK, label="Four-disk safe set"),
162                             Patch(facecolor=BLUE, edgecolor=BLUE, label="Source prior outer bound"),
163                             Line2D([0], [0], marker=".", color="none", markerfacecolor="#888888",
        ↪ markersize=6, label="Visited points within 10%")],
164                 loc="lower center", bbox_to_anchor=(0.5, 0.005), ncol=3, frameon=False,
165                 columnspacing=1.4, handlelength=1.5)
166         finish(fig, "p2_candidates", preview_dir)
167
168
169 def main() -> None:
170     parser = argparse.ArgumentParser(description=__doc__)
171     parser.add_argument("--preview-dir", type=Path)
172     args = parser.parse_args()
173     style()
174     triangle_figure(args.preview_dir)
175     candidates_figure(args.preview_dir)
176
177
178 if __name__ == "__main__":
179     main()

```

程序 41: paper/plot_search_results.py

```

1  """ 依据已保存的结果重建论文搜索图；不运行算法或连接测试平台。"""
2  from __future__ import annotations
3
4  import json
5  import math
6  from pathlib import Path
7
8  import matplotlib
9
10 matplotlib.use("Agg")
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager
13 from matplotlib.patches import Circle
14
15
16 PAPER = Path(__file__).resolve().parent
17 ROOT = PAPER.parent

```

```

18 DEST = PAPER / "figures"
19
20
21 def read_result(relative: str):
22     return json.loads((ROOT / relative).read_text(encoding="utf-8"))
23
24
25 def setup_style():
26     for candidate in (
27         Path("C:/Windows/Fonts/simhei.ttf"),
28         Path("/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"),
29     ):
30         if candidate.exists():
31             font_manager.fontManager.addfont(str(candidate))
32             family = font_manager.FontProperties(fname=str(candidate)).get_name()
33             break
34     else:
35         family = "DejaVu Sans"
36     plt.rcParams.update(
37         {
38             "font.family": family,
39             "font.size": 8,
40             "axes.titlesize": 9,
41             "axes.labelsize": 8,
42             "xtick.labelsize": 7,
43             "ytick.labelsize": 7,
44             "legend.fontsize": 7,
45             "axes.unicode_minus": False,
46             "pdf.fonttype": 42,
47             "ps.fonttype": 42,
48             "axes.linewidth": 0.6,
49             "savefig.facecolor": "white",
50         }
51     )
52
53
54 def coverage_figure():
55     """P4 坐标直接读取当前冻结的解析覆盖证书。"""
56     certificate = read_result("problem4/results/coverage_certificate_v2.json")
57     if not certificate["certified"] or certificate["point_count"] != 23:
58         raise ValueError(" 第四问论文图要求通过认证的 23 点冻结布局")
59     a = (read_result("problem3/results/benchmark_development.json")["config"] or {}).get(
60         "ring_radius", 1150.0
61     )
62     points3 = [(0, 0)] + [
63         (a * math.cos(k * math.pi / 3), a * math.sin(k * math.pi / 3))
64         for k in range(6)
65     ]
66     points4 = certificate["points"]
67     fig, axes = plt.subplots(1, 2, figsize=(6.4, 3.6))
68     for ax in axes:
69         ax.add_patch(Circle((0, 0), 1800, fill=False, color="#222222", lw=1.1, zorder=3))
70         ax.axhline(0, color="#dddddd", lw=0.5, zorder=0)
71         ax.axvline(0, color="#dddddd", lw=0.5, zorder=0)
72         ax.set(xlim=(-2400, 2400), ylim=(-2400, 2400), xlabel="$x$ / 米", ylabel="$y$ / 米")

```

```

73     ax.set_xticks([-2000, -1000, 0, 1000, 2000])
74     ax.set_yticks([-2000, -1000, 0, 1000, 2000])
75     ax.set_aspect("equal")
76     for point in points3:
77         axes[0].add_patch(Circle(point, 1000, facecolor="#2670a6", edgecolor="#2670a6", alpha=0.10,
78             ↪ lw=0.65))
79     axes[0].scatter(*zip(*points3), s=16, c="#125986", zorder=4)
80     axes[0].set_title("(a) 第三问: 七点全向覆盖", pad=9)
81     axes[1].scatter(*zip(*([points4[0]] + points4[13:])), s=16, c="#125986", zorder=4, label=" 原点及内点")
82     axes[1].set_title("(b) 第四问: 23 点认证布局", pad=9)
83     handles, labels = axes[1].get_legend_handles_labels()
84     fig.legend(handles, labels, loc="center", bbox_to_anchor=(0.76, 0.10), ncol=2, frameon=False,
85         ↪ handletextpad=0.3, columnspacing=1.1)
86     fig.text(0.285, 0.10, " 浅色圆盘: 保证接收半径 1000 米", ha="center", va="center", fontsize=7)
87     fig.text(0.5, 0.03, "黑色圆为1800米目标边界: 第四问任意发射朝向的覆盖由正文有限极值证书保证。",
88         ↪ ha="center", fontsize=7)
89     fig.subplots_adjust(left=0.10, right=0.98, bottom=0.24, top=0.91, wspace=0.32)
90     fig.savefig(DEST / "search_coverage.pdf", metadata={"Title": "第三、四问的冻结覆盖布局", "Subject":
91         ↪ "来源: P3生产配置; P4 coverage_certificate_v2.json"})
92     plt.close(fig)
93
94 def label_bars(ax, bars, percentage=False):
95     for bar in bars:
96         value = bar.get_height()
97         text = f"{value:.1f}%" if percentage else f"{value:.1f}"
98         ax.annotate(text, (bar.get_x() + bar.get_width() / 2, value), xytext=(0, 3), textcoords="offset
99             ↪ points", ha="center", va="bottom", fontsize=7)
100
101 def comparison_figure():
102     """ 各条形均来自现有冻结结果, 不将不同样本集合并评分。"""
103     p3 = read_result("problem3/results/benchmark_development.json")["summary"]
104     p4 = read_result("problem4/results/speed_comparison_v2.json")["paired"]
105     fig, axes = plt.subplots(2, 2, figsize=(6.4, 4.85))
106     names3 = ["先覆盖后清除", " 本文策略", " 直接光学"]
107     strategy_keys = ["baseline", "adaptive", "optical"]
108     colors3 = ["#8d959e", "#2670a6", "#cc8451"]
109     for col, field in enumerate(["mean_average_clear_time_s", "threshold_pass_rate"]):
110         values = [p3[key][field] * (100 if col else 1) for key in strategy_keys]
111         bars = axes[0, col].bar(range(3), values, color=colors3, width=0.57, edgecolor="#333333",
112             ↪ linewidth=0.4)
113         for bar, hatch in zip(bars, ["/", "", ""]):
114             bar.set_hatch(hatch)
115         label_bars(axes[0, col], bars, percentage=bool(col))
116         axes[0, col].set_xticks(range(3), names3)
117     axes[0, 0].set(title="(a) 第三问: 同一开发集 96 例", ylabel=" 平均时间 / (秒/源)", ylim=(0, 720))
118     axes[0, 1].set(title="(b) 第三问: 300 秒/源阈值", ylabel=" 全清且达到阈值 / %", ylim=(0, 100))
119     keys4 = ["development_v2", "holdout_v2", "stress_v2"]
120     labels4 = ["开发\n384 例", " 留出\n512 例", " 压力\n256 例"]
121     for col, pair in enumerate(
122         [("old_seconds_per_source", "new_seconds_per_source"), ("old_threshold_pass_rate",
123             ↪ "new_threshold_pass_rate")]):
124         axes[1, col].set(title=f"{pair[0]} vs {pair[1]}",
125             ylabel=f"{pair[0]} / {pair[1]}",
126             xlim=(0, len(keys4)),
127             ylim=(0, 100))
128         axes[1, col].set_xticks(range(len(keys4)), labels4)
129         axes[1, col].set_yticks([0, 50, 100])
130         axes[1, col].set_yticklabels(["", "50%", "100%"])
131         for key, color in zip(keys4, colors3):
132             axes[1, col].plot(key, pair[0], color=color, lw=2, ls="solid")
133             axes[1, col].plot(key, pair[1], color=color, lw=2, ls="dashed")
134         axes[1, col].legend()
135     fig.tight_layout()
136     fig.savefig(DEST / "comparison.pdf", metadata={"Title": "第三、四问的冻结覆盖布局", "Subject":
137         ↪ "来源: P3生产配置; P4 coverage_certificate_v2.json"})
138     plt.close(fig)

```

```

121         for index, (field, color, offset, legend, hatch) in enumerate(
122             [(pair[0], "#a3a8ad", -0.19, "旧版25点", "/"), (pair[1], "#2670a6", 0.19, "新版23点", "")]
123         ):
124             values = [p4[key][field] * (100 if col else 1) for key in keys4]
125             bars = axes[1, col].bar([x + offset for x in range(3)], values, width=0.34, color=color,
126                                     ↪ edgecolor="#333333", linewidth=0.4, label=legend, hatch=hatch)
127             label_bars(axes[1, col], bars, percentage=bool(col))
128             axes[1, col].set_xticks(range(3), labels4)
129 axes[1, 0].set(title="(c) 第四问: 集合内新旧配对", ylabel=" 平均时间 / (秒/源)", ylim=(0, 740))
130 axes[1, 1].set(title="(d) 第四问: 500 秒/源阈值", ylabel=" 全清且达到阈值 / %", ylim=(0, 100))
131 for ax in axes.flat:
132     ax.spines[["top", "right"]].set_visible(False)
133     ax.grid(axis="y", linewidth=0.4, color="#dddddd", zorder=0)
134     ax.set_axisbelow(True)
135 handles, labels = axes[1, 0].get_legend_handles_labels()
136 fig.legend(handles, labels, loc="lower center", bbox_to_anchor=(0.5, 0.055), ncol=2, frameon=False)
137 fig.text(0.5, 0.023, "全部为自建样本; 每例先计算总虚拟时间/清除数, 再取均值。阈值为本地预设标准。",
138         ↪ ha="center", fontsize=7)
139 fig.subplots_adjust(left=0.10, right=0.98, top=0.93, bottom=0.18, hspace=0.63, wspace=0.31)
140 fig.savefig(DEST / "search_comparison.pdf", metadata={"Title": "第三、四问本地冻结算法对照", "Subject":
141             ↪ "P3同一96例开发集; P4 v2各集合内逐例配对"})
142 plt.close(fig)
143
144
145 def main():
146     DEST.mkdir(parents=True, exist_ok=True)
147     setup_style()
148     coverage_figure()
149     comparison_figure()
150     print(f" 已根据现有结果生成两张论文图: {DEST}")
151
152
153 if __name__ == "__main__":
154     main()

```

程序 42: paper/build_paper.py

```

1  """Assemble the manuscript and freeze its reproducible source-code appendix."""
2  from __future__ import annotations
3
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8
9  PAPER = Path(__file__).resolve().parent
10 ROOT = PAPER.parent
11
12
13 def figure(name: str, caption: str, label: str, width: str = '.88') -> str:
14     return (r'\begin{figure}[htbp]' + '\n' + r'\centering' + '\n'
15           + rf'\includegraphics[width={width}\textwidth]{{{name}.pdf}}' + '\n'
16           + rf'\caption{{{caption}}}\label{{{label}}}' + '\n'
17           + r'\end{figure}' + '\n')
18

```



```

19
20 def assemble() -> None:
21     sec = PAPER / 'sections'
22     front = (sec / 'front.tex').read_text(encoding='utf-8')
23     p12 = (sec / 'p12.tex').read_text(encoding='utf-8')
24     p34 = (sec / 'p34.tex').read_text(encoding='utf-8')
25     method, checks = p34.split('% 以下片段建议放入“模型的检验”章节。', 1)
26     p12 = p12.replace(r'\subsection{问题二：保证收信的第二检测点选择}',
27         figure('p12_geometry', '合法示向观测形成的三角形及两类覆盖圆', 'fig:triangles')
28         + r'\subsection{问题二：保证收信的第二检测点选择}')
29     p12 += figure('p2_candidates', r'第二检测点安全区域与目标圆条件化选点（灰点为已访问的10\%近优代表点）',
30         ↪ 'fig:p2')
31     method = method.replace('将圆缺按 $V_{\ell}^+$ 的 Voronoi 区域划分。',
32         '将圆缺按 $V_{\ell}^+$ 的 Voronoi 区域划分\cite{deberg2008}。')
33     text = front + '\n' + p12 + '\n' + method
34     text += '\n' + r'\section{模型的检验}\label{sec:6}' + '\n' + checks
35     text += '\n' + (sec / 'discussion.tex').read_text(encoding='utf-8')
36     (PAPER / 'main.tex').write_text(text, encoding='utf-8', newline='\n')
37
38 def freeze_code() -> list[str]:
39     paths = []
40     for folder in ('problem1', 'problem2', 'problem3', 'problem4'):
41         paths += sorted((ROOT / folder).glob('*.*py'))
42         paths += sorted((ROOT / folder / 'tests').glob('*.*py'))
43     paths += [p for p in sorted((ROOT / 'scripts').glob('*.*py')) if p.name != 'experiment_search.py']
44     paths += [PAPER / 'plot_geometry.py', PAPER / 'plot_search_results.py', PAPER / 'build_paper.py',
45         ↪ PAPER / 'package_support.py']
46     files = []
47     hashes = []
48     for source in paths:
49         if not source.exists():
50             raise FileNotFoundError(source)
51         name = source.relative_to(ROOT).as_posix()
52         target = PAPER / 'code' / name
53         target.parent.mkdir(parents=True, exist_ok=True)
54         shutil.copy2(source, target)
55         data = target.read_bytes()
56         hashes.append({'path': name, 'bytes': len(data), 'sha256': hashlib.sha256(data).hexdigest()})
57         files.append(name)
58     (PAPER / 'source_snapshot.json').write_text(json.dumps(hashes, ensure_ascii=False, indent=2),
59         ↪ encoding='utf-8')
60
61 source = r'''\documentclass[a4paper,10pt,UTF8,fontset=none]{ctexart}
62 \usepackage[left=2.5cm,right=2.5cm,top=2.5cm,bottom=2.5cm]{geometry}
63 \setCJKmainfont[Path=fonts/,FontIndex=0,AutoFakeBold=true]{simsun.ttc}
64 \setCJKsansfont{FandolHei-Regular.otf}
65 \setCJKmonofont[Path=fonts/,FontIndex=0,AutoFakeBold=true]{simsun.ttc}
66 \xeCJKDeclareCharClass{CJK}{"2190 -> "22FF}
67 \usepackage{fontspec}
68 \setmonofont{Latin Modern Mono}
69 \usepackage{fvextra}
70 \usepackage[hidelinks]{hyperref}
71 \hypersetup{pdftitle={完整程序附录},pdfauthor={}}
72 \pagestyle{empty}
73 \setlength{\parindent}{0pt}

```

```

71 \begin{document}
72 '''
73     for index, name in enumerate(files, 1):
74         source += r'\subsection*{程序 ' + str(index) + r': \texttt{\detokenize{' + name + '}}}\n'
75         source += rf'\VerbatimInput[breaklines=true,breakanywhere=true,fontsize=\fontsize{{8}}{{10}}\selectfont,numbers=left,numbersep=5pt]{{{code}/{name}}}' + '\n'
76     source += '\\end{document}\n'
77     (PAPER / 'code_appendix.tex').write_text(source, encoding='utf-8', newline='\n')
78     return files
79
80
81 if __name__ == '__main__':
82     assemble()
83     files = freeze_code()
84     print(f'Assembled main.tex and froze {len(files)} Python files.')

```

程序 43: paper/package_support.py

```

1 """Package only the sources and evidence used by the current manuscript."""
2 from __future__ import annotations
3
4 import csv
5 import hashlib
6 import io
7 import json
8 import re
9 from pathlib import Path
10 import zipfile
11
12 PAPER = Path(__file__).resolve().parent
13 ROOT = PAPER.parent
14
15
16 def selected_files() -> dict[str, Path]:
17     selected = {}
18     for item in json.loads((PAPER / 'source_snapshot.json').read_text(encoding='utf-8')):
19         selected[item['path']] = PAPER / 'code' / item['path']
20     for folder in ('problem1', 'problem2'):
21         for sub in ('examples', 'results'):
22             for path in (ROOT / folder / sub).rglob('*'):
23                 if path.is_file() and path.suffix.lower() in {'.json', '.csv', '.md', '.svg', '.txt'}:
24                     selected[path.relative_to(ROOT).as_posix()] = path
25     for folder in ('problem3', 'problem4'):
26         base = ROOT / folder
27         for path in (base / 'examples').glob('*.json'):
28             if 'v3' not in path.name and 'threshold_lower_bound' not in path.name:
29                 selected[path.relative_to(ROOT).as_posix()] = path
30     for pattern in ('benchmark*.json', 'benchmark*.md', 'speed_comparison_v2.*',
31                    'coverage_certificate_v2.json', 'stress_verification.*', 'demo_result.json'):
32         for path in (base / 'results').glob(pattern):
33             selected[path.relative_to(ROOT).as_posix()] = path
34     for path in (base / 'results' / 'online').glob('*'):
35         if path.suffix in ('.json', '.jsonl'):
36             selected[path.relative_to(ROOT).as_posix()] = path
37     for folder in ('problem1', 'problem2', 'problem3', 'problem4'):

```

```

37     for name in ('README.md', 'model.md', 'references.md'):
38         path = ROOT / folder / name
39         if path.exists():
40             selected[path.relative_to(ROOT).as_posix()] = path
41 for name in ('report.md', 'report.json', 'reference_geometry.json', 'problem4_freeze_v2.json',
42 ↪ 'search_experiments.md', 'merge_notes.md'):
43     path = ROOT / 'validation' / name
44     if path.exists():
45         selected[path.relative_to(ROOT).as_posix()] = path
46 for name in ('simulation_guide.md', 'requirements.txt'):
47     selected[name] = ROOT / name
48 for path in (ROOT / 'materials' / 'problem_b').rglob('*'):
49     if path.is_file():
50         selected[path.relative_to(ROOT).as_posix()] = path
51 for folder in ('sources', 'sections', 'figures'):
52     for path in (PAPER / folder).glob('*'):
53         if path.is_file():
54             selected[path.relative_to(ROOT).as_posix()] = path
55 for name in ('main.tex', 'JXUSTmodeling.cls', 'LICENSE', 'code_appendix.tex', 'ai_usage.tex',
56 ↪ 'source_snapshot.json', 'README.md'):
57     selected['paper/' + name] = PAPER / name
58 selected['AI 工具使用详情.pdf'] = PAPER / 'results' / 'AI 工具使用详情.pdf'
59 return selected
60
61 def main() -> None:
62     items = selected_files()
63     rows = []
64     blobs = {}
65     for name, source in sorted(items.items()):
66         data = source.read_bytes()
67         # Remove machine-local paths from textual report copies only. Original
68         # code, inputs and repository reports are left untouched.
69         if source.suffix in ('.md', '.json', '.jsonl', '.csv') and not name.endswith('.py'):
70             text = data.decode('utf-8-sig')
71             for prefix in (str(ROOT), str(ROOT).replace('\\', '/'), str(ROOT).replace('\\', '\\\\')):
72                 text = text.replace(prefix, '<PROJECT>')
73                 text = re.sub(r'[A-Za-z]:[/\\]+Users[/\\]+[^\s"<>]+' + '<USER>', text)
74                 text = re.sub(r'("robot_id"\s*:\s*)"[^"]*"', r'\1"ANONYMOUS"', text)
75             data = text.encode('utf-8')
76             rows.append([name, len(data), hashlib.sha256(data).hexdigest()])
77             blobs[name] = data
78 manifest = io.StringIO(newline='')
79 writer = csv.writer(manifest)
80 writer.writerow(['relative_path', 'bytes', 'sha256'])
81 writer.writerows(rows)
82 blobs['manifest.csv'] = manifest.getvalue().encode('utf-8-sig')
83 blobs['README.md'] = ('# 支撑材料说明\n\n 这是 2026 年 9 月 11 日论文工作稿的可复现支撑包。'
84     '保留相对目录，用 Python 3.12 运行各问题 solve.py；统一检查入口为 scripts/verify_project.py。'
85     'matplotlib 用于绘图，pypdf 用于资料检查；可选参考验需要 SciPy。'
86     '六次正式测试成绩及原名加密日志尚未纳入；本包不是已完成正式提交的证明。'
87     '演练来源说明见 paper/sources/p34_evidence.md。'
88     '报告副本中的机器绝对路径已替换为通用标记，原仓库文件未修改。'
89     '源代码保留实际执行版本，文本报告均为本地结果。').encode('utf-8')
90 output = PAPER / 'results' / 'supporting_materials.zip'

```

```
90     with zipfile.ZipFile(output, 'w', compression=zipfile.ZIP_DEFLATED, compresslevel=9) as archive:
91         for name, data in sorted(blobs.items()):
92             archive.writestr(name, data)
93     if output.stat().st_size > 20_000_000:
94         raise RuntimeError('Supporting material exceeds 20 MB.')
95     print(f'{len(blobs)} files; {output.stat().st_size} bytes; {output}')
96
97
98 if __name__ == '__main__':
99     main()
```